

文档编号	TY-SUM-20231030A
版 本	A
页 数	102 页

系统建模仿真环境 APP 开发用户手册

苏州同元软控信息技术有限公司版权所有，未经许可不得复制、传播或以其他方式使用；
版权所有，侵权必究。

科学计算与系统建模仿真项目组

编制	王天飞、周雯、王越洲	生效日期	2023-10-30
审核	郭俊峰	批准	刘奇

文件变更摘要

日期	版本	变更说明	修订	审核	批准
2023/3/17	V0.1	初始建立	王天飞	郭俊峰	刘奇
2023/4/15	V0.5	补充车辆 APP 案例和批量仿真 APP 案例	王越州	郭俊峰	刘奇
2023/5/19	V0.7	修改大纲结构,并补充内容,主要补充 第 一 章 节 概 述 , 和 MwGraphicsView 机制、修改完整 APP 测试	周雯	郭俊峰	刘奇
2023/10/30	V1.0	根据测试意见修改	周雯	郭俊峰	刘奇

苏州同元软控信息技术有限公司版权所有，未经许可不得复制、传播或以其他方式使用；
版权所有，侵权必究。

目 录

1. 概述	1
1.1 MWORKS.SDK 简介	1
1.2 什么是 APP	2
1.3 APP 的开发运行模式	2
1.3.1 运行模式	2
1.4 开发流程	3
1.4.1 APP 开发环境部署	4
1.4.2 APP 开发	12
1.4.3 APP 测试	13
1.4.4 APP 打包	14
1.4.5 APP 安装	17
1.4.6 APP 使用	17
2. 如何快速开发 APP	17
2.1 质量弹簧阻尼 APP 介绍	17
2.2 模型搭建	20
2.3 开发工程构建	21
2.4 App 界面及功能详解	28
2.4.1 main 函数	28
2.4.2 QMainWindow 界面	29
2.4.3 主要功能详解	32
2.5 APP 运行及打包	36
2.6 案例所用类和接口汇总	38
3. 综合案例介绍	39
3.1 Python 版 APP	错误!未定义书签。
3.2 C++版 App	39
3.2.1 车辆仿真 APP	39
3.2.2 批量仿真 APP	62
4. 专题说明	75
4.1 Qt 界面开发教程	75
4.2 开发工程构建	83
4.3 内置案例库	90

4.3.1 MassSpringDamperApp.....	91
4.3.2 MwBatchSimPlugin.....	91
4.3.3 MwVehicleApp	92
4.3.4 Step Forward Demo	92
4.3.5 Component Parameter Panel Demo	94
4.3.6 GraphicsViewDropEvent	96
4.4 License 配置	98
4.5 MwGraphicsView 响应机制	99
4.5.1 模块结构.....	99
4.5.2 运行机制.....	101
5. 常见问题	102

苏州同元软控信息技术有限公司版权所有，未经许可不得复制、传播或以其他方式使用；
版权所有，侵权必究。

1. 概述

1.1 MWORKS.SDK 简介

MWORKS.Sysplorer 是新一代多领域工程系统建模、仿真、分析与优化通用 CAE 平台，基于多领域统一建模规范 Modelica，提供了从可视化建模、仿真计算到结果分析的完整功能，支持多学科多目标优化、硬件在环(Hardware-In-the-Loop, HIL) 仿真以及与其他工具的联合仿真。利用现有大量可重用的 Modelica 领域库，MWORKS.Sysplorer 可以广泛地满足机械、电子、控制、液压、气压、热力学、电磁等专业，以及航空、航天、车辆、船舶、能源等行业的知识积累、建模仿真与设计优化需求。MWORKS.Sysplorer 作为多领域工程系统研发平台，能够使不同的领域专家与企业工程师在统一的开发环境中对复杂工程系统进行多领域协同开发、试验和分析。

MWORKS.SDK 是基于 Qt 和 C++ 开发的，具备 MWORKS.Sysplorer 全部功能模块的 API，支持用户对 MWORKS.Sysplorer 进行功能扩展或专业 APP 开发。

MWORKS.Sysplorer 作为 Modelica IDE，本身即采用 MWORKS SDK 开发的，它既是 MWORKS.SDK 的生成环境，也是其自身的第一用户。

同时，MWORKS.SDK 已在航空、航天、汽车、核能等多个领域的专业设计仿真 APP 中得以应用和验证。

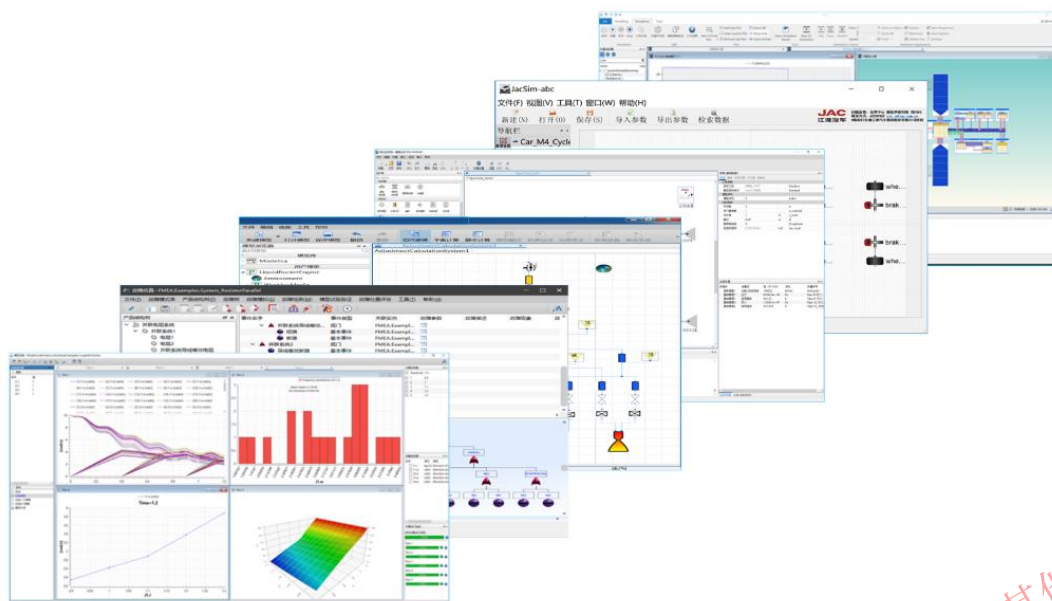


图 1-1 SDK 相关应用

1.2 什么是 APP

APP 是带交互界面的应用程序，提供面向特定场景的专业应用，APP 依赖 Sysplorer 的 API 集成 Sysplorer 的建模、编译、仿真等能力，并将软件打包为一个独立可运行的软件，可脱离 Sysplorer 进行使用也可嵌入 Sysplorer 作为插件在 Sysplorer 软件中直接打开，以达到扩展 Sysplorer 功能的目的。

1.3 APP 的开发运行模式

1.3.1 运行模式

MWORKS SDK 提供了多种模型相关操作 API，并提供相关 Qt 图形界面供用户使用，用户利用 C++/Qt 图形应用开发平台来开发 APP，可实现一个带界面交互操作的、专业设计的仿真类型 APP。

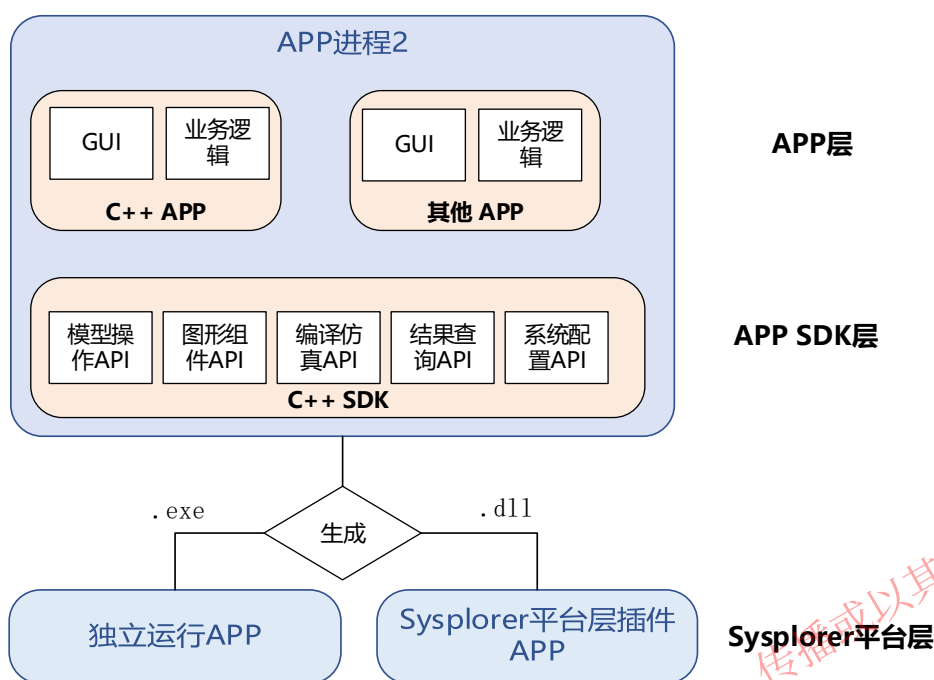


图 1-2 SDK 相关应用

APP 层：负责开发 GUI 图形用户界面和 APP 的业务逻辑。用户可以使用主流的图形应用开发平台（PyQt、C++/Qt、JavaScript 等）来开发 APP，并通过使用 APP SDK 来实现与 Syslab 平台集成和通信。

APP SDK 层：负责提供模型模型文件、参数操作、属性获取、元素及属性判定、属性查找、编译仿真、结果数据查询、图形组件类和系统配置共 9 类 API 供用户完成仿真操作等一系列专业仿真 APP。

Sysplorer 层：若将 APP 编译成 exe 类型，则可直接独立运行，若将 APP 编译成 dll 类型，并在插件中增加该工具，可依赖 Sysplorer 环境进行使用和打开。

1.4 开发流程

APP 的全生命周期流程如下图所示。

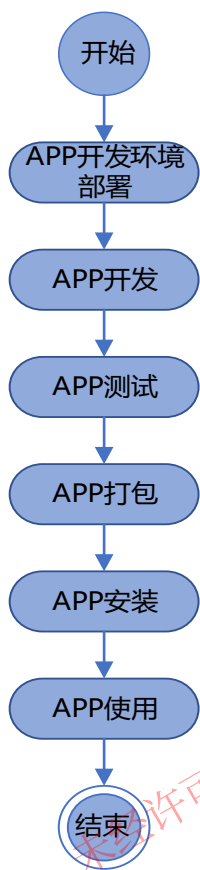


图 1-3 APP 开发流程

1.4.1 APP 开发环境部署

A. SDK 安装

获取 SDK 安装包，如下：

 MWWORKS.Sysplorer 2023a(vs2017_x64)_5.2.1.7362_Setup.exe

图 1-4 SDK 安装包

双击安装包，即可弹出以下界面，一直点击下一步即可安装完成：



图 1-5 SDK 安装界面

B. SDK 目录介绍

MWORKS SDK 安装完成后，在安装目录下存在以下目录：

- bin：SDK 二进制库，包含静态库、动态库、cmake 配置、Sysplorer 运行所需的运行文件；
- docs：SDK 帮助文档，基于 Qt 帮助文档框架开发的帮助文档工具，双击文件夹中的应用程序或 mw_help 快捷方式即可打开；
- examples：SDK 开发示例，提供了各个组件的定制化开发 demo，包括了 SDK 所有的常用开发场景。包括但不限于 HelloWorld 入门程序、Ribbon 界面定制化开发、图形建模定制化开发、曲线窗口定制化开发、模型浏览器拖拽、静默模式编译仿真、仿真数据实时通信、插件配置等。
- include：SDK 依赖的第三方库头文件。
- interface：SDK 同元库的头文件。

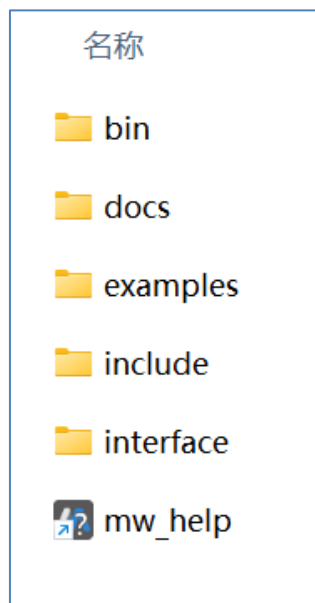


图 1-6 SDK 目录

C. SDK 开发环境

对外发布的 Sysplorer 软件为基于 vs2017 编译出的软件，推荐使用 vs2017 版本 SDK，便于和发布版软件进行集成。

- VS2017 版本

表 1-1 VS2017 配置

类型	环境
操作系统	Win7 SP1 及以上版本
Qt	Qt5.14.2 x86 或 x64 版本
IDE	Microsoft Visual Studio 2017（简称 VS2017）
IDE 插件	VS2017 的 Qt 开发插件

D. 新建工程

基于 VS2017 新建一个 Qt Application 工程：

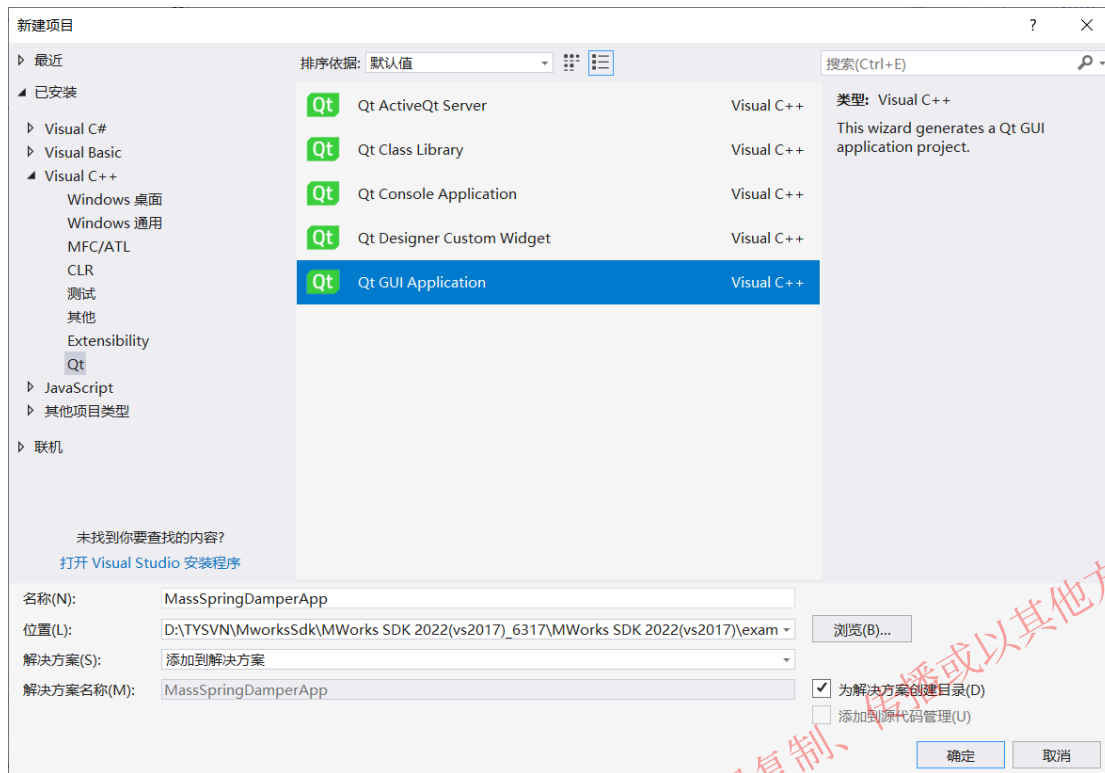


图 1-7 新建工程

E. 输出目录配置

- 将输出目录配置到安装的 SDK 路径的 bin 目录：\$(MWBIn)\$(Configuration)
- 将 MWBin 换为实际 SDK 对应路径：

SDK 安装路径\bin\win_msvc2017x64

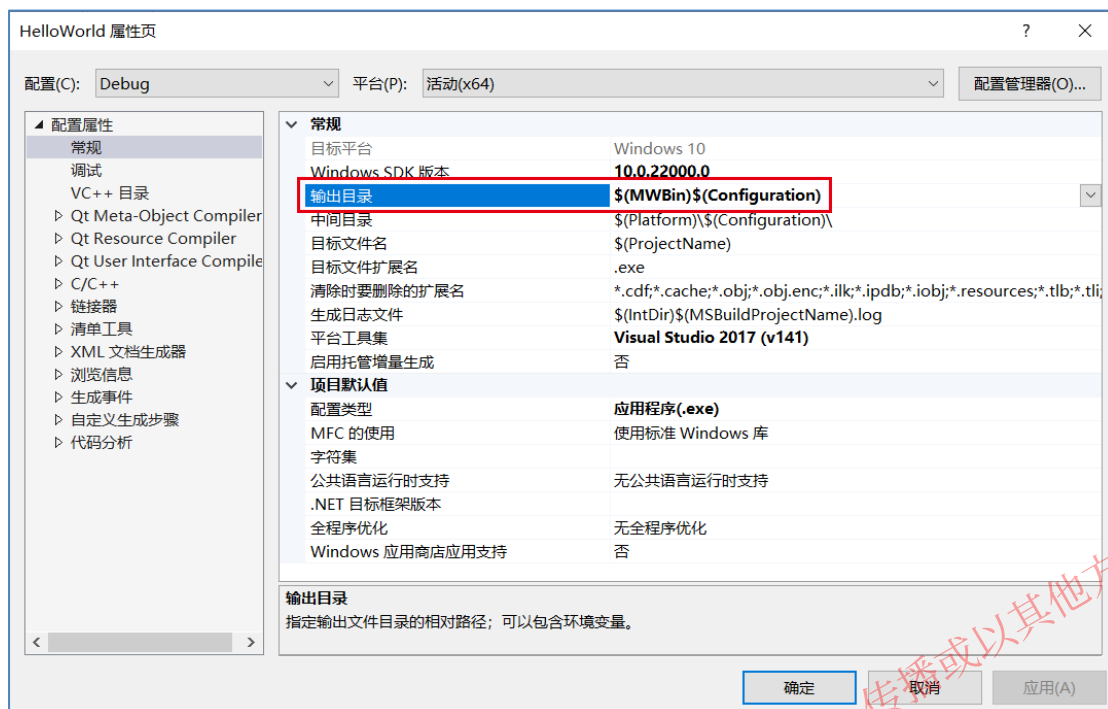


图 1-8 输出目录配置

F. 附加包含目录配置

- 包含目录添加 SDK 的 include 与 interface 目录：

.\GeneratedFiles

.

\$(QTDIR)\include

.\GeneratedFiles\\$(ConfigurationName)

\$(QTDIR)\include\QtCore

\$(QTDIR)\include\QtGui

\$(QTDIR)\include\QtWidgets

\$(MWInclude)

\$(MWInclude)\boost161

\$(MWInterface)

\$(MWInterface)\modelica_services

\$(MWInterface)\common_kits

- MWInclude——SDK 安装路径/include
- MWInterface——SDK 安装路径/interface

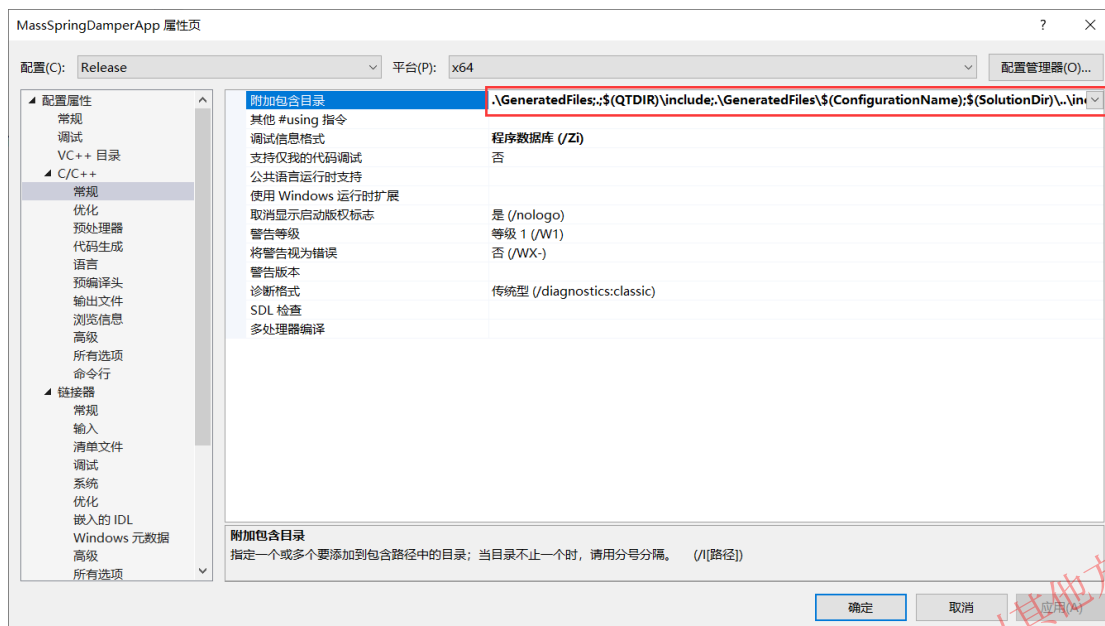


图 1-9 附加包含目录配置

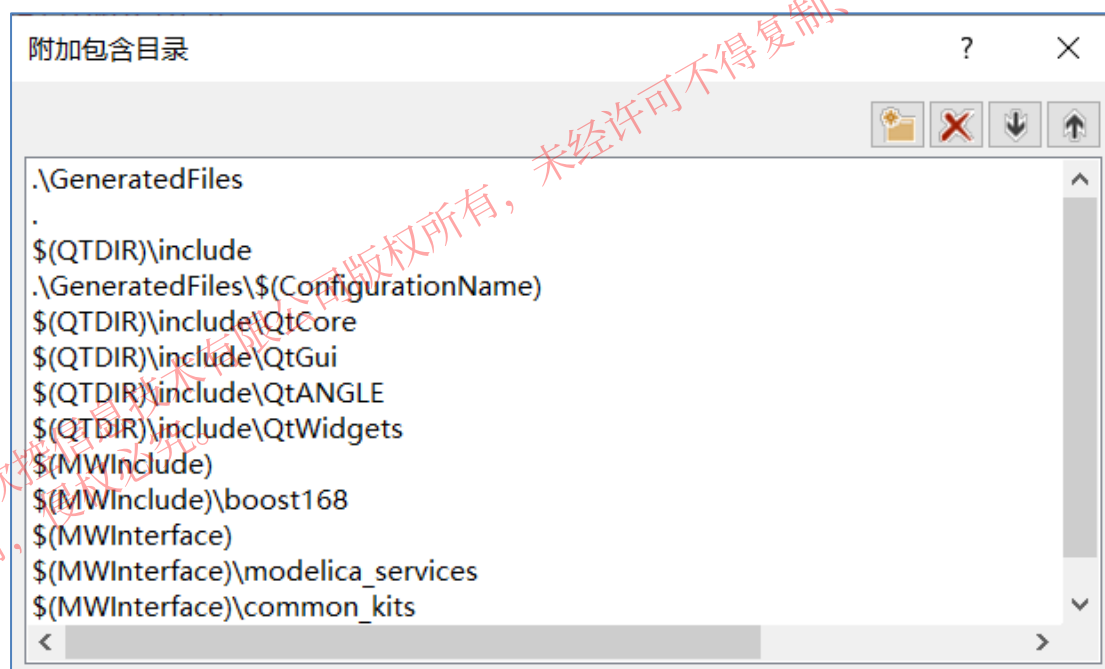


图 1-10 附加包含目录

G. 链接库依赖项配置

- 依赖库目录添加 SDK 的 bin\lib 与文件输出目录
\$(OutDir)
\$(QTDIR)\lib
\$(MWBIn)\lib

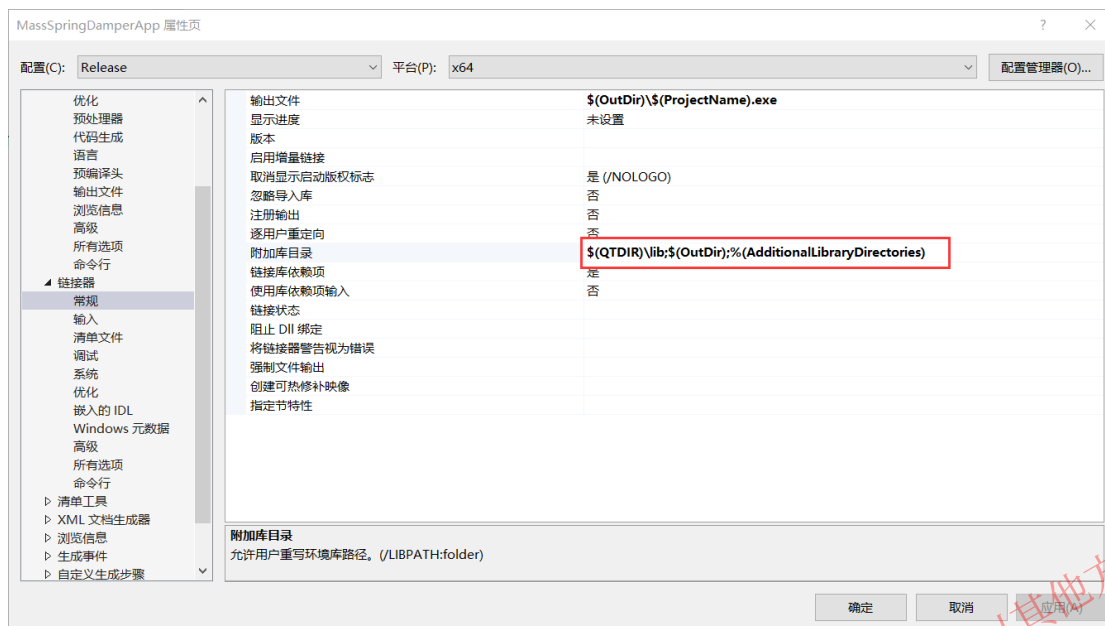


图 1-11 附加依赖项配置

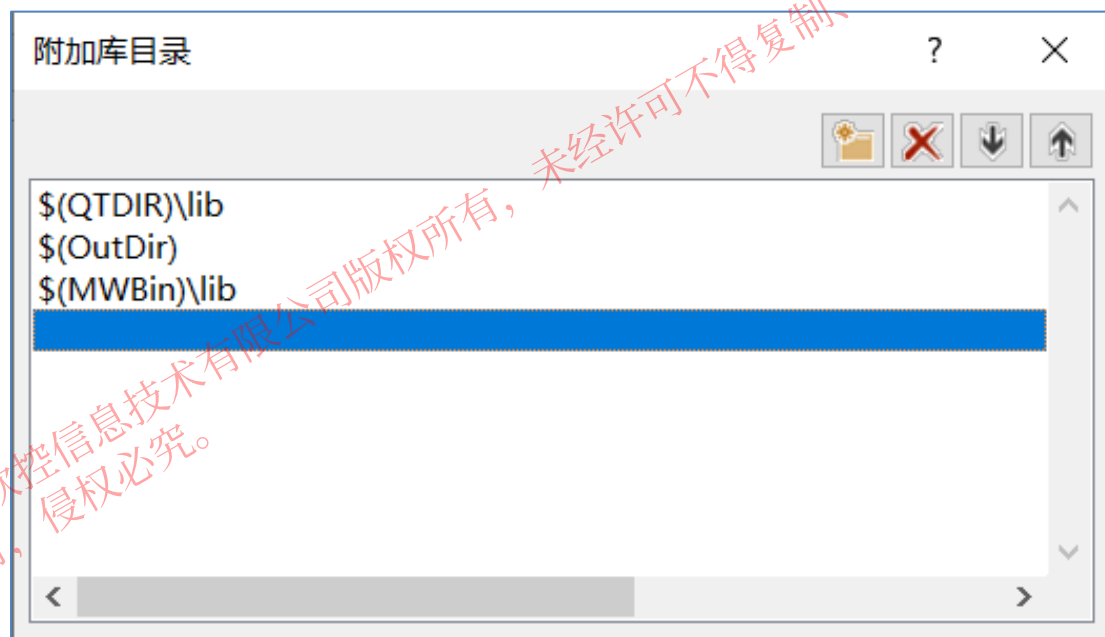


图 1-12 附加库目录

H. 附加依赖项配置

- Debug 下的附加依赖项设置为:

qtmaind.lib

Qt5Cored.lib

Qt5Guid.lib

Qt5Widgets.lib

mw_develop_d.lib

modelica_compiler_d.lib
mw_graphics_view_d.lib
mw_class_manager_d.lib
mw_global_d.lib
mw_help_d.lib
mw_sim_inst_d.lib
mw_sim_plot_d.lib
model_var_tree_d.lib
mw_develop_d.lib

● Release 下的附件依赖项设置为:

qtmain.lib
Qt5Core.lib
Qt5Gui.lib
Qt5Widgets.lib
mw_develop.lib
modelica_compiler.lib
mw_graphics_view.lib
mw_class_manager.lib
mw_global.lib
mw_help.lib
mw_sim_inst.lib
mw_sim_plot.lib
model_var_tree.lib
mw_develop.lib

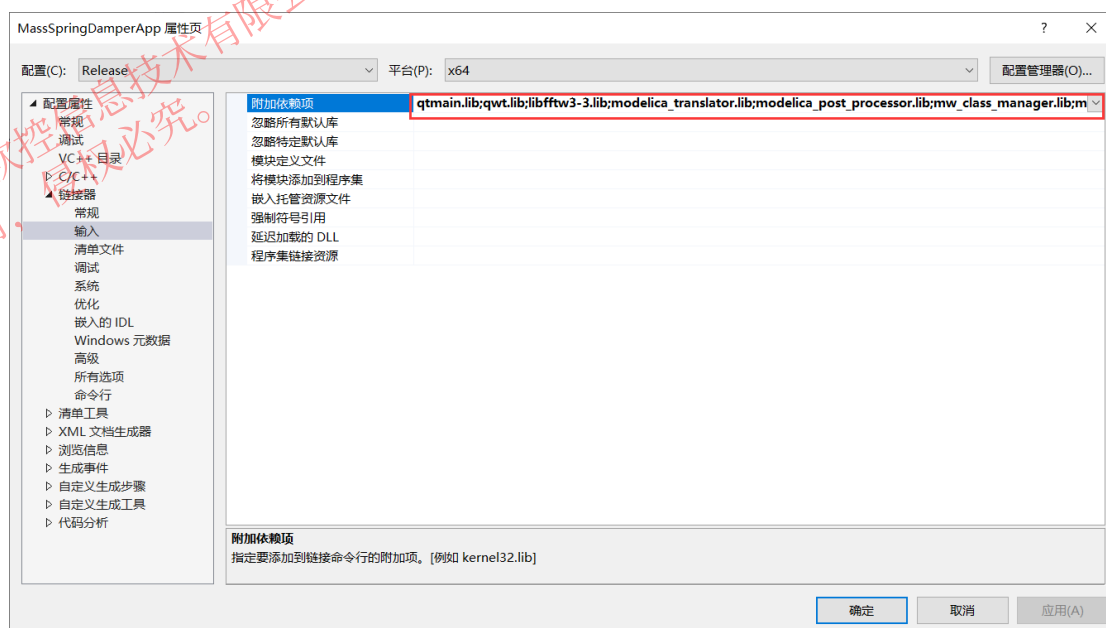


图 1-13 附加依赖项配置

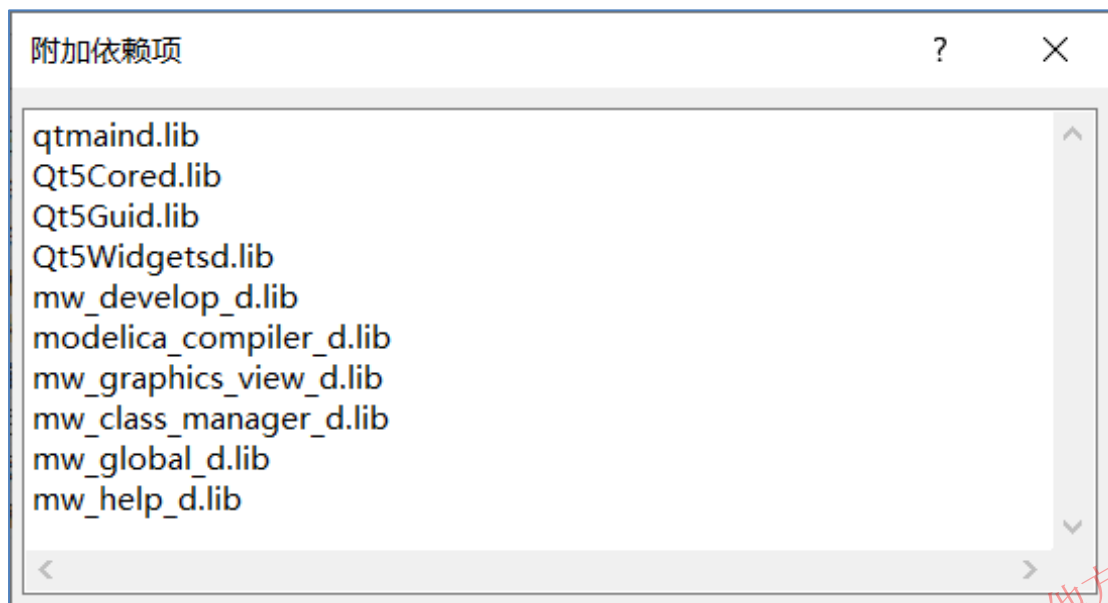


图 1-14 附加依赖项

1.4.2 APP 开发

用户可以使用多种语言结合 Qt 进行界面图形开发 App，用户开发时首先按照 1.4.1 章节将开发环境搭建好并集成 SDK 相关依赖库，然后使用 Qt 进行图形界面开发，结合本身的业务需求调用相关模型接口，SDK 也提供了图形组件供用户直接显示数据，App 开发过程如下：

苏州同元软控信息技术有限公司
版权所有，侵权必究。

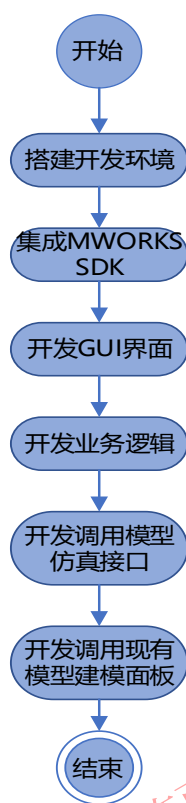


图 1-15 APP 开发过程

1.4.3 APP 测试

App 测试可在开发过程中依赖编译器进行调试，开发完成后可进行系统整体软件测试，最后也可集成到 Sysplorer 软件中进行集成测试。测试流程如下：

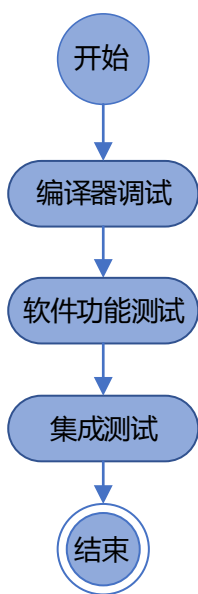


图 1-16 APP 测试流程

1.4.4 APP 打包

APP 打包即遵循具体 APP 开发环境的要求，打包好的 APP 程序需独立可运行，不需再另外安装软件或执行其他的操作。APP 打包也分为仅打包 APP、整个软件打包、插件打包；

A. 仅打包 APP(不带运行环境)

将 APP 生成的 exe 和自行编译所依赖的 dll 集中放在一个文件夹下，即可完成 APP 打包，将该文件夹下的内容复制到任意版本对应的 SDK 的 Release 路径下能够正常运行该 exe。

B. 整个软件打包

将生成的 exe 及依赖的 dll 放入“SDK 安装目录/bin/win_msvc2017x64/Release”目录下：

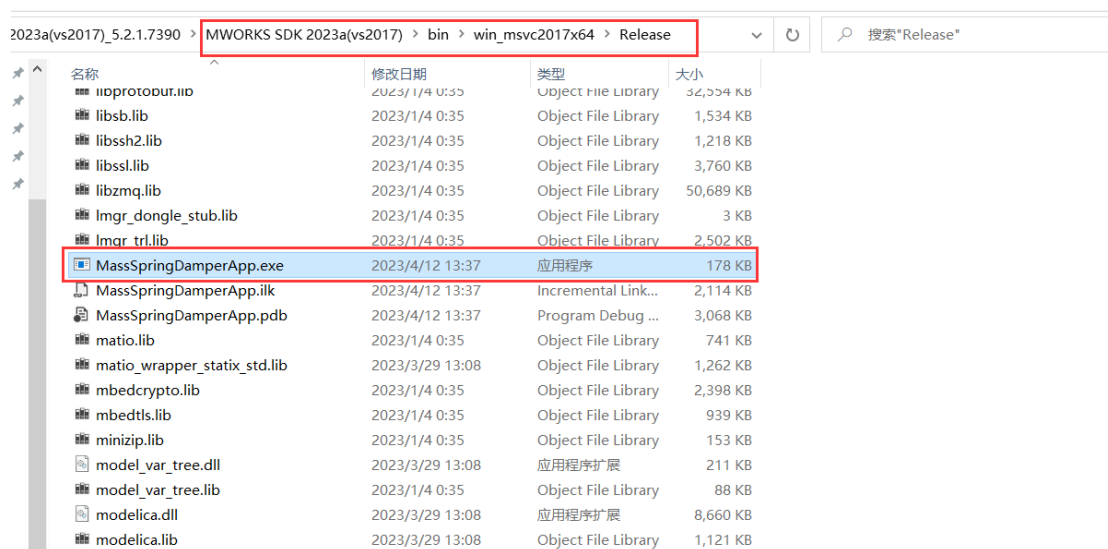


图 1-17 App 运行路径

双击开发的“*.exe”即可启动软件。放入 exe 后可对软件实现打包，打包步骤如下：

1. 将 Release 改为 bin 名称；
2. 保存 bin\win_msvc2017x64 目录下的以下目录：
 - external
 - initial_files
 - Library
 - bin(原 Release 文件)
 - setting
 - simulator
 - tools

3. 最终打包目录如下所示：

名称	修改日期	类型
external	2022/12/12 15:30	文件夹
initial_files	2022/12/12 15:30	文件夹
Library	2023/3/13 9:29	文件夹
Release	2023/3/13 10:33	文件夹
setting	2022/12/12 15:30	文件夹
simulator	2022/12/12 15:30	文件夹
tools	2022/12/12 15:30	文件夹

图 1-18 App 打包目录

C. 插件打包

若开发的是 dll 类型插件，打包方法如下：将嵌入式生成的 dll 库，放入“SDK 安装目录/bin/win_msvc2017x64/Release/Addins”目录下的新建文件夹(文件夹名称任意)并放入 dll 所依赖动态库，即可运行 Sysplorer。

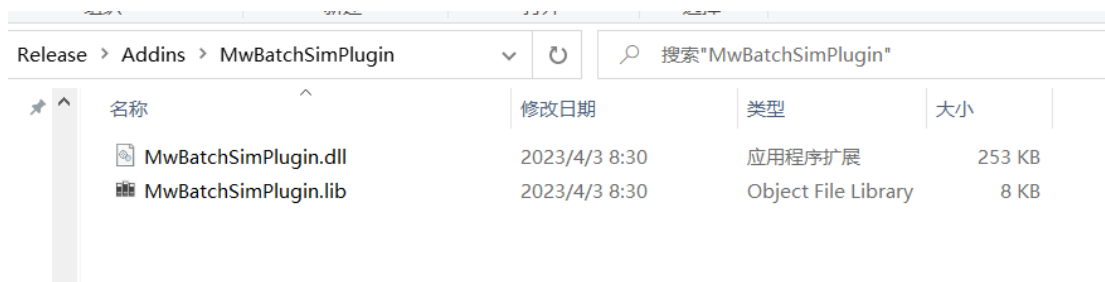


图 1-19 插件打包

点击/bin/win_msvc2017x64/Release/mworks.exe 运行 Sysplorer 软件，可查看到放入的插件。

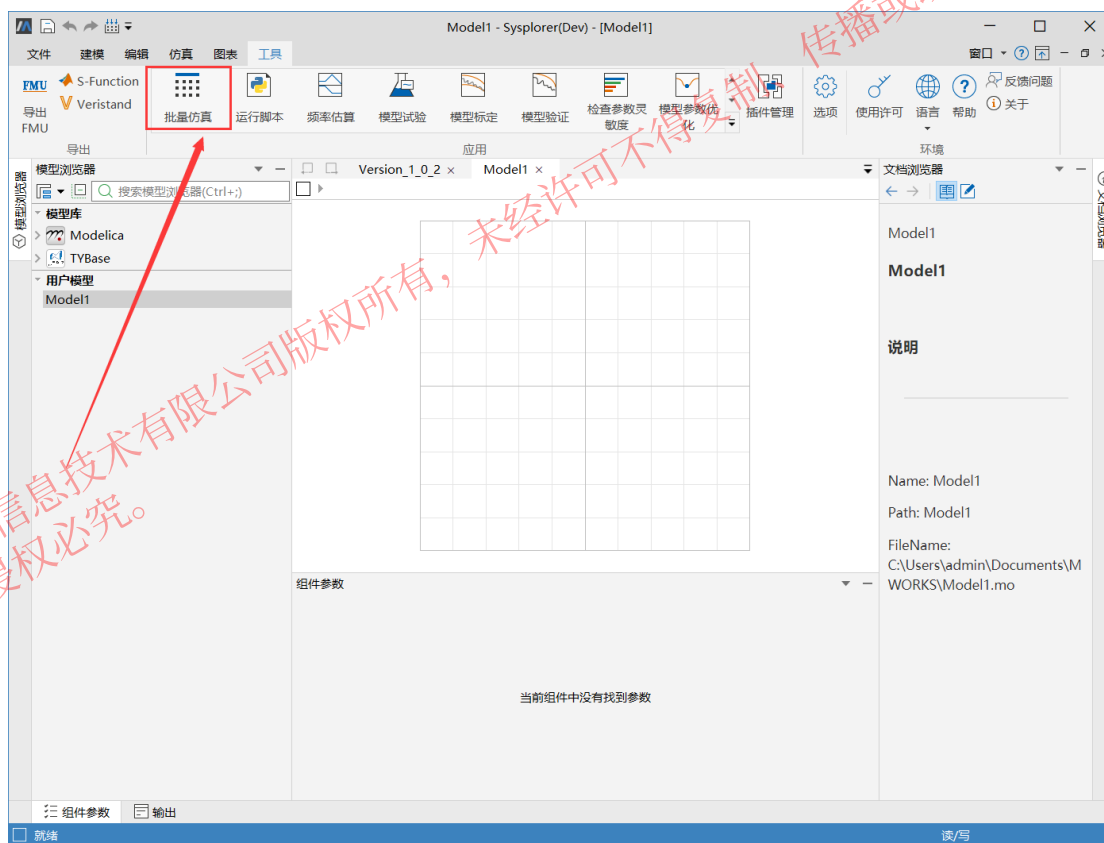


图 1-20 插件效果

将 dll 和文件夹打包，放入任意 Sysplorer 的 bin 目录下的 Addins 文件夹下，然后运行 Sysplorer 可查看到开发的插件功能。

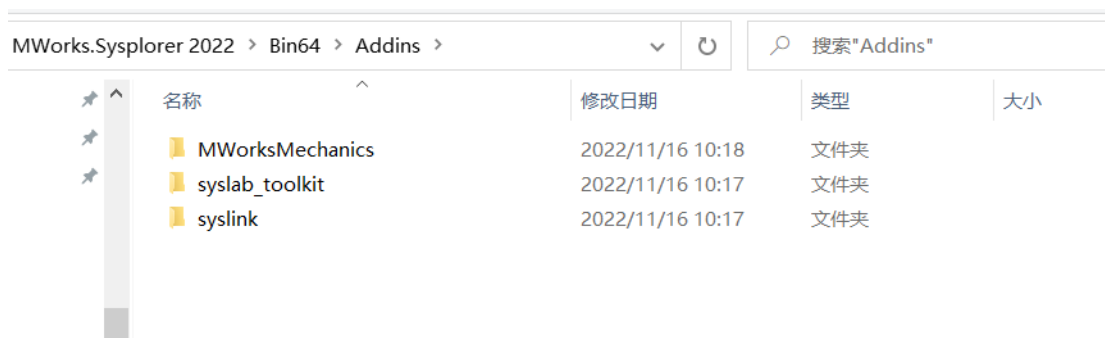


图 1-21 插件发布

1.4.5 APP 安装

生成独立 exe 的 APP 无需进行软件安装，直接点击 exe 即可运行成功。

若是生成 dll 的 APP 软件需依赖 Sysplorer 进行启动，将打包好的安装包放在任意 Sysplorer 安装下 Bin/Addins 目录下即可。

1.4.6 APP 使用

APP 开发完成后，用户可独立运行 exe 类型 APP；对于开发的 dll 动态库需依赖 Sysplorer 进行使用，将该动态库放入 Sysplorer 安装下 Bin/Addins 目录下即可，启动 Sysplorer，按照 dll 开发的入口方式打开即可。

2. 如何快速开发 APP

2.1 质量弹簧阻尼 APP 介绍

本章节将使用 SDK 快速开发完成一个弹簧阻尼模型案例，从模型搭建-工程配置-SDK 功能开发流程开发一个质量弹簧阻尼仿真 APP，最终开发效果如下所示：

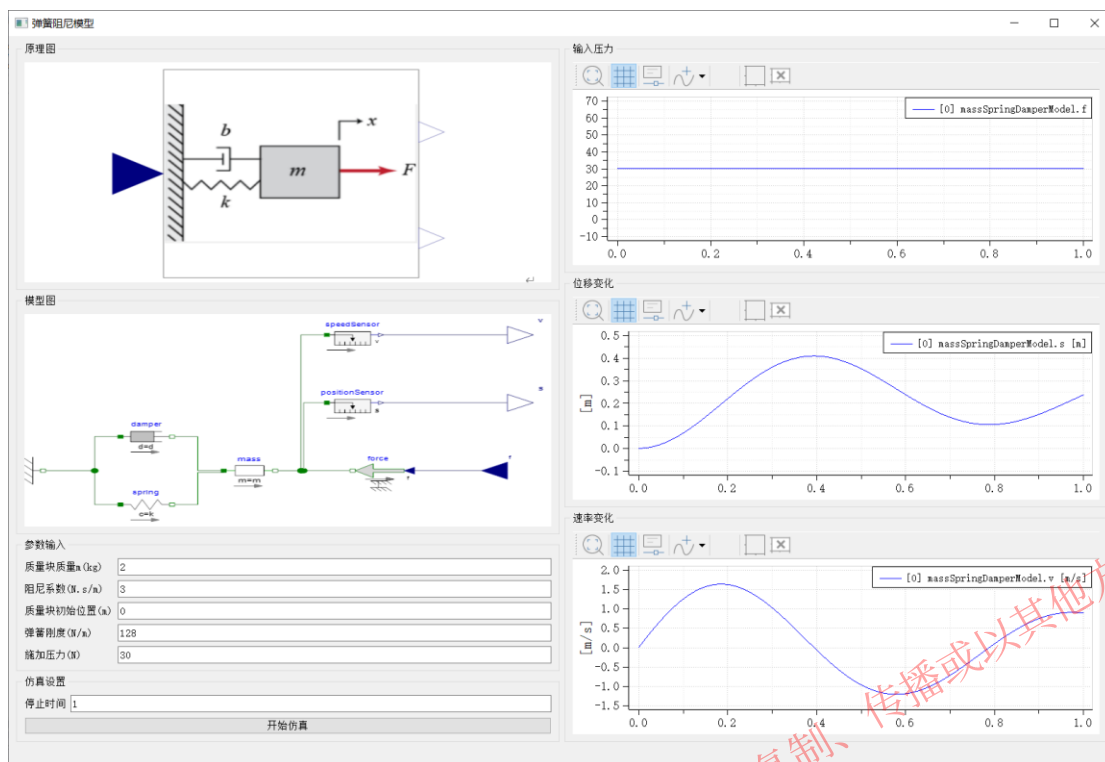


图 2-1 质量-弹簧-阻尼模型 APP

该 App 可满足以下功能点：

- 底层加载 Modelica3.2.1 标准模型库；
- 打开已搭建好的质量弹簧阻尼模型；
- 可对质量、阻尼系数、质量块初始位置、弹簧刚度、施加压力等参数和输入进行设置；
- 可对仿真设置中的仿真停止时间进行设置；
- 可对打开的质量弹簧阻尼模型进行仿真；
- 对仿真后的压力、位移变化、速率变化等数据进行获取；
- 将获取的数据显示到 SDK 提供的曲线组件上。

运行实例：

打开 vs2017.sln，右键项目，将其设置为启动项目。

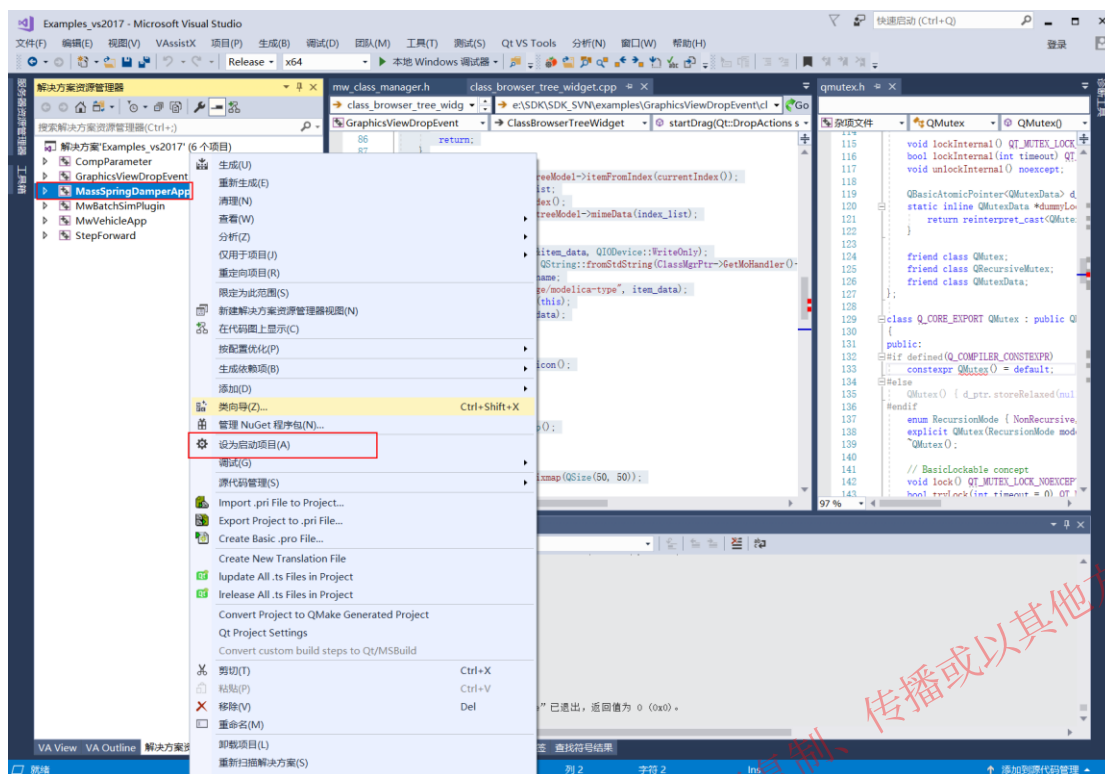


图 2-2 打开项目

选择 ReleaseX64 点击  运行代码，首次启动要加载各种库文件，比较慢，可以优先选择 Release 启动。

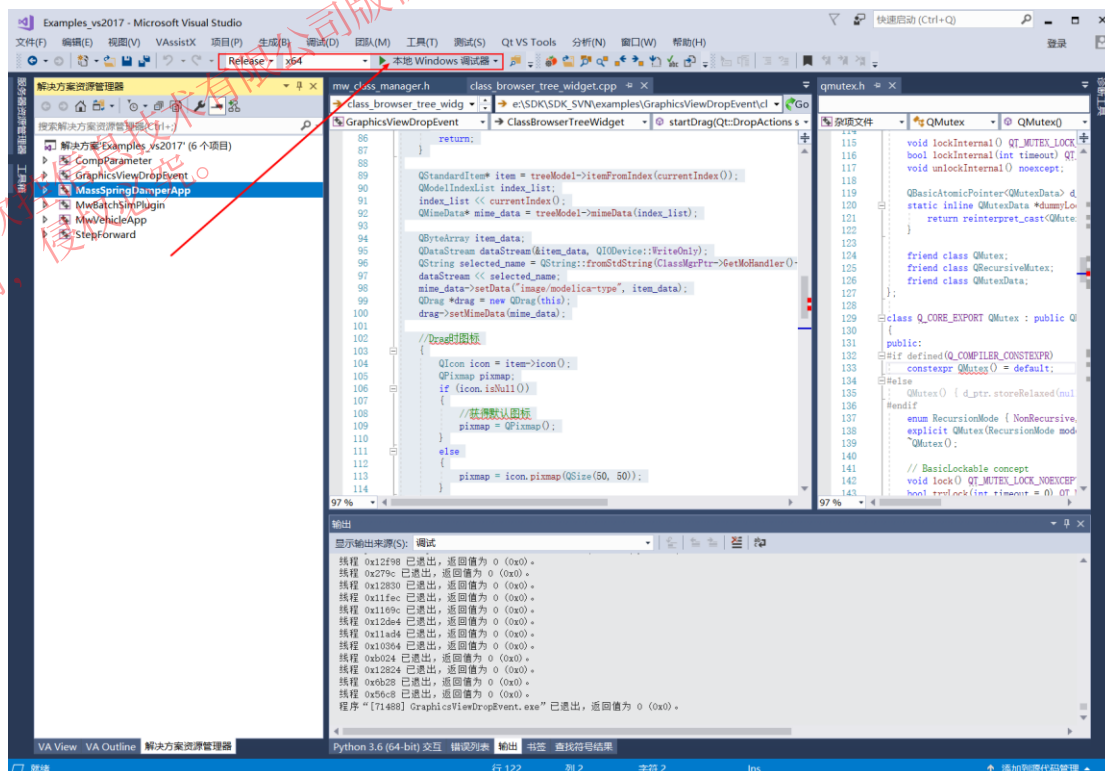


图 2-3 运行项目

如果第一次启动调试报错如下：

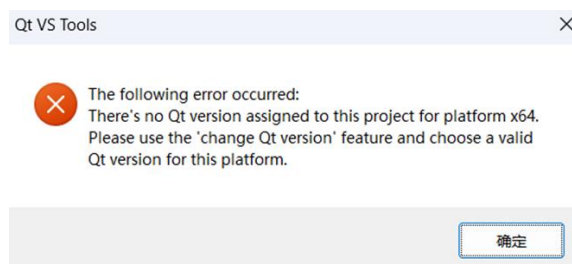


图 2-4 qt 运行错误

则需要更新下 QT 版本即可：

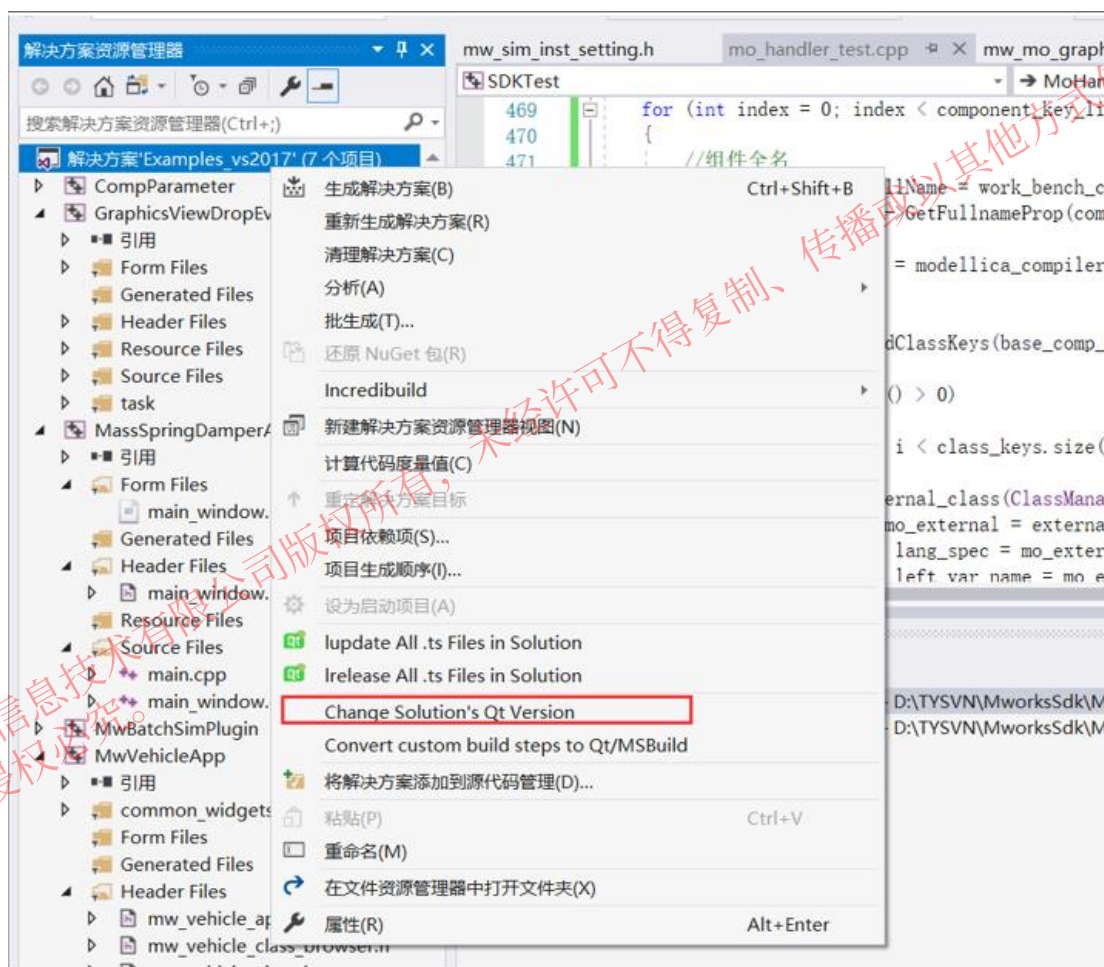


图 2-5 设置 Qt 版本

2.2 模型搭建

本次开发 APP 需依赖模型，打开 MWORKS.Sysplorer 搭建一个质量弹簧阻尼系统，质量—弹簧—阻尼器系统可用简单的数学模型表示，即数轴建模法，建立与系统平行方向的数轴，把弹簧或阻尼器的实际位移值当做有理数标在数轴上

对应位置, 然后按照有理数比较大小的结果确定相应质量块所受弹簧力或阻尼力的大小和方向, 进而求得系统的微分方程。分别以单自由度系统、两自由度振动系统及多自由度系统为例, 阐述了数轴法在建立机械振动系统的数学模型中的应用。最终搭建效果如下所示:

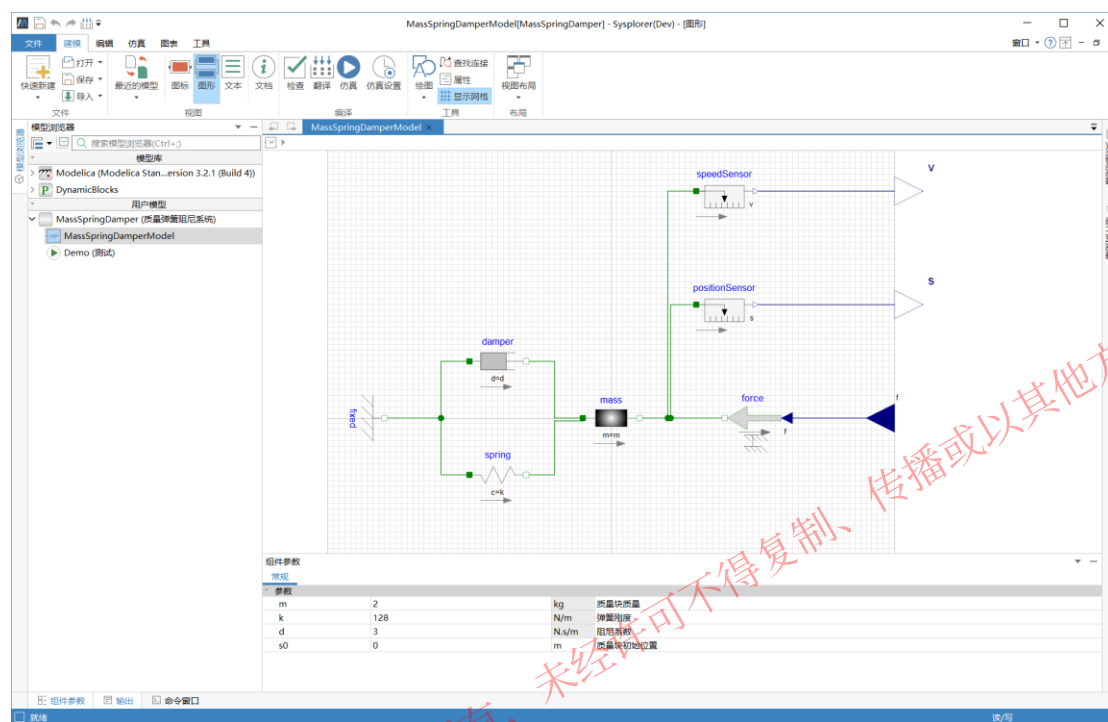


图 2-6 质量-弹簧-阻尼模型搭建

记录下搭建该模型的名称, 及路径位置, 便于后续使用。

2.3 开发工程构建

基于 VS2017 版本进行开发, 请确保已安装 VS2017 Qt 插件, 并安装 Qt5.14.2 等相关开发环境。

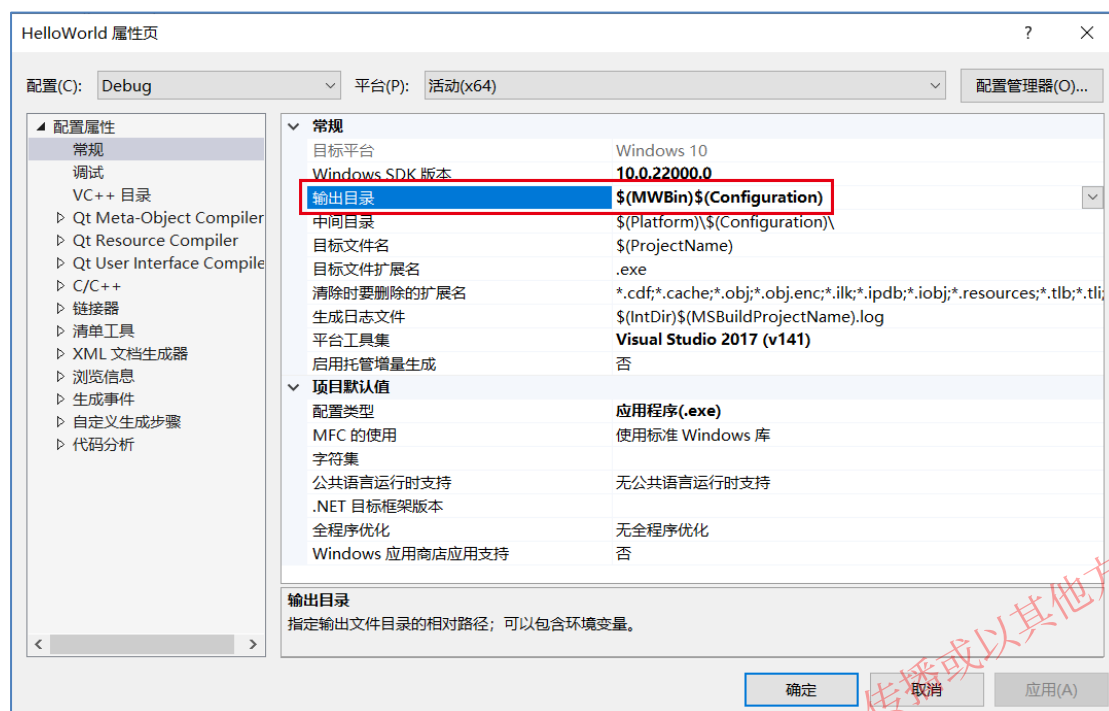
A. 新建工程

基于 VS2017 新建一个 Qt Application 工程, 命名为 MassSpringDamperApp:

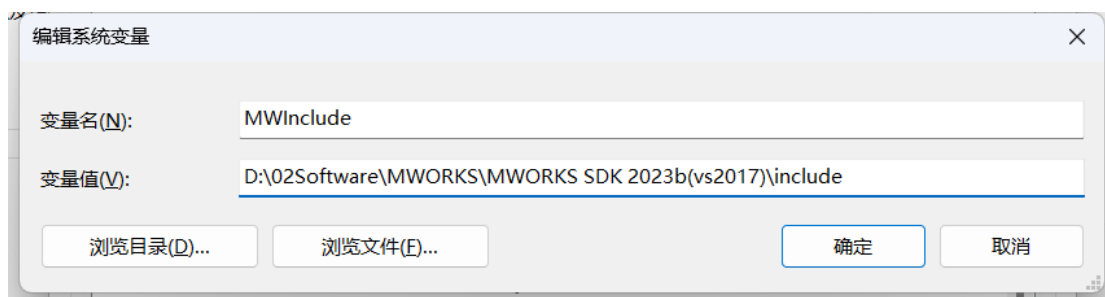
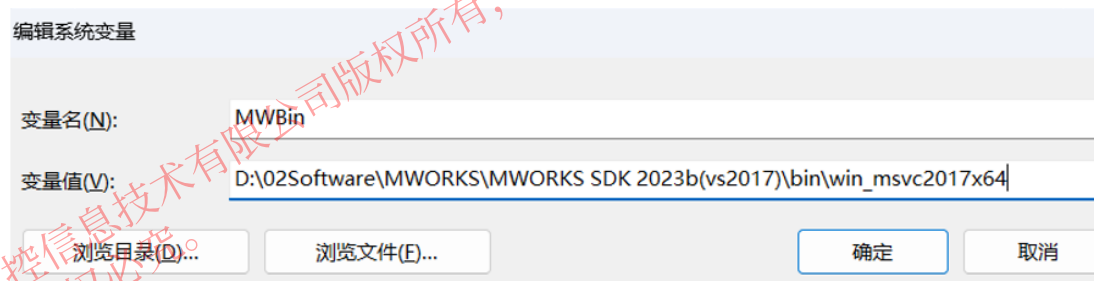
B. 输出目录配置

- SDK 安装路径\bin\win_msvc2017x64

SDK 安装路径\bin\win_msvc2017x64



\$(MWBIn)、\$(MWInterface)、\$(MWInclude)需在电脑中的系统环境变量中将 SDK 安装路径配置上：



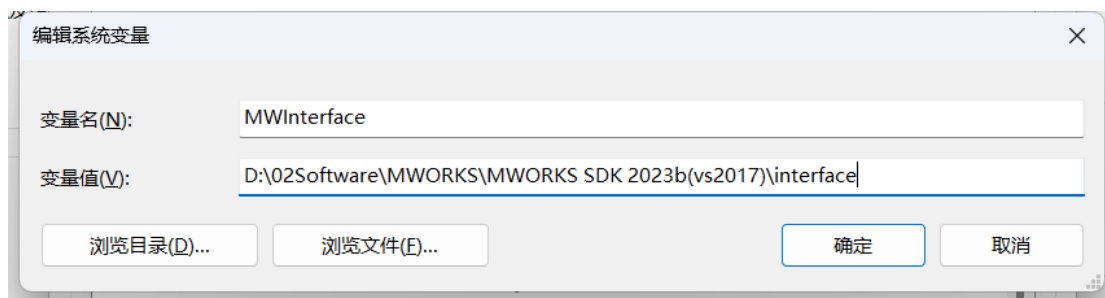


图 2-11 MWInterface 环境变量配置

C. 附加包含目录配置

- 包含目录添加 SDK 的 include 与 interface 目录：

.\GeneratedFiles

.

\$(QTDIR)\include

.\GeneratedFiles\\$(ConfigurationName)

\$(QTDIR)\include\QtCore

\$(QTDIR)\include\QtGui

\$(QTDIR)\include\QtWidgets

\$(MWInclude)

\$(MWInclude)\boost161

\$(MWInterface)

\$(MWInterface)\modelica_services

\$(MWInterface)\common_kits

- MWInclude——SDK 安装路径/include
- MWInterface——SDK 安装路径/interface

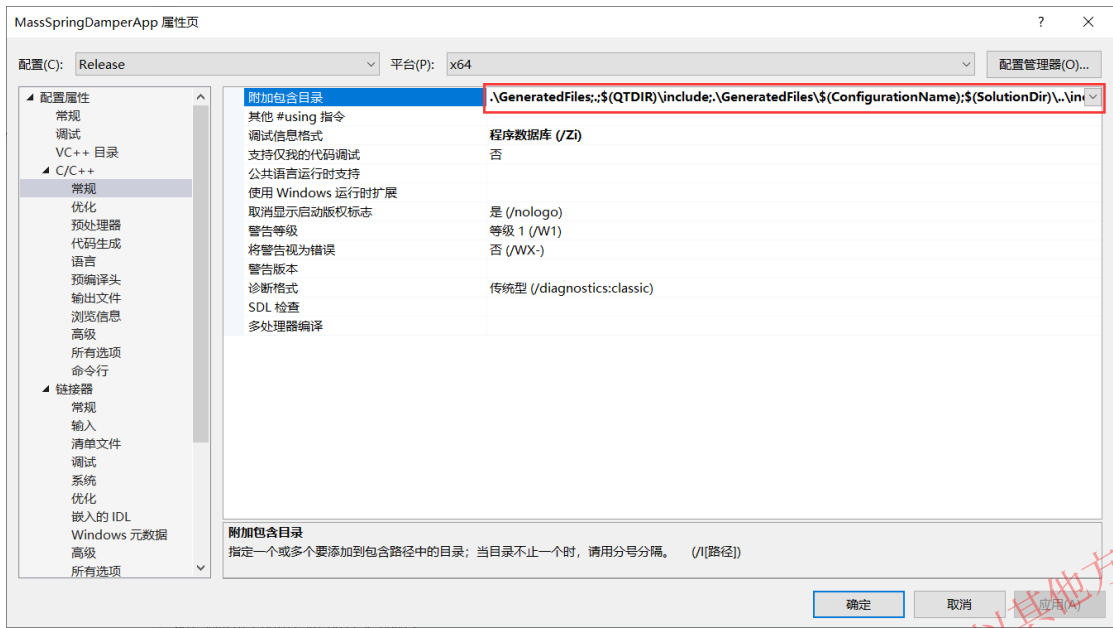


图 2-12 附加包含目录配置

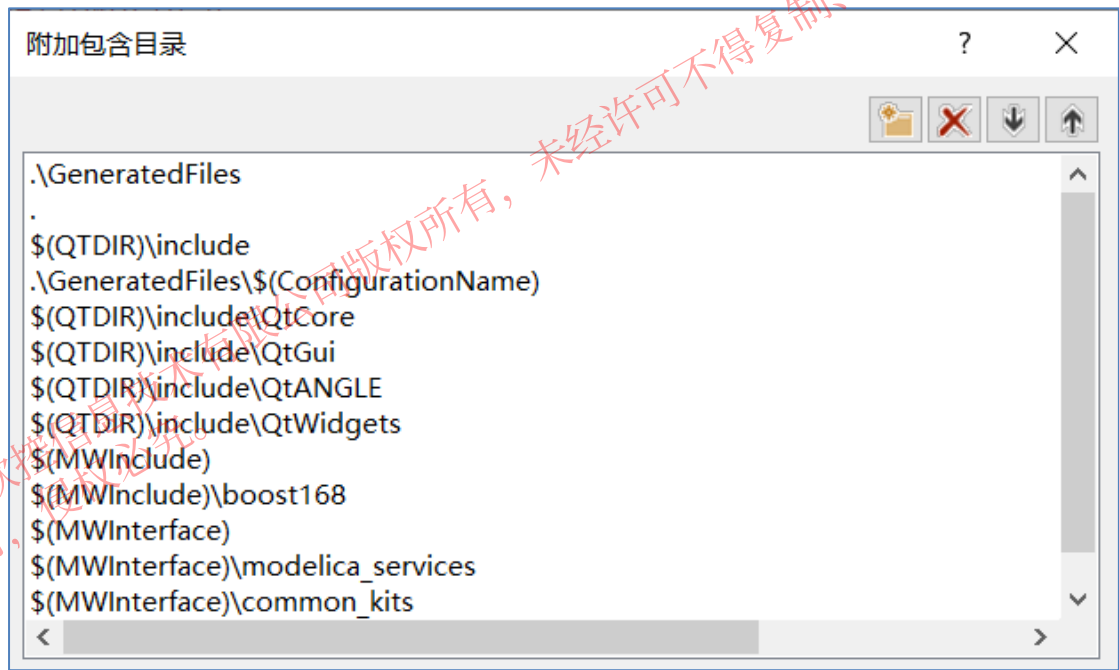


图 2-13 附加包含目录

D. 链接库依赖项配置

- 依赖库目录添加 SDK 的 bin\lib 与文件输出目录
\$(OutDir)
\$(QTDIR)\lib
\$(MWBIn)\lib

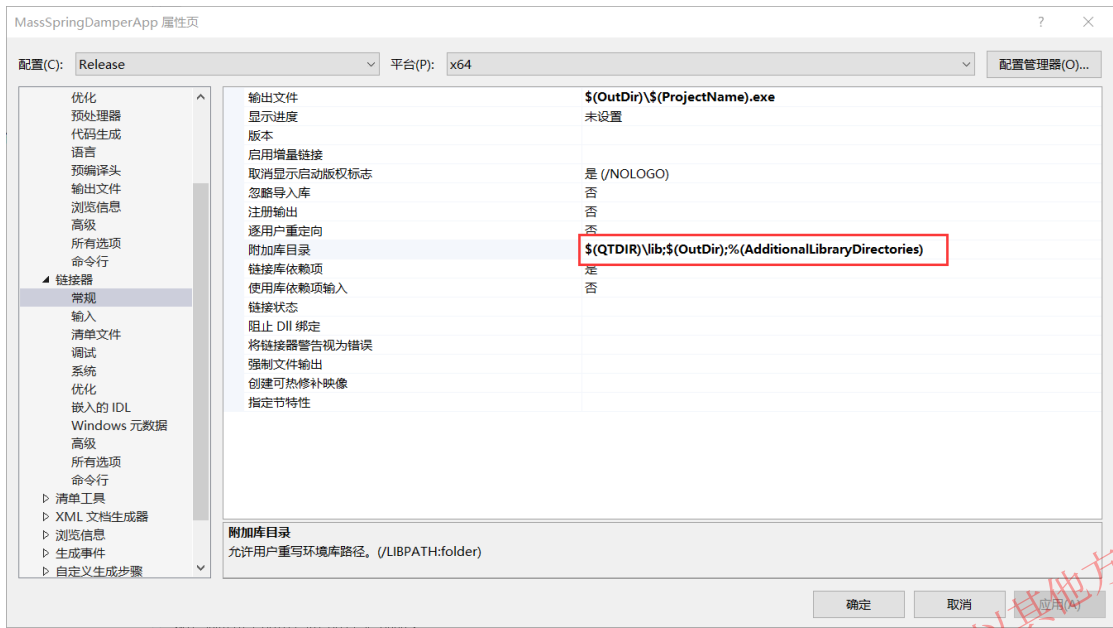


图 2-14 附加依赖项配置

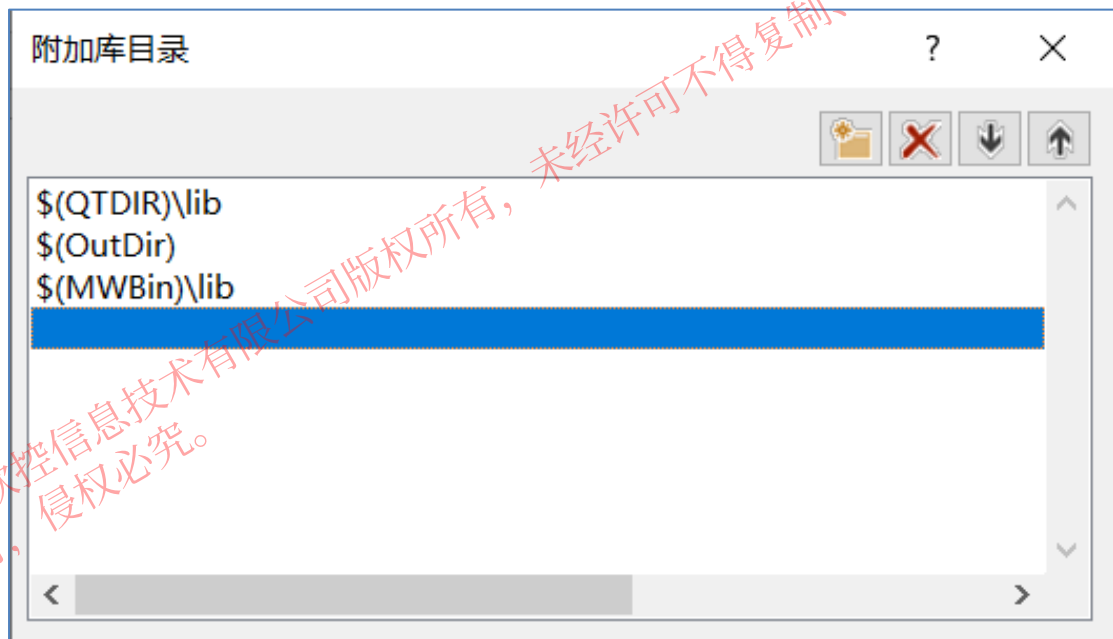


图 2-15 附加库目录

E. 附加依赖项配置

- Debug 下的附加依赖项设置为：

qtmaind.lib

Qt5Cored.lib

Qt5Guid.lib

Qt5Widgets.lib

mw_develop_d.lib

modelica_compiler_d.lib
mw_graphics_view_d.lib
mw_class_manager_d.lib
mw_global_d.lib
mw_help_d.lib
mw_sim_inst_d.lib
mw_sim_plot_d.lib
model_var_tree_d.lib
mw_develop_d.lib

● Release 下的附件依赖项设置为:

qtmain.lib
Qt5Core.lib
Qt5Gui.lib
Qt5Widgets.lib
mw_develop.lib
modelica_compiler.lib
mw_graphics_view.lib
mw_class_manager.lib
mw_global.lib
mw_help.lib
mw_sim_inst.lib
mw_sim_plot.lib
model_var_tree.lib
mw_develop.lib

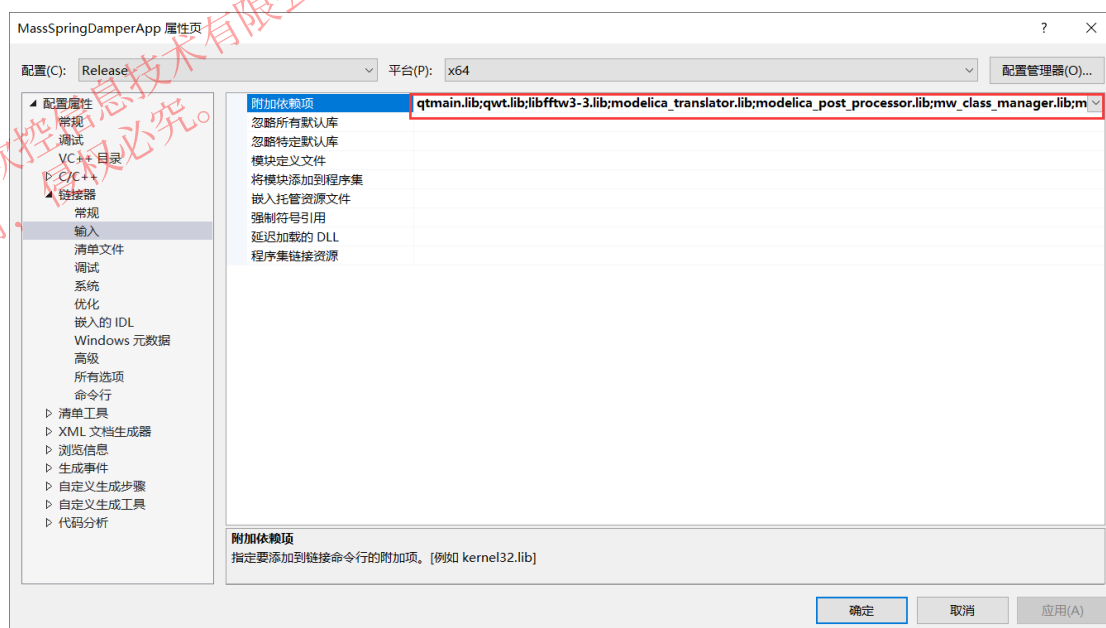


图 2-16 附加依赖项配置

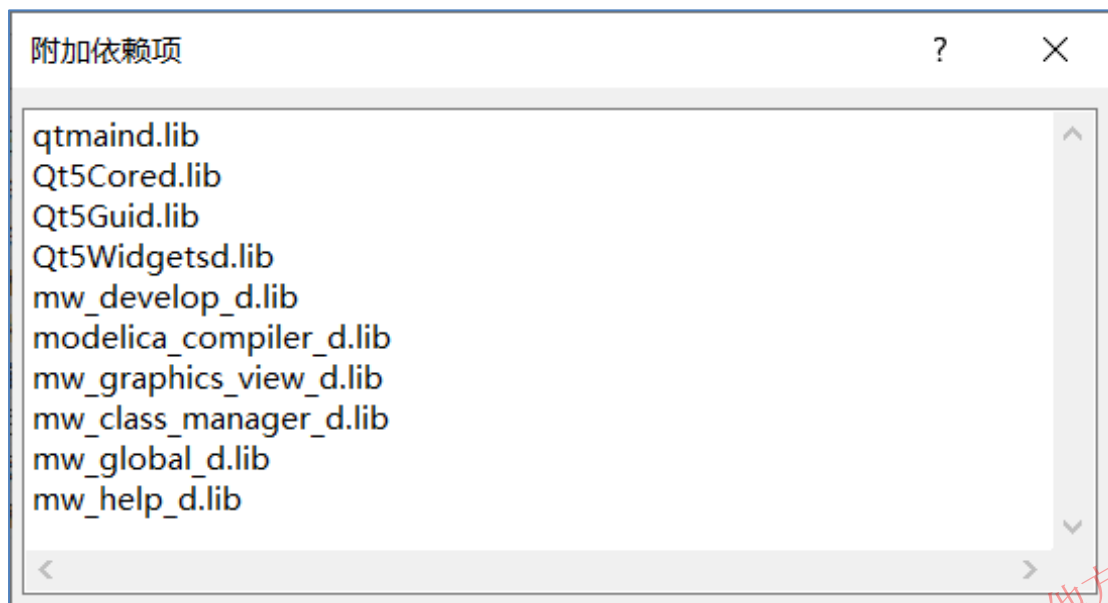


图 2-17 附加依赖项

2.4 App 界面及功能详解

2.4.1 main 函数

main 函数中主要做以下功能点：

- 初始化 QApplication
- 初始化 MwClassManager 类
- 加载翻译文件
- 显示 QMainWindow 主窗口

```
int main(int argc, char *argv[])
{
    QApplication app(argc,argv);

    //初始化 SDK 内核管理类接口
    MwClassManager* classMgr = new MwClassManager();
    classMgr->Initialize();

    //加载中文配置文件
    classMgr->GetCoreOption()->LoadChineseTranslateFile(qApp);

    //显示主窗口
    MainWindow main_win(classMgr);
```

```
main_win.show();

return app.exec();
}
```

2.4.2 QMainWindow 界面

A. Qt Designer 设计界面

利用 Qt 控件搭建 MainWindow 界面如下, 使用 Qt 搭建界面详细材料可查看专题说明部分 Qt 界面开发教程 4.1 章节。

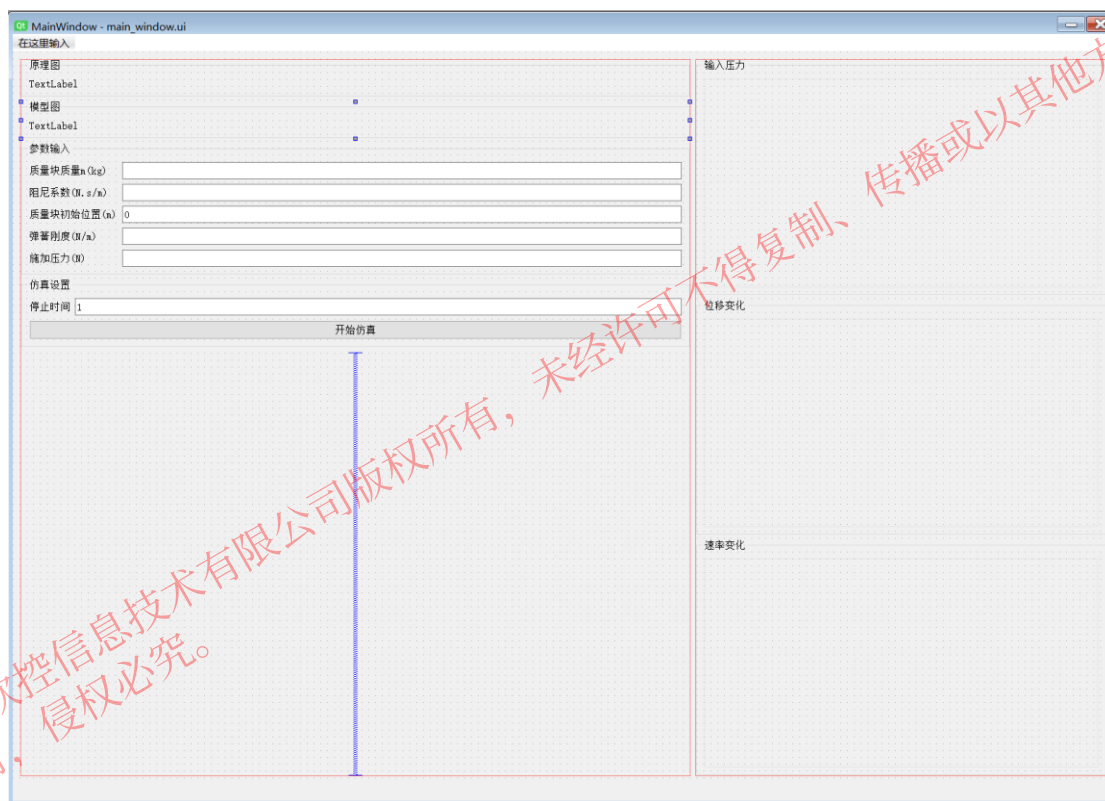


图 2-18 Qt 设计界面

B. 界面加载模型图片

将模型图增加到界面中的 label:

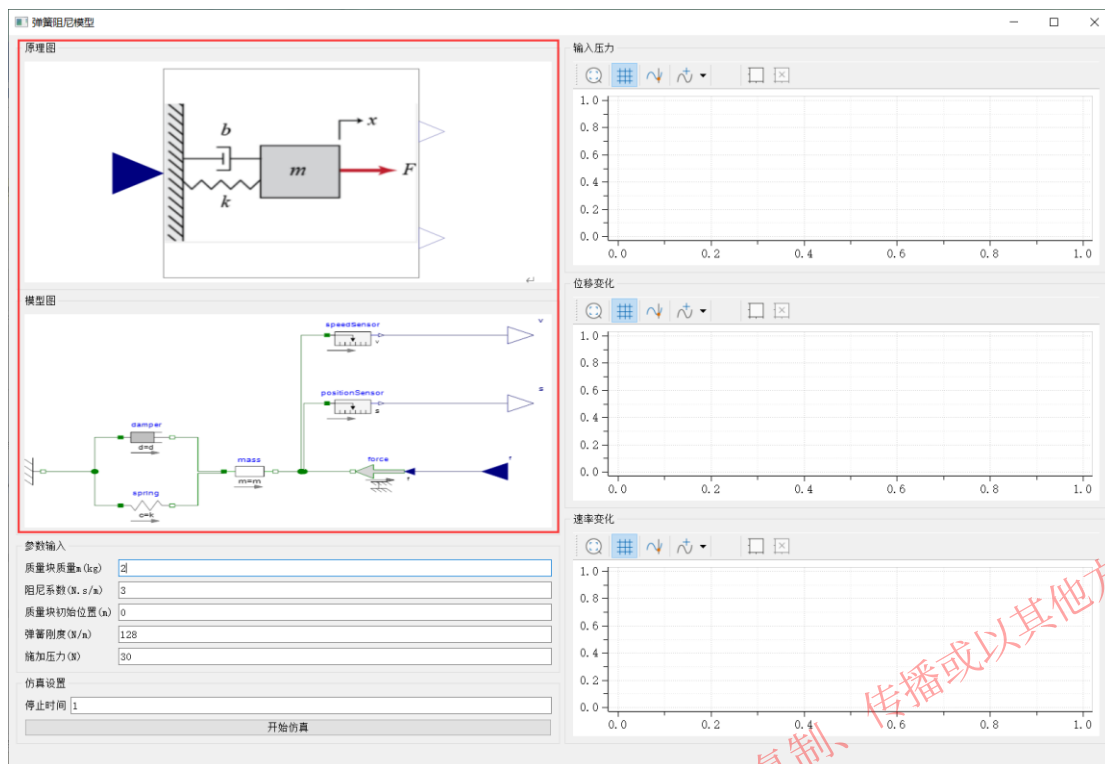


图 2-19 增加图片

代码如下：

```
//初始化原理图
QString app_path = QApplication::applicationDirPath();
QString mo_path = app_path +
"/../../examples/MassSpringDamperApp/Resource/springDamper.png";
QPixmap pixmap(mo_path);
pixmap.scaled(ui.label->size(), Qt::IgnoreAspectRatio);
ui.label->setScaledContents(true);
ui.label->setPixmap(pixmap);

//初始化模型图
QString mo_path1 = app_path +
"/../../examples/MassSpringDamperApp/Resource/springDamper_model.png";
QPixmap pixmap1(mo_path1);
pixmap1.scaled(ui.label_8->size(), Qt::IgnoreAspectRatio);
ui.label_8->setScaledContents(true);
ui.label_8->setPixmap(pixmap1);
```

C. 界面中加载曲线窗口组件

使用 SDK 中的 `MwSimPlotWindow` 类创建曲线视图嵌入 `QMainWindow` 中：

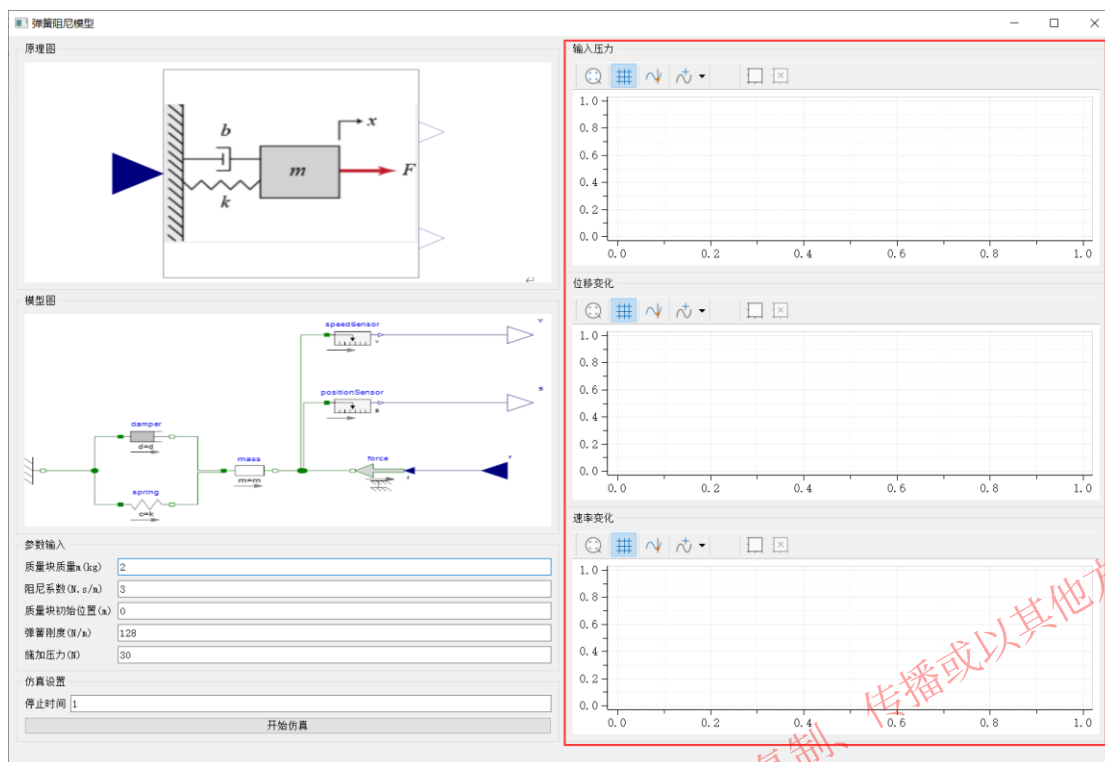


图 2-20 增加曲线界面

代码如下：

```
//增加曲线界面
delete ui.widget;
forcePlotWin = new MwSimPlotWindow(1, true, classMgr, this, false);
ui.gridLayout_3->addWidget(forcePlotWin, 0, 0, 1, 1);
forcePlotWin->show();

delete ui.widget_2;
sPlotWin = new MwSimPlotWindow(1, true, classMgr, this, false);
ui.gridLayout_4->addWidget(sPlotWin, 0, 0, 1, 1);
sPlotWin->show();

delete ui.widget_3;
vPlotWin = new MwSimPlotWindow(1, true, classMgr, this, false);
ui.gridLayout_5->addWidget(vPlotWin, 0, 0, 1, 1);
vPlotWin->show();

forcePlotWin->menuBar()->hide();
sPlotWin->menuBar()->hide();
vPlotWin->menuBar()->hide();

forcePlotWin->statusBar()->hide();
```

```
sPlotWin->statusBar()->hide();
vPlotWin->statusBar()->hide();
```

2.4.3 主要功能详解

主要开发流程如下，提前加载底层模型及数据、开始仿真、仿真完成后等操作。

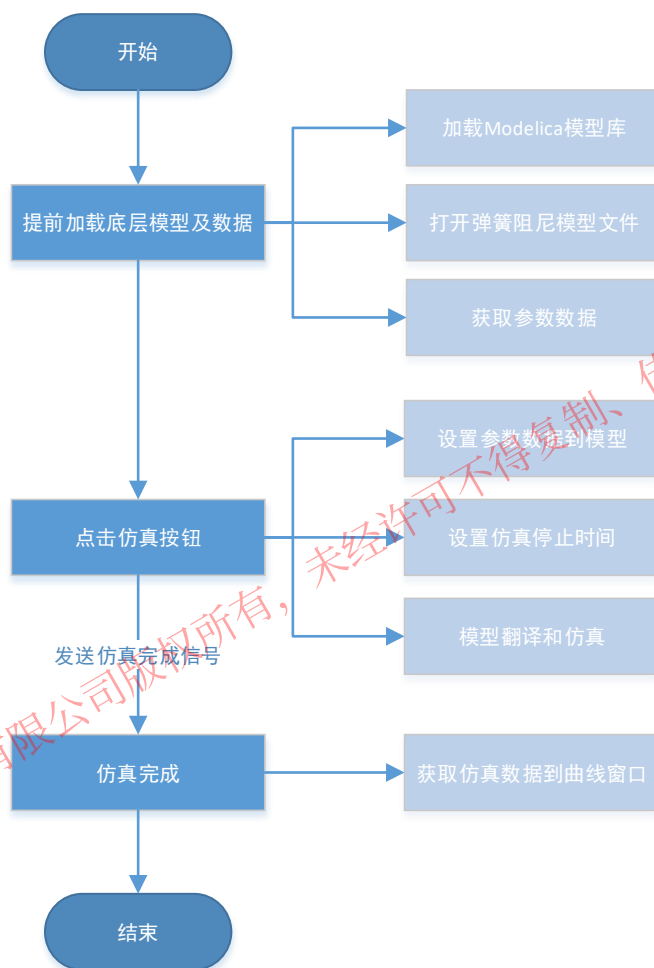


图 2-21 开发流程

2.4.3.1 提前加载底层模型及数据

MainWindow 中的构造函数中主要加载如下几点功能：底层加载标准模型库、打开搭建好的质量-弹簧-阻尼模型文件、提前获取参数值显示在面板中。

A. 加载标准模型库

获取 Modelica 标准模型库 3.2.1 版本保证，需 bin 目录下的 Library 路径下存在改模型库，代码如下：

```
bool is_success = classMgr->GetMoHandler()->LoadMoLibrary("Modelica", "3.2.1");
```

B. 加载模型文件

将 3.2 章节搭建的模型在底层打开，代码如下：

```
QString app_path = QApplication::applicationDirPath();
QString mo_path = app_path +
"/../../examples/MassSpringDamperApp/Resource/MassSpringDamper.mo";

bool is_success = classMgr->GetMoHandler()->OpenFile(mo_path.toStdWString());
```

C. 获取参数数据

为保证打开软件前，界面能够显示如下数据，需对模型中的相关参数进行获取：

参数输入	
质量块质量 m (kg)	2
阻尼系数 $(N \cdot s / m)$	3
质量块初始位置 (m)	0
弹簧刚度 (N / m)	128
施加压力 (N)	30

图 2-22 参数数据

获取参数数据代码如下：

```
//获取模型数据
MWin key
= classMgr->GetMoHandler()->GetKeyByTypeName("MassSpringDamper.Demo");
if (key != 0)
{
    QString value_force =
    QString::fromStdString(classMgr->GetMoHandler()->GetParamValue(key,
    GetMwStrList("const.k")));
    QString value_m =
    QString::fromStdString(classMgr->GetMoHandler()->GetParamValue(key,
    GetMwStrList("massSpringDamperModel.m")));
    QString value_k =
    QString::fromStdString(classMgr->GetMoHandler()->GetParamValue(key,
    GetMwStrList("massSpringDamperModel.k")));
    QString value_d =
    QString::fromStdString(classMgr->GetMoHandler()->GetParamValue(key,
    GetMwStrList("massSpringDamperModel.d")));
```

```

QString value_s0 =
QString::fromStdString(classMgr->GetMoHandler()->GetParamValue(key,
GetMwStrList("massSpringDamperModel.s0")));

ui.lineEdit_force->setText(value_force);
ui.lineEdit_m->setText(value_m);
ui.lineEdit_k->setText(value_k);
ui.lineEdit_d->setText(value_d);
ui.lineEdit_s0->setText(value_s0);
}

```

2.4.3.2 点击仿真按钮

模型库及模型文件加载完成后，可在界面中输入参数数据和仿真设置，完成后，可点击如下仿真按钮：

图 2-23 开始仿真按钮

点击仿真后，会将参数设置到模型中、设置仿真停止时间、编译和仿真等过程。

A. 设置参数到模型

获取输入框中对应的值，将其设置到模型中，代码如下：

```

//修改参数值
MWin key =
classMgr->GetMoHandler()->GetKeyByTypeName("MassSpringDamper.Demo");
std::string value_force = ui.lineEdit_force->text().toStdString();
classMgr->GetMoHandler()->SetParamValue(key, GetMwStrList("const.k"), value_force);
std::string value_m = ui.lineEdit_m->text().toStdString();
classMgr->GetMoHandler()->SetParamValue(key,
GetMwStrList("massSpringDamperModel.m"), value_m);

```

```
std::string value_k = ui.lineEdit_k->text().toStdString();
classMgr->GetMoHandler()->SetParamValue(key,
GetMwStrList("massSpringDamperModel.k"),value_k);
std::string value_d = ui.lineEdit_d->text().toStdString();
classMgr->GetMoHandler()->SetParamValue(key,
GetMwStrList("massSpringDamperModel.d"),value_d);
std::string value_s0 = ui.lineEdit_s0->text().toStdString();
classMgr->GetMoHandler()->SetParamValue(key,
GetMwStrList("massSpringDamperModel.s0"),value_s0);
```

B. 设置仿真停止时间

获取输入框中的仿真停止时间，将其设置到仿真数据中，仿真数据需提前初始化好，代码如下：

```
/应用仿真设置数据
MwSimData *sim_data = new
MwSimData(L"MassSpringDamper",data_path.toStdWString(), path.toStdWString());
bool flag = sim_data->InitializeSimInst();

MwExperimentData *experiment_data = new MwExperimentData;
experiment_data->stopTime = ui.lineEdit_stopTime->text().toDouble();
sim_data->ApplyExperimentData(experiment_data);
```

C. 编译和仿真

在仿真前需要提前翻译模型，在构造函数中提前 new 出用于仿真 MwSimControl 类，并将该类与用于仿真的数据进行绑定，即可开始进行仿真，代码如下：

```
/仿真前翻译模型
classMgr->GetMoHandler()->CompileModel("MassSpringDamper.Demo",path.toStdWString
0);

//仿真模型
simCtrl->RebindSimData(sim_data);
bool is_success = simCtrl->StartSimulate(MwSimControl::Sim_ContinueMode);
```

2.4.3.3 仿真完成

使用 SigSimStopped 信号用于接收仿真完成，代码如下：

```
connect(simCtrl, &MwSimControl::SigSimStopped, this, &MainWindow::SlotSimFinshed);
```

仿真完成后，主要功能为获取仿真结果数据，并将其显示到曲线窗口中，仿

真效果如下：

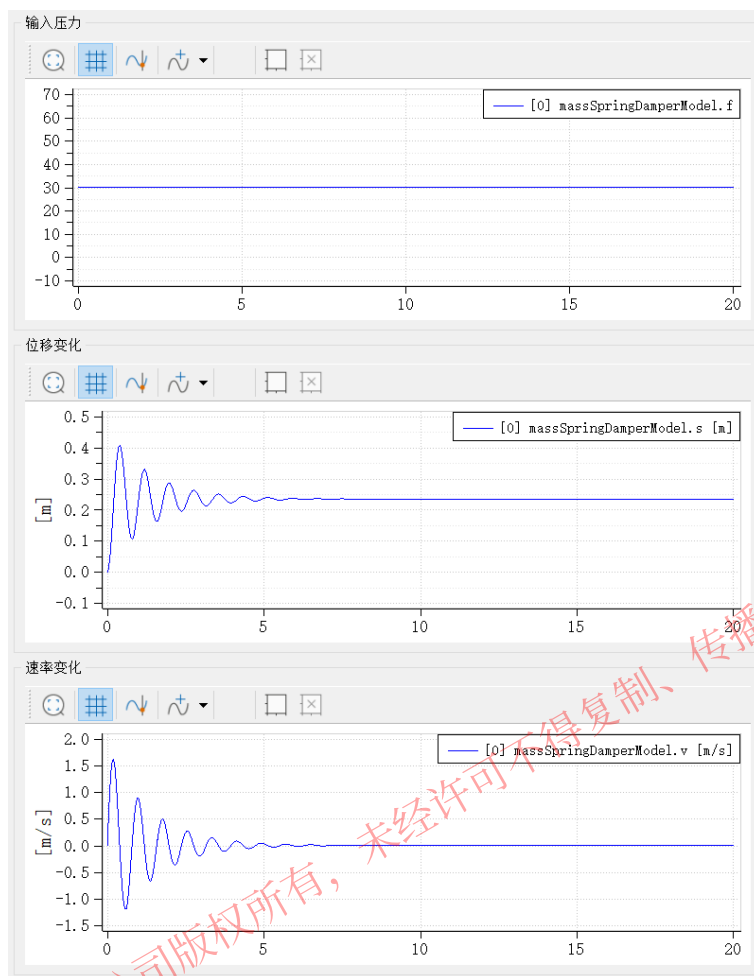


图 2-24 仿真结果曲线

主要代码如下：

```
MwSimData* sim_data = simCtrl->GetSimData();
forcePlotWin->AddCurveToCurrentView("massSpringDamperModel.f", sim_data);
sPlotWin->AddCurveToCurrentView("massSpringDamperModel.s", sim_data);
vPlotWin->AddCurveToCurrentView("massSpringDamperModel.v", sim_data);
```

2.5 APP 运行及打包

将生成的 exe 及依赖的 dll 放入“SDK 安装目录/bin/win_msvc2017x64/Release”目录下：

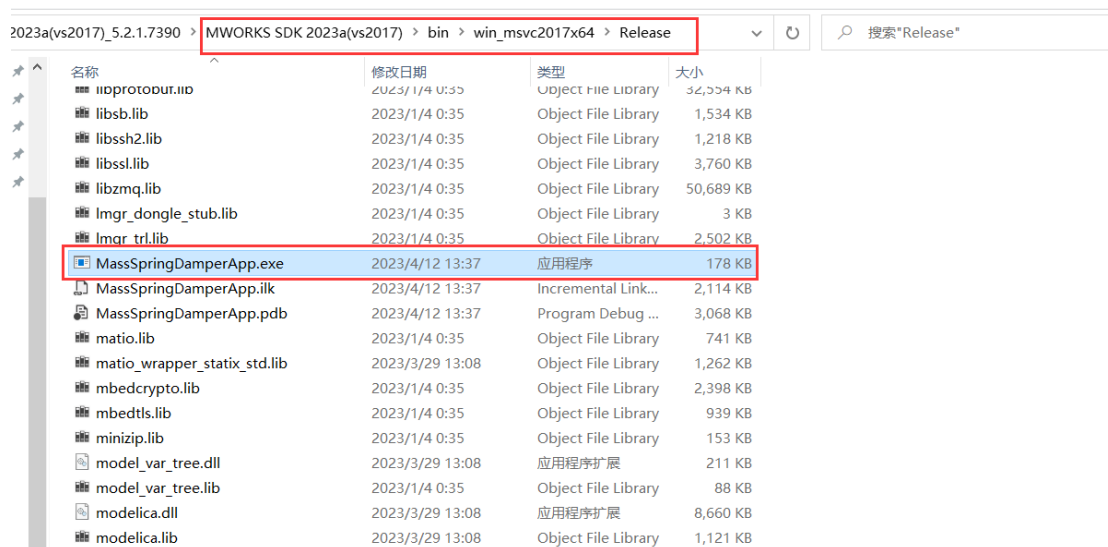


图 2-25 App 运行路径

双击“MassSpringDamperApp.exe”即可启动软件。放入 exe 后可对软件实现打包，打包步骤如下：

4. 将 Release 改为 bin 名称；
5. 保存 bin\win_msvc2017x64 目录下的以下目录：
 - external
 - initial_files
 - Library
 - bin(原 Release 文件)
 - setting
 - simulator
 - tools
6. 最终打包目录如下所示；

名称	修改日期	类型
external	2022/12/12 15:30	文件夹
initial_files	2022/12/12 15:30	文件夹
Library	2023/3/13 9:29	文件夹
Release	2023/3/13 10:33	文件夹
setting	2022/12/12 15:30	文件夹
simulator	2022/12/12 15:30	文件夹
tools	2022/12/12 15:30	文件夹

图 2-26 App 打包目录

2.6 案例所用类和接口汇总

该质量-弹簧-阻尼模型 APP 案例中使用到的 SDK 相关类和接口如下：

表 2-1 接口汇总

类	接口	注释
MwClassManager	Initialize()	MwClassManager 初始化
	GetCoreOption()	获取 MwCoreOption 类
	GetMoHandler()	获取 MwMoHandler 类
MwCoreOption	LoadChineseTranslateFile()	加载中文翻译文件
MwMoHandler	LoadMoLibrary()	加载模型库
	OpenFile()	打开模型文件
	GetParamValue()	获取参数值
	GetKeyByTypeName()	根据模型名获取模型 key
	SetParamValue()	设置参数值
	CompileModel()	编译模型
MwSimControl	SigSimStopped()	发送仿真停止信号
	RebindSimData()	绑定仿真数据
	StartSimulate()	开始仿真
	GetSimData()	获取仿真结果数据
MwSimData	InitializeSimInst()	初始化仿真数据
	ApplyExperimentData()	应用仿真设置
MwSimPlotWindow	AddCurveToCurrentView()	增加曲线数据到曲线窗口
MwExperimentData		仿真设置结构体

3. 综合案例介绍

3.1 C++版 App

3.1.1 车辆仿真 APP

3.1.1.1 案例介绍

车辆 APP 用于车辆的设计验证, 学生可通过修改车辆的参数进行车辆设计, 可通过仿真车辆行驶性能进行设计验证, 即通过加载车辆模型库 (简化版), 选择对应的型号的车辆, 通过修改车辆属性, 包括车辆类型、尺寸、重量、动力等特征, 实现车辆的设计; 通过仿真计算模拟车辆在道路上运行情况, 得到车辆运行性能数据, 包括车辆的加速度、悬挂系统的振动情况、轮胎的压力分布等, 基于性能数据分析车辆设计的合理性、优越性。

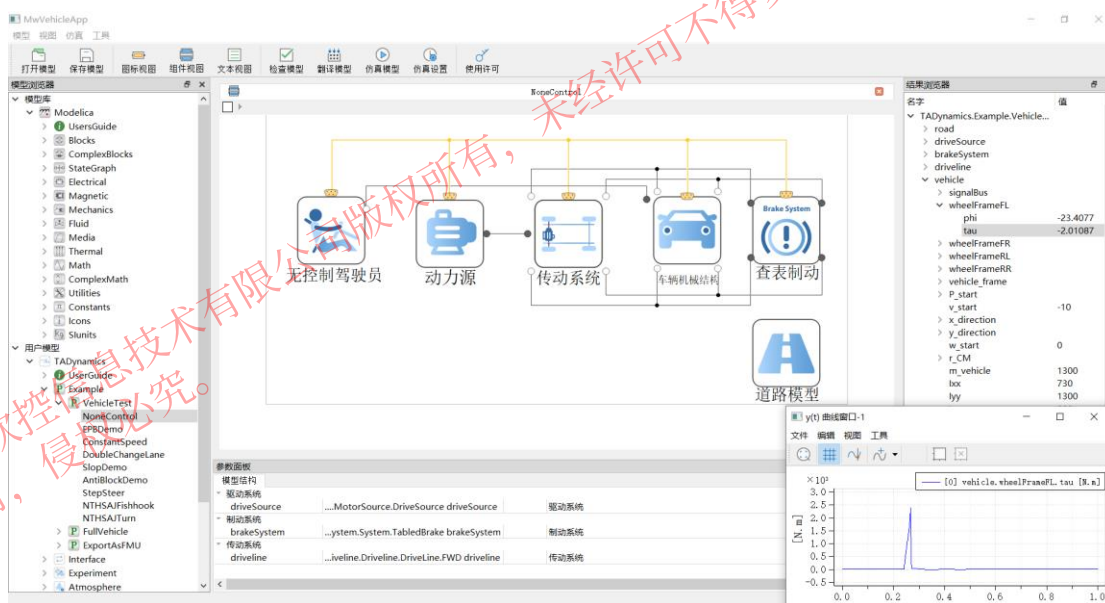


图 3-1 车辆 APP

车辆 APP 主窗口如图所示, 主要由五个部分组成, 分别是中间的模型视图, 左侧的模型浏览器, 底部的参数面板, 右侧的仿真结果浏览器以及显示变量曲线的曲线窗口。

本车辆模型依赖 Modelica 库版本修改为 3.2.3, 打开 TADynamics 模型库和 TYBase 模型库。

3.1.1.2 APP 开发

3.1.1.2.1 搭建框架

```
#include "mw_vehicle_app.h"

int main(int argc, char *argv[])
{
    MwVehicleApp mw_app(argc, argv);
    mw_app.Initialize();
    return mw_app.Exec();
}
```

程序入口创建一个 MwVehicleApp 实例，调用 Initialize 初始化后调用 Exec 开启车辆 APP。

A. MwVehicleApp

MwVehicleApp 作为车辆 APP 应用程序和 Qt 事件循环的入口，继承 QApplication，负责初始化使用 SDK 开发 APP 依赖的一些必要的环境以及创建车辆 APP 主窗口。

```
void MwVehicleApp::Initialize()
{
    classManager = new MwClassManager();
    classManager->Initialize();
    LoadChineseTranslateFile();

    pLicenseService = new MwLicenseService(nullptr);
    pLicenseService->Initialize();

    taskManager = new MwVehicleTaskManager();
    simManager = new MwVehicleSimManager();
    mainWin = new MwVehicleMainWindow();

    mainWin->showMaximized();
}
```

其中 LoadChineseTranslateFile 为加载 SDK 的中文翻译，调用 MwCoreOption 的 LoadChineseTranslateFile 接口。

```
void MwVehicleApp::LoadChineseTranslateFile()
{
    ClassMgrPtr->GetCoreOption()->LoadChineseTranslateFile(this);
}
```

B. MwVehicleMainWindow

MwVehicleMainWindow 为车辆 APP 主窗口，继承 QMainWindow，负责如下事务：

- (1). 提供各功能的按钮及接口实现，包括打开模型、保存模型、切换模型视图、检查模型、翻译模型、仿真模型、仿真设置和使用许可。
- (2). 构建各个面板组件，包括中央模型视图、左侧模型浏览器、底部参数面板、右侧仿真结果浏览器。

初始化界面前需要先创建 MwMoGraphicsViewController 实例，其负责同步各个面板因模型变化而产生的界面刷新。

```
void MwVehicleMainWindow::SetupUI()
{
    moController = new MwMoGraphicsViewController(ClassMgrPtr);

    mdiModelView = new MwMoWindowMdi(moController, this);
    moController->SetMdiInterface(mdiModelView);
    centralLayout->addWidget(mdiModelView, 0, 0, 1, 1);

    dockClassBrowser = new MwVehicleClassBrowserPanel(
        QStringLiteral("模型浏览器"), this);
    dockParamBrowser = new MwVehicleParamBrowserPanel(
        moController, QStringLiteral("参数面板"), this);
    dockResultBrowser = new MwVehicleResultBrowserPanel(
        QStringLiteral("结果浏览器"), this);

    this->addDockWidget(Qt::LeftDockWidgetArea, dockClassBrowser);
    this->addDockWidget(Qt::BottomDockWidgetArea, dockParamBrowser);
    this->addDockWidget(Qt::RightDockWidgetArea, dockResultBrowser);
    this->setCorner(Qt::BottomLeftCorner, Qt::LeftDockWidgetArea);
    this->setCorner(Qt::BottomRightCorner, Qt::RightDockWidgetArea);

    window()->restoreGeometry(
        QSettings().value("windowGeometry").toByteArray());
}
```

通过连接 MwMoGraphicsViewController 的 SigUpdate 信号和各面板的 SlotUpdate 槽函数来同步界面更新。

```
void MwVehicleMainWindow::InitConnections()
{
    connect(moController, &MwMoGraphicsViewController::SigUpdate,
        mdiModelView, &MwMoWindowMdi::SlotUpdate);
}
```

```

connect(moController, &MwMoGraphicsViewController::SigUpdate,
        dockClassBrowser->GetClassBrowser()->GetClassBrowserTreeModel(),
        &MwMoClassTreeModel::SlotUpdate);
connect(moController, &MwMoGraphicsViewController::SigUpdate,
        dockParamBrowser->GetParamBrowser(),
        &MwModelParameterTabWidget::SlotUpdate);
}

```

C. MwMoWindowMdi

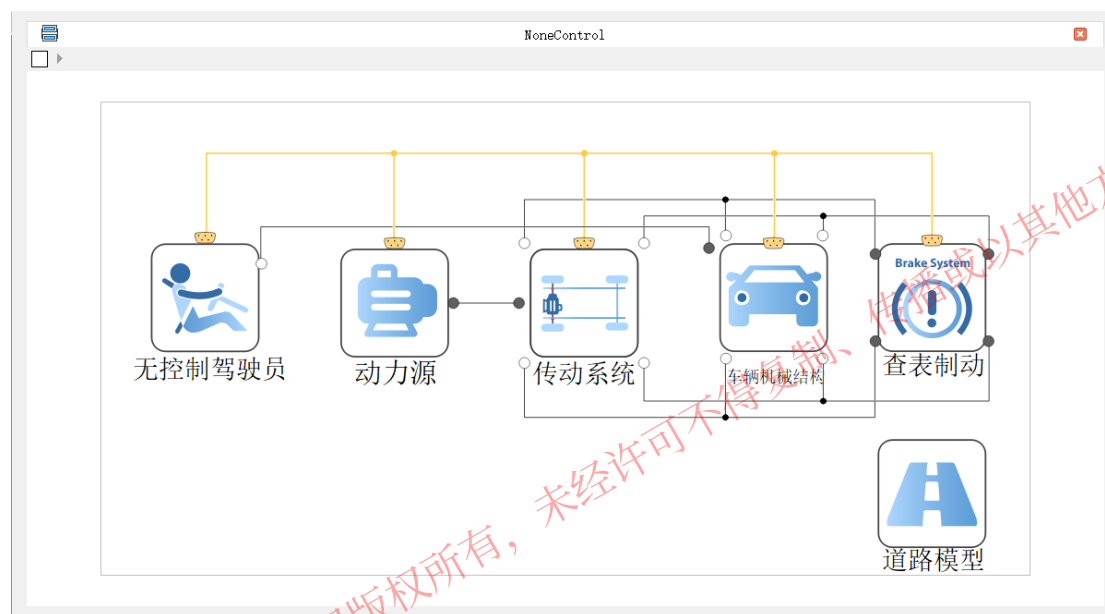


图 3-2 中央模型视图

MwMoWindowMdi 为 SDK 提供的模型视图组件，基类为 QMdiArea，支持显示组件的图标视图、组件视图和文本视图，并支持编辑。

使用时注意创建对象后，调用 MwMoGraphicsViewController 的 SetMdiInterface 接口将 MwMoWindowMdi 设置进去。

```

mdiModelView = new MwMoWindowMdi(moController, this);
moController->SetMdiInterface(mdiModelView);

```

D. MwVehicleClassBrowserPanel

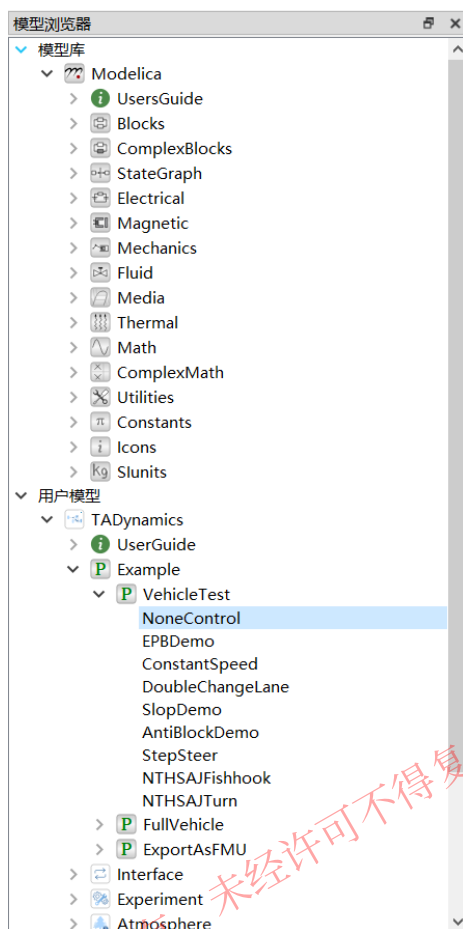


图 3-3 模型浏览器

MwVehicleClassBrowserPanel 继承 QDockWidget, 为模型浏览器的面板容器, 其内部创建 MwVehicleClassBrowser。

```
classBrowser = new MwVehicleClassBrowser(this);  
this->setWidget(classBrowser);
```

MwVehicleClassBrowser 继承 QTreeView, 用于显示模型树, 其内部创建 MwMoClassTreeModel, 并将其设置为模型浏览器的数据模型。

```
treeModel = new MwMoClassTreeModel(ClassMgrPtr);  
this->setModel(treeModel);
```

MwMoClassTreeModel 基类为 QStandardItemModel, 是 SDK 提供的模型浏览器数据模型, 负责构造内核加载的模型对应的上层数据树状结构, 设置到 QTreeView 中使用。

E. [MwVehicleParamBrowserPanel](#)

参数面板			
常规	右前制动器参数	左前制动器参数	右后制动器参数
参数			
esc_active	true		true: 激活ESC功能
FMax	1e4		最大制动力
k_vy	1		车辆横向速度比重系数
k_wz	10		车辆横摆角速度比重系数
steer_eps	0.001		方向盘转角临界值
SteeringGain	1 / 23		方向盘转角与车轮转角比值
a1	1.3368		质心离前轴距离
a2	1.4282		质心离后轴距离
m	1482		整车质量
Izz	800		车辆绕z轴转动惯量
D1	65000		前轮侧偏刚度 (单轮胎)
D2	55000		后轮侧偏刚度 (单轮胎)

图 3-4 参数面板

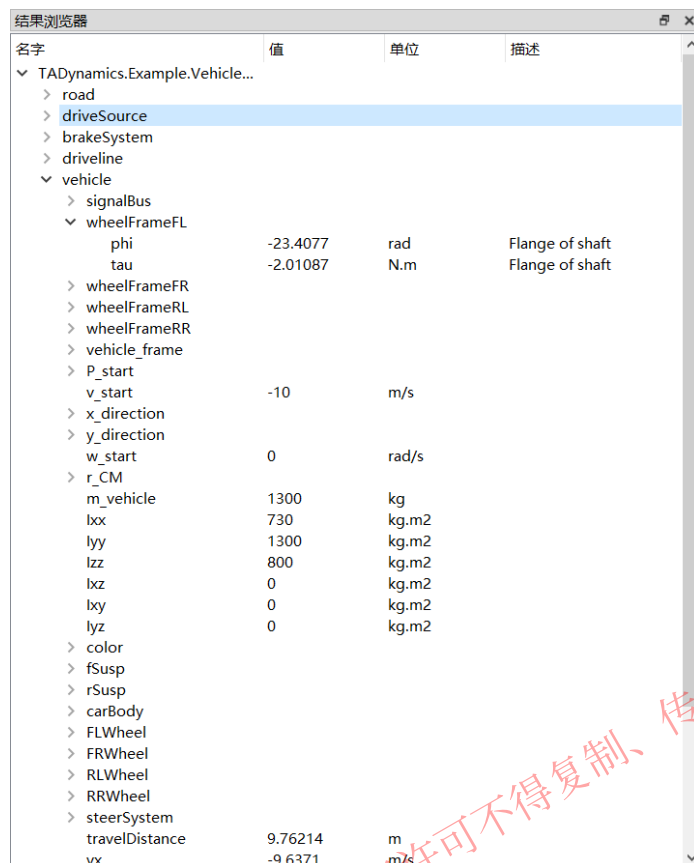
MwVehicleParamBrowserPanel 继承 QDockWidget，为参数面板的面板容器，其内部创建 MwModelParameterTabWidget。

```
paramBrowser = new MwModelParameterTabWidget(
    mo_controller, MwParamEditMode::PEM_Panel, this);
this->setWidget(paramBrowser);
```

MwModelParameterTabWidget 为 SDK 提供的参数面板组件，其可以通过连接 MwMoGraphicsViewController 的 SigUpdate 信号，实现对当前选中组件的参数显示和编辑。

F. [MwVehicleResultBrowserPanel](#)

苏州同元软控信息技术有限公司版权所有，侵权必究。



名字	值	单位	描述
TADynamics.Example.Vehicle...			
> road			
> driveSource			
> brakeSystem			
> driveline			
> vehicle			
> signalBus			
> wheelFrameFL			
phi	-23.4077	rad	Flange of shaft
tau	-2.01087	N.m	Flange of shaft
> wheelFrameFR			
> wheelFrameRL			
> wheelFrameRR			
> vehicle_frame			
> P_start			
> v_start	-10	m/s	
> x_direction			
> y_direction			
> w_start	0	rad/s	
> r_CM			
> m_vehicle	1300	kg	
> lxx	730	kg.m2	
> lyy	1300	kg.m2	
> lzz	800	kg.m2	
> lxz	0	kg.m2	
> lxy	0	kg.m2	
> lyz	0	kg.m2	
> color			
> fSusp			
> rSusp			
> carBody			
> FLWheel			
> FRWheel			
> RLWheel			
> RRWheel			
> steerSystem			
> travelDistance	9.76214	m	
> vx	-9.6371	m/s	

图 3-5 结果浏览器

MwVehicleResultBrowserPanel 继承 QDockWidget，为结果浏览器的面板容器，其内部创建 MwVehicleResultBrowser。

```
resultBrowser = new MwVehicleResultBrowser(this);  
this->setWidget(resultBrowser);
```

MwVehicleResultBrowser 继承 QTreeWidget，用于显示仿真结果变量树。

3.1.1.2.2 功能实现

接下来将针对单个功能分析如何使用 SDK 完成应用级功能的开发，注意以下功能默认上述框架均已完成搭建。

加载模型库

MwVehicleMainWindow 中实现了加载系统模型库的接口 LoadLibrary，接口中创建一个 MwVehicleLoadLibraryTask 线程进行模型库加载。

```
void MwVehicleMainWindow::LoadLibrary(const QString &lib_name, const QString &lib_v  
er)  
{  
    if (TaskMgrPtr->IsTaskRunning())  
    {
```

```

        ShowStatus(QStringLiteral("当前有任务正在进行，无法加载模型库。"));
        return;
    }

    if (ClassMgrPtr->GetMoHandler()->GetKeyByTypeName(lib_name.toStdString()))
    {
        ShowStatus(QStringLiteral("模型库已加载。"));
        return;
    }

    ShowStatus(QStringLiteral("正在加载模型库") + lib_name + " " + lib_ver);
    EnableUI(false, true);
    MwVehicleLoadLibraryTask *load_library_task = new MwVehicleLoadLibraryTask(lib_name, lib_ver);
    TaskMgrPtr->Execute(load_library_task);
}

```

MwVehicleLoadLibraryTask 继承 QThread，负责在线程中执行加载模型库的操作，通过调用 SDK 提供的 MwMoHandler 中的 LoadMoLibrary 接口实现。

```

void MwVehicleLoadLibraryTask::run()
{
    loadSuccess = ClassMgrPtr->GetMoHandler()->LoadMoLibrary(libName.toStdString(), libVersion.toStdString());
    emit SigLoadLibraryFinish();
}

```

模型库加载成功后，触发 MwVehicleMainWindow 中的 SlotLoadLibraryFinish 槽函数，调用 MwVehicleClassBrowser 中的 AddLibrary 接口更新模型浏览器界面，显示模型树。

```

void MwVehicleMainWindow::SlotLoadLibraryFinish(QThread *thread)
{
    MwVehicleLoadLibraryTask *load_lib_task = dynamic_cast<MwVehicleLoadLibraryTask*>(thread);
    if (load_lib_task != nullptr && load_lib_task->IsLoadLibrarySuccess())
    {
        QString lib_name;
        load_lib_task->GetLibName(lib_name);
        dockClassBrowser->GetClassBrowser()->AddLibrary(lib_name);
    }
}

```

AddLibrary 接口调用 MwMoClassTreeModel 的 AppendTopClass 接口实现模型库树结构的显示。

```
void MwVehicleClassBrowser::AddLibrary(const QString &lib_name)
{
    clearSelection();
    MWInt mo_key = ClassMgrPtr->GetMoHandler()->GetKeyByTypeName(lib_name.toStdString());
    treeModel->AppendTopClass(mo_key, QStringLiteral("模型库"));
    UpdateModelIcon(mo_key);
    this->expand(treeModel->GetItem(mo_key)->index());
}
```

接口中的 UpdateModelIcon 为更新模型树中指定模型及其嵌套模型的图标，调用 MwMoGraphicsViewController 中的 GetIcon 接口获取模型图标。

```
void MwVehicleClassBrowser::UpdateModelIcon(MWInt key)
{
    QStandardItem *item = treeModel->GetItem(key);
    if (item == nullptr)
    {
        return;
    }
    MwMoGraphicsViewController *mo_controller = AppPtr->GetMainWindow()->GetMoGraphicsViewController();
    item->setIcon(mo_controller->GetIcon(key, QSize(20, 20)));
    for (int i = 0; i < item->rowCount(); ++i)
    {
        UpdateNestedModelIcon(item->child(i, 0));
    }
}
```

打开模型

MwVehicleMainWindow 中实现了打开模型的接口 SlotOpenModel，接口中创建一个 MwVehicleOpenFileTask 线程来打开模型文件。

```
void MwVehicleMainWindow::SlotOpenModel()
{
    if (TaskMgrPtr->IsTaskRunning())
    {
        ShowStatus(QStringLiteral("当前有任务正在进行，无法打开模型。"));
        return;
    }

    QString str_file = QFileDialog::getOpenFileName(this, QStringLiteral("打开模型文件"), "", tr("*.mo"));
    if (str_file.isEmpty())
```

```

{
    return;
}

if (ClassMgrPtr->GetMoHandler()->GetTopClassInFile(str_file.toStdWString()))
{
    ShowStatus(QStringLiteral("模型已加载。"));
    return;
}

ShowStatus(QStringLiteral("正在打开模型") + str_file);
EnableUI(false, true);
MwVehicleOpenFileTask *open_file_task = new MwVehicleOpenFileTask(str_file);
TaskMgrPtr->Execute(open_file_task);
}

```

MwVehicleOpenFileTask 继承 Qthread，负责在线程中执行打开模型文件的操作，通过调用 SDK 的 MwMoHandler 中的 OpenFile 接口实现。

```

void MwVehicleOpenFileTask::run()
{
    openSuccess = ClassMgrPtr->GetMoHandler()->OpenFile(filePath.toStdWString());
    emit SigOpenFileFinish();
}

```

模型文件加载成功后，触发 MwVehicleMainWindow 中的 SlotOpenModelFinish 槽函数，调用 MwVehicleClassBrowser 中的 AddUserModel 接口更新模型浏览器界面，显示模型树，并且调用 MwMoWindowMdi 的 OpenMoWindow 接口，在模型视图中显示打开的模型。

```

void MwVehicleMainWindow::SlotOpenModelFinish(QThread *thread)
{
    MwVehicleOpenFileTask *open_file_task = dynamic_cast<MwVehicleOpenFileTask*>(thread);
    if (open_file_task != nullptr && open_file_task->IsOpenFileSuccess())
    {
        MWint mo_key = open_file_task->GetTopClassKeyInFile();
        dockClassBrowser->GetClassBrowser()->AddUserModel(mo_key);
        mdiModelView->OpenMoWindow(mo_key, true, MwModelicaLayer::MLAYER_DIAGRAM);
    }
}

```

AddUserModel 接口调用 MwMoClassTreeModel 的 AppenTopClass 接口实现用户模型树结构的显示，并调用 UpdateModelIcon 接口更新模型图标。

```
void MwVehicleClassBrowser::AddUserModel(MWint mo_key)
{
    clearSelection();
    treeModel->AppendTopClass(mo_key, QStringLiteral("用户模型"));
    UpdateModelIcon(mo_key);
    this->expand(treeModel->GetItem(mo_key)->index());
}
```

卸载模型

MwVehicleMainWindow 中实现了卸载模型的接口 SlotUnloadModel，接口中调用 MwVehicleClassBrowser 中的 UnloadModel 接口卸载模型树，然后调用 MoWindowMdi 的 CloseMoWindow 关闭模型窗口，最后调用 MwMoHandler 的 UnloadModel 接口卸载底层模型数据。

```
void MwVehicleMainWindow::SlotUnloadModel()
{
    if (TaskMgrPtr->IsTaskRunning())
    {
        ShowStatus(QStringLiteral("当前有任务正在进行，无法卸载模型。"));
        return;
    }

    MWint mo_key = dockClassBrowser->GetClassBrowser()->GetCurrentClassKey();
    QString mo_name =
QString::fromStdString(ClassMgrPtr->GetMoHandler()->GetFullnameProp(mo_key));
    ShowStatus(QStringLiteral("正在卸载模型") + mo_name);

    dockClassBrowser->GetClassBrowser()->UnloadModel(mo_key);
    mdiModelView->CloseMoWindow(mo_key);
    if (ClassMgrPtr->GetMoHandler()->UnloadModel(mo_name.toStdString()))
    {
        ShowStatus(QStringLiteral("模型") + mo_name + QStringLiteral("卸载成功。"));
    }
    else
    {
        ShowStatus(QStringLiteral("模型") + mo_name + QStringLiteral("卸载失败。"));
    }
}
```

MwVehicleClassBrowser 的 UnloadModel 接口调用 MwMoClassTreeModel 的 RemoveClass 接口，卸载模型树。

```
void MwVehicleClassBrowser::UnloadModel(MWint mo_key)
```

```
{
    treeModel->RemoveClass(mo_key);
}
```

保存模型

MwVehicleMainWindow 中实现了保存模型的接口 SlotSaveModel，接口首先调用 MwMoWindowMdi 的 GetCurrentClassKey 获取当前模型窗口的模型键值，然后调用 MwMoHandler 的 GetTopClassFileByKey 获取当前模型的顶层类键值，以此顶层模型为保存的对象。接着调用 MwMoWindowMdi 的 SaveCurrentWindow 接口以及 MwMoHandler 的 SaveModel 接口保存窗口状态和底层模型文件，最后调用 MwMoGraphicsViewController 的 SetClassDirty 移除顶层模型的脏标志。

```
void MwVehicleMainWindow::SlotSaveModel()
{
    if (TaskMgrPtr->IsTaskRunning())
    {
        ShowStatus(QStringLiteral("当前有任务正在进行，无法保存模型。"));
        return;
    }

    MWinT class_key = mdiModelView->GetCurrentClassKey();
    if (class_key == 0)
    {
        return;
    }

    QString mo_name =
        QString::fromStdString(ClassMgrPtr->GetMoHandler()->GetFullnameProp(class_key));
    MWinT top_class_key =
        ClassMgrPtr->GetMoHandler()->GetTopClassInFileByKey(class_key);
    ShowStatus(QStringLiteral("正在保存模型") + mo_name);
    if (mdiModelView->SaveCurrentWindow() &&
        ClassMgrPtr->GetMoHandler()->SaveModel(ClassMgrPtr->GetMoHandler()->GetFullnameProp(top_class_key)) == MWStat_Ok)
    {
        moController->SetClassDirty(top_class_key, false);
        ShowStatus(QStringLiteral("模型") + mo_name + QStringLiteral("保存成功。"));
    }
    else
    {
        ShowStatus(QStringLiteral("模型") + mo_name + QStringLiteral("保存失败。"));
    }
}
```

```
}  
}
```

切换模型视图

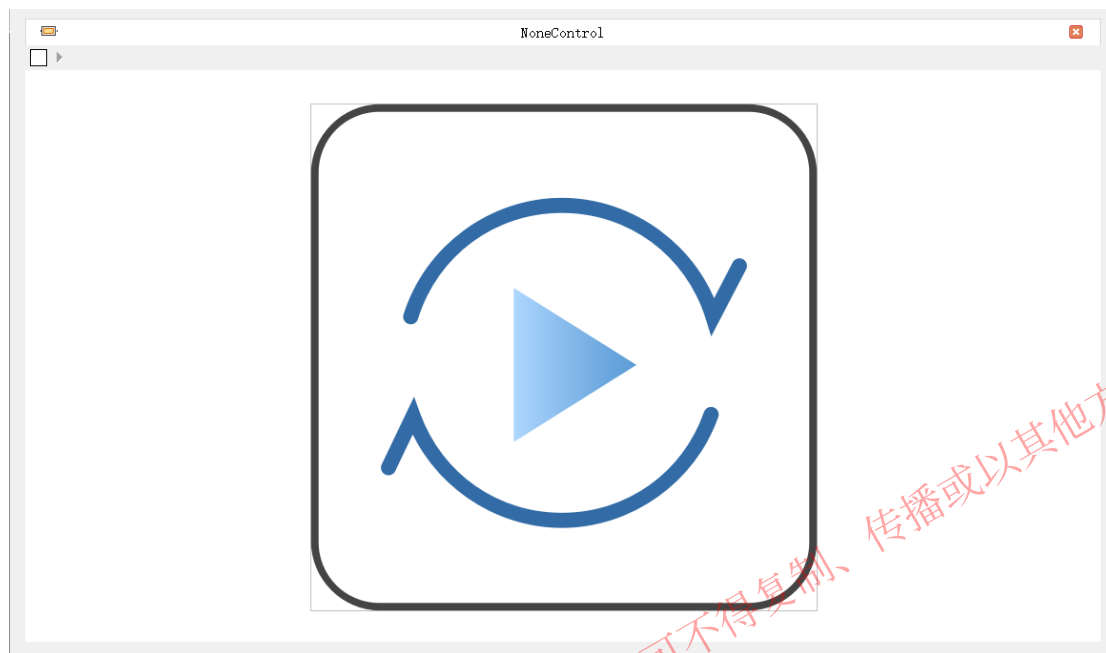


图 3-6 图标视图

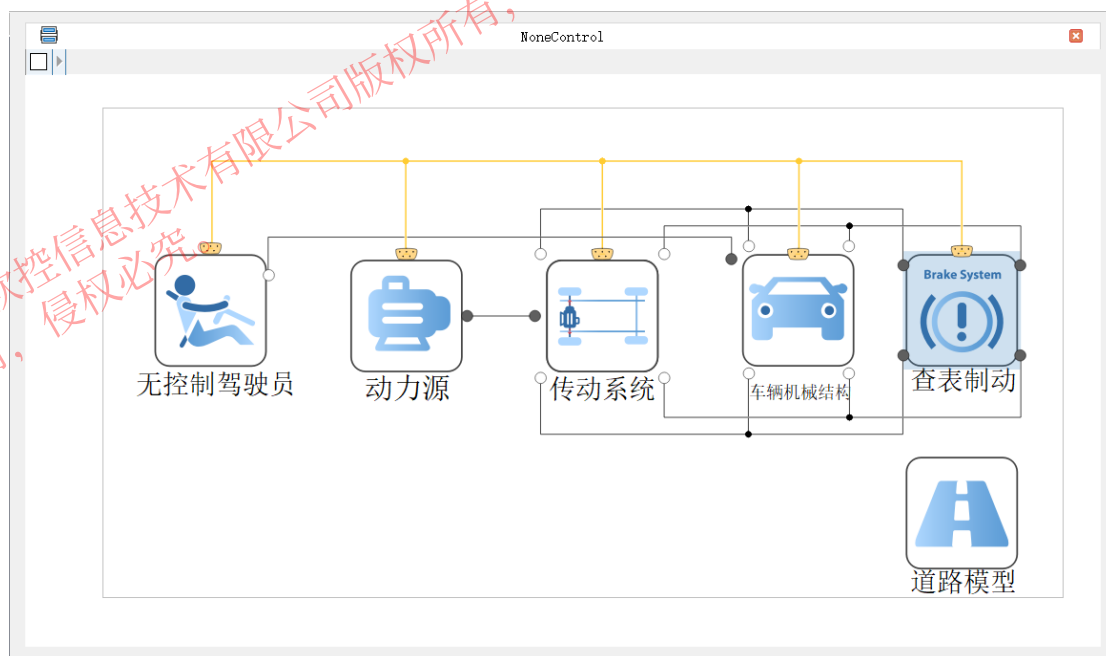


图 3-7 组件视图

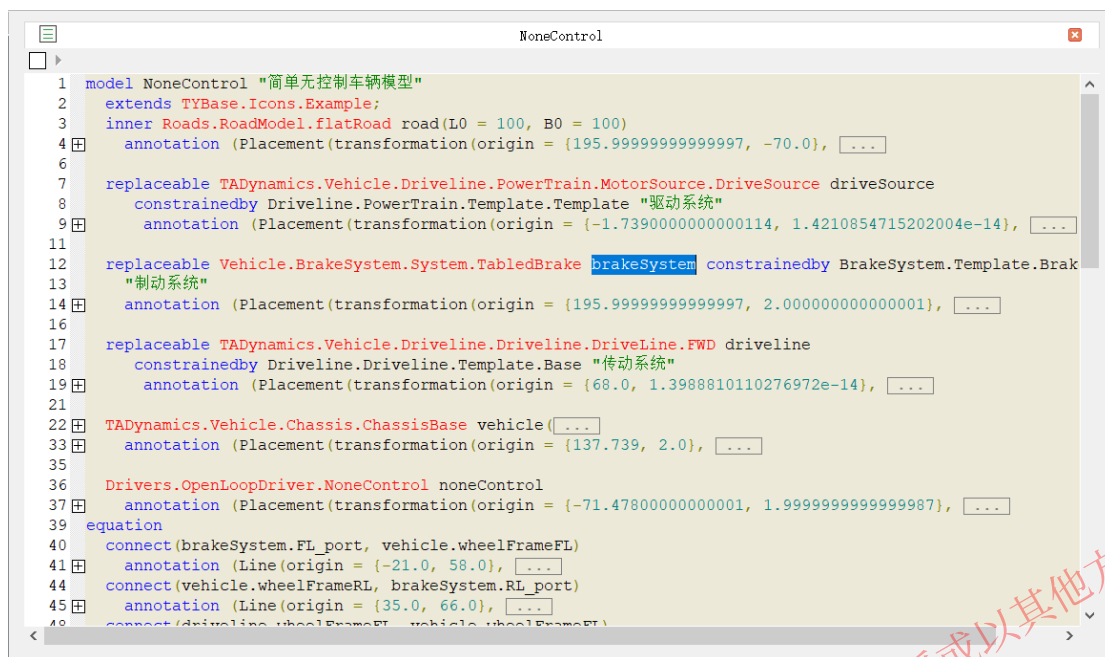


图 3-8 文本视图

中央模型视图可以切换三种视图类型，分别是图标视图，组件视图和文本视图。

图标视图可以查看和编辑当前模型的图标。

组件视图可以查看模型的结构和连接关系，并通过鼠标拖拽的方式修改模型结构和连接。

文本视图可以查看模型的文本（基于 Modelica 建模语言），支持编辑文本，检查语法是否正确。

MwVehicleMainWindow 中实现了切换中央视图类型的接口 SlotSwitchView，接口调用 MwMoWindowMdi 的 SwitchCurrentMoView 接口实现视图类型切换。

```
void MwVehicleMainWindow::SlotSwitchView()
{
    QAction *action_sender = qobject_cast<QAction*>(sender());
    if (action_sender == action2IconView)
    {
        mdiModelView->SwitchCurrentMoView(MLAYER_ICON);
    }
    else if (action_sender == action2DiagramView)
    {
        mdiModelView->SwitchCurrentMoView(MLAYER_DIAGRAM);
    }
    else if (action_sender == action2TextView)
    {

```

```
mdiModelView->SwitchCurrentMoView(MLAYER_TEXT);
}else{}
}
```

检查模型

MwVehicleMainWindow 中实现了检查模型语法语义是否正确的模型检查接口 SlotCheckModel，接口中创建一个 MwVehicleCheckModelTask 线程进行模型检查。

```
void MwVehicleMainWindow::SlotCheckModel()
{
    if (TaskMgrPtr->IsTaskRunning())
    {
        ShowStatus(QStringLiteral("当前有任务正在进行，无法检查模型。"));
        return;
    }

    QString model_name = QString::fromStdString(moController->GetCurrentClassName());
    if (model_name.isEmpty())
    {
        return;
    }

    ShowStatus(QStringLiteral("正在检查模型") + model_name);
    EnableUI(false, true);
    MwVehicleCheckModelTask *check_model_task = new MwVehicleCheckModelTask(model_name);
    TaskMgrPtr->Execute(check_model_task);
}
```

MwVehicleCheckModelTask 继承 QThread，负责在线程中执行检查模型的操作，通过调用 SDK 的 CheckModel 接口实现。

```
void MwVehicleCheckModelTask::run()
{
    checkSuccess = ClassMgrPtr->GetMoHandler()->CheckModel(modelName.toStdString());
    emit SigCheckModelFinish();
}
```

翻译模型

MwVehicleMainWindow 中实现了翻译模型的接口 CompileModel，接口中创建一个 MwVehicleCompileModelTask 线程进行模型翻译。

```
void MwVehicleMainWindow::CompileModel(bool translate_and_sim)
```

```

{
    if (TaskMgrPtr->IsTaskRunning())
    {
        ShowStatus(QStringLiteral("当前有任务正在进行，无法翻译模型。"));
        return;
    }

    QString model_name = QString::fromStdString(moController->GetCurrentClassName());
    if (model_name.isEmpty())
    {
        return;
    }

    ShowStatus(QStringLiteral("正在翻译模型") + model_name);
    dockResultBrowser->GetResultBrowser()->CloseAllPlotWindow();
    EnableUI(false, true);
    MwVehicleCompileModelTask *compile_model_task = new MwVehicleCompileModelTask(
        model_name, translate_and_sim);
    TaskMgrPtr->Execute(compile_model_task);
}

```

MwVehicleCompileModelTask 继承 QThread，负责在线程中创建仿真实例目录并调用 MwMoHandler 的 CompileModel 接口翻译模型。

```

void MwVehicleCompileModelTask::run()
{
    SimMgrPtr->CreateSimInstDir(modelName);
    SimMgrPtr->GetSimInstDir(modelName, simInstDir);
    compileSuccess = ClassMgrPtr->GetMoHandler()->CompileModel(modelName.toStdString(),
        simInstDir.toStdWString());
    emit SigCompileModelFinish();
}

```

仿真模型

MwVehicleMainWindow 中实现了仿真模型的接口 StartSimulate，接口调用 MwVehicleSimManager 的 SimulateModel 接口实现仿真。

```

void MwVehicleMainWindow::StartSimulate()
{
    if (TaskMgrPtr->IsTaskRunning())
    {
        ShowStatus(QStringLiteral("当前有任务正在进行，无法仿真模型。"));
        return;
    }
}

```

```

QString model_name = QString::fromStdString(moController->GetCurrentClassName());
if (model_name.isEmpty())
{
    return;
}

ShowStatus(QStringLiteral("正在仿真模型") + model_name);
EnableUI(false, true);
SimMgrPtr->SimulateModel(model_name);
}

```

SimulateModel 接口首先将先前与 MwSimControl 绑定的 MwSimData 销毁，并创建新的 MwSimData 与之绑定，然后调用 MwSimData 的 InitializeSimInst 初始化仿真实例，并调用 MwSimData 的 ApplyExperimentData 应用仿真设置。最后调用 SimControl 的 StartSimualte 启动仿真。

```

bool MwVehicleSimManager::SimulateModel(const QString &model_name)
{
    modelName = model_name;
    MwSimData *sim_data = simCtrl->GetSimData();
    if (sim_data)
    {
        delete sim_data;
        simCtrl->RebindSimData(nullptr);
    }

    QString result_path, inst_path;
    GetSimInstDir(model_name, inst_path);
    GetSimResultPath(model_name, result_path);
    sim_data = new MwSimData(model_name.toStdWString(),
                             result_path.toStdWString(),
                             inst_path.toStdWString());
    sim_data->InitializeSimInst();
    sim_data->ApplyExperimentData(expData);

    simCtrl->RebindSimData(sim_data);
    return simCtrl->StartSimulate(MwSimControl::Sim_ContinueMode);
}

```

仿真设置

MwVehicleMainWindow 中实现了仿真设置的接口 SlotSimConfig，接口创建

一个 MwVehicleSimConfigDialog 对话框并弹出。

```
void MwVehicleMainWindow::SlotSimConfig()
{
    MwVehicleSimConfigDialog sim_config_dlg(SimMgrPtr->GetExperimentData(), this);
    sim_config_dlg.exec();
}
```

MwVehicleSimConfigDialog 中创建一个 SDK 提供的 MwSimConfigWidget 仿真设置组件，并将其设置到对话框内显示。

```
void MwVehicleSimConfigDialog::SetupUi()
{
    centralLayout = new QVBoxLayout(this);
    simConfigWgt = new MwSimConfigWidget(expData, this);
    centralLayout->addWidget(simConfigWgt);
}
```

加载仿真结果

MwVehicleResultBrowser 中实现了加载仿真结果的接口 LoadSimData，接口加载仿真结果并构造一棵变量树。

```
void MwVehicleResultBrowser::LoadSimData(MwSimData *sim_data)
{
    this->clear();
    popro::MwVarTree* var_root = sim_data->GetVarTreeRoot();
    if (var_root == nullptr)
        return;

    QTreeWidgetItem* top_item = new QTreeWidgetItem();
    top_item->setText(0, QString::fromStdString(var_root->Name()));
    top_item->setData(0, Qt::UserRole, QVariant::fromValue(QString::fromStdString(var_root->GetFullName())));
    this->addTopLevelItem(top_item);

    AppendChildItem(sim_data, top_item, var_root);

    this->expandToDepth(0);
}
```

变量树的数据结构通过 MwSimData 中的 MwVarTree 获得，MwVarTree 中还包含有变量简名、全名、描述、单位、值等信息。

显示变量曲线

MwVehicleResultBrowser 中实现了显示变量曲线的接口 PlotCurve，接口创建

一个 MwSimPlotWindow 并添加曲线到曲线窗口中显示。

```
void MwVehicleResultBrowser::PlotCurve(QTreeWidgetItem *item)
{
    QString var_name = item->data(0, Qt::UserRole).value<QString>();
    MwSimPlotWindow *plot_win = new MwSimPlotWindow(1, true, ClassMgrPtr, this, false);
    vecPlotWin.append(plot_win);
    connect(plot_win, &MwSimPlotWindow::SigWindowClosed, this, &MwVehicleResultBrowser::SlotWindowClosed);
    plot_win->setAttribute(Qt::WA_DeleteOnClose);
    MwAbstractData *abs_data = SimMgrPtr->GetCurrentSimData();
    plot_win->AddCurveToCurrentView(var_name, abs_data);
    plot_win->show();
}
```

调用 MwSimPlotWindow 的 AddCurveToCurrentView 可以将变量曲线添加到当前窗口视图中。

使用许可

MwVehicleMainWindow 实现了 License 配置的接口 SlotOpenLicense，接口调用 MwLicenseService 的 StartupLicenseSetDialog 弹出 License 配置对话框。

```
void MwVehicleMainWindow::SlotOpenLicense()
{
    LicenseServicePtr->StartupLicenseSetDialog();
}
```

3.1.1.3 APP 测试

为保证 APP 能够正常完整运行，在打包和编译代码期间可对 App 进行测试保证 APP 功能完整。其他详细测试方法查看 1.4.3 章节。

A. 编译调试测试

Debug 模式下模型的加载速度、打开速度会降低很多，这里不推荐使用 Debug 模式进行调试。

Release 模式下可进行调试，使用 Release 模式调试，会提高模型打开速度、加载速度、编译速度，默认的 release 版本不能进行调试，需要设置项目属性。

1. 右键打开项目属性，切换配置至对应的 Release 版本

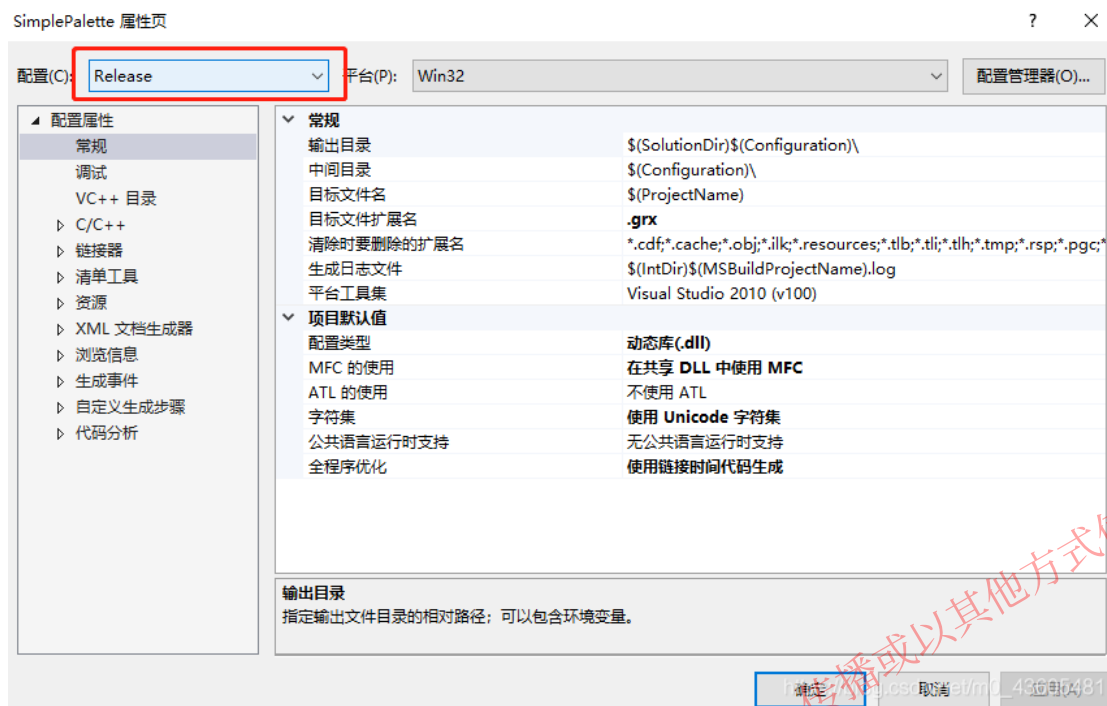


图 3-9 切换 Release 版本

2. 在左侧配置属性树中找到 C/C++ --> 优化，将右侧优化设置为 已禁用 (/Od)

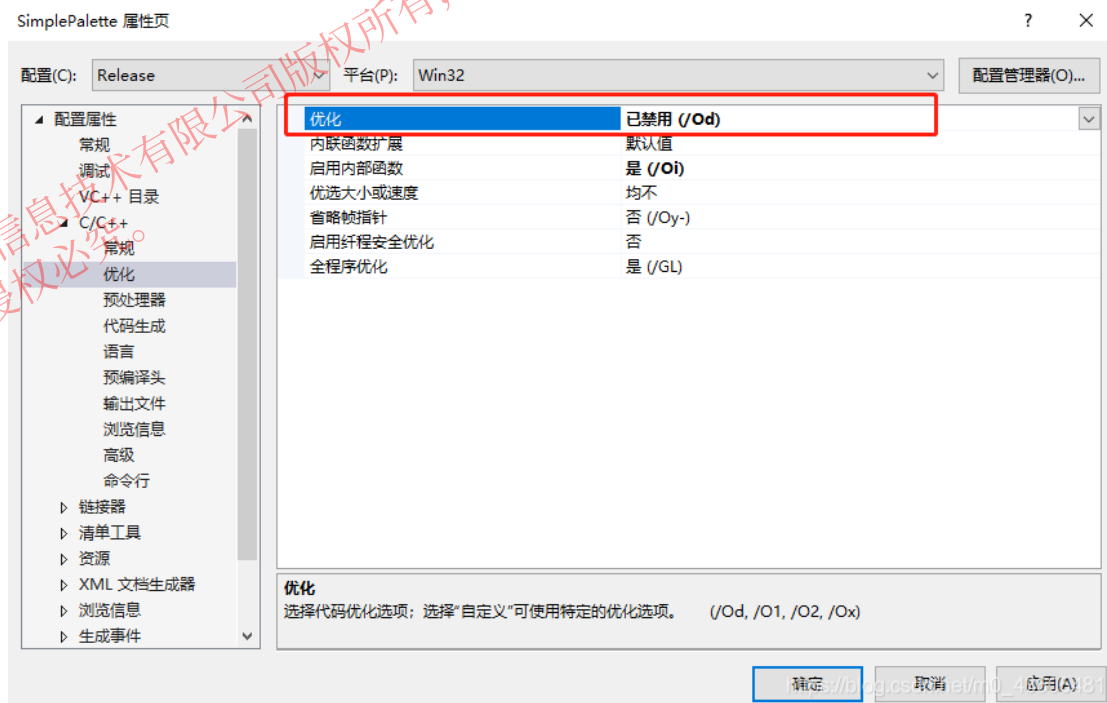


图 3-10 禁用优化

3. 左侧属性配置中选择链接器 --> 调试，将右侧生成调试信息 设置为 是 (/DEBUG)

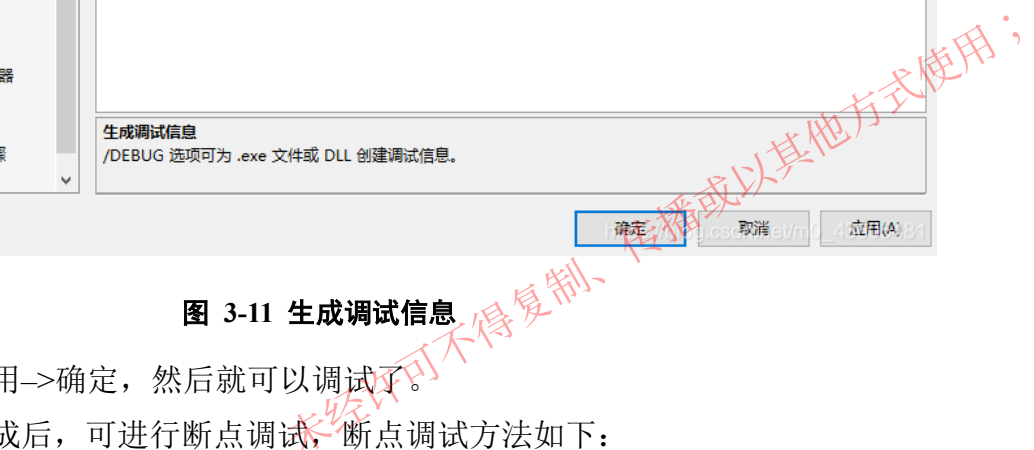


图 3-11 生成调试信息

4. 点击 应用->确定，然后就可以调试了。

以上配置完成后，可进行断点调试，断点调试方法如下：

1. 关注的代码附近打断点;



图 3-12 断点调试

2. 将鼠标放到参数值，可右键“添加监视变量”，也可直接查看到当前变量数据：



图 3-13 添加监视变量

B. 软件功能测试

APP 开发完成后，按照相应的软件测试方法，对照功能点功能列表进行详细的功能测试，确保软件在功能上完整，保证用户者的体验。

C. 集成测试

独立 APP 无需集成。

3.1.1.4 APP 打包及发布

APP 打包即遵循具体 APP 开发环境的要求，打包好的 APP 程序需独立可运行，不需再另外安装软件或执行其他的操作。将生成的 exe 及依赖的 dll 放入“SDK 安装目录/bin/win_msvc2017x64/Release”目录下：

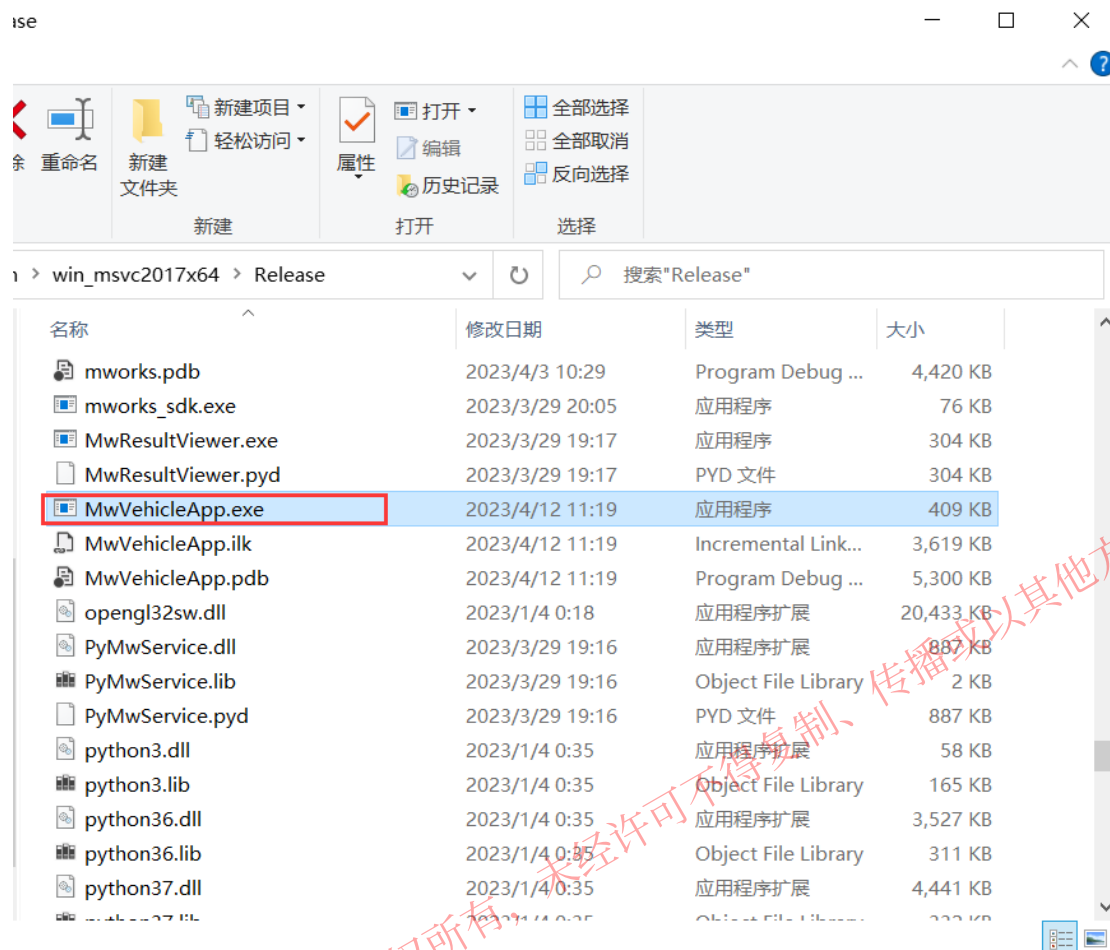


图 3-14 App 打包

双击“MassSpringDamperApp.exe”即可启动软件。放入 exe 后可对软件实现打包，打包步骤如下：

1. 将 Release 改为 bin 名称；
2. 保存 bin\win_msvc2017x64 目录下的以下目录：
 - external
 - initial_files
 - Library
 - bin(原 Release 文件)
 - setting
 - simulator
 - tools
3. 最终打包目录如下所示；

名称	修改日期	类型
external	2022/12/12 15:30	文件夹
initial_files	2022/12/12 15:30	文件夹
Library	2023/3/13 9:29	文件夹
Release	2023/3/13 10:33	文件夹
setting	2022/12/12 15:30	文件夹
simulator	2022/12/12 15:30	文件夹
tools	2022/12/12 15:30	文件夹

图 3-15 App 打包目录

3.1.2 批量仿真 APP

3.1.2.1 案例介绍

批量仿真插件围绕模型试验，提供了模型参数修改，批量仿真和结果批量显示功能。学生可以通过选择想要试验的参数，对参数试验数据进行批量编辑，并应用该数据完成批量仿真，显示结果，比较不同模型参数对试验结果的影响情况。

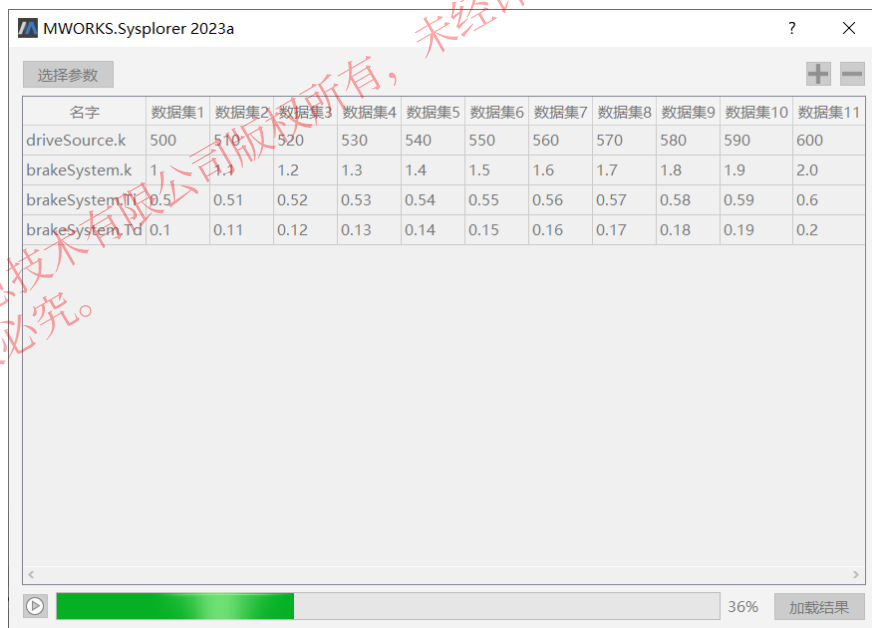


图 3-16 批量仿真插件

批量仿真插件如图所示，插件的使用流程是首先启动 MWORKE，打开需要批量仿真的模型，至少翻译一次之后，在工具栏中点击批量仿真按钮，弹出批量仿真配置窗口。

首先点击选择参数，在弹出的面板中选择需要批量仿真的参数集合，然后点

击加号添加数据集，接着修改数据集并点击开始批量仿真按钮，待进度条到达 100%后，点击加载结果按钮，将批量仿真的结果加载到 MWORKS 结果面板。

3.1.2.2 APP 开发

以下将从总体的框架搭建到细节的功能实现讲解使用 SDK 开发 MWORKS 插件的方法。

3.1.2.2.1 搭建框架

SDK 提供插件基类 MwPlugin，插件的初始化和析构都基于 MwPlugin 进行，进行插件开发时需要继承 MwPlugin 并至少实现它的所有纯虚函数，MwPlugin 类如下图所示：

```
class EXTENSIONSYSTEM_EXPORT MwPlugin : public QObject
{
    Q_OBJECT

public:
    enum ShutdownFlag {
        SynchronousShutdown,
        AsynchronousShutdown
    };

    MwPlugin();
    ~MwPlugin();

    virtual bool initialize(QString *errorString) = 0;
    virtual void extensionsInitialized() = 0;
    virtual bool delayedInitialize() { return false; }
    virtual ShutdownFlag aboutToShutdown() { return SynchronousShutdown; }

signals:
    void asynchronousShutdownFinished();
};
```

(1). initialize: 该接口用于初始化插件内部状态，在加载插件并创建 MwPlugin 实例后被 MWORKS 调用。该接口按照插件注册顺序依次调用，并且保证被依赖插件初始化先于依赖插件初始化（插件依赖在 json 文件的 Dependencies 条目配置）。

(2). extensionsInitialized: 所有插件的 initialize 执行完毕后，会按照插件注册顺序依次执行 extensionsInitialized，并且保证被依赖插件的 extensionsInitialized

先于依赖插件执行（插件依赖在 json 文件的 Dependencies 条目配置）。

(3). **delayedInitialize**: 所有插件的 **extensionsInitialized** 执行完毕后，会按照插件注册顺序依次执行 **delayedInitialize**，并且保证被依赖插件的 **delayedInitialize** 先于依赖插件执行（插件依赖在 json 文件的 Dependencies 条目配置）。

(4). **aboutToShutdown**: 该接口用于断开与其他插件的连接并优化关闭。如果插件需要将实际关闭延迟一段时间，则插件可以从该接口返回 **MwPlugin::AsynchronousShutdown**，MWORKS 将在收到所有延迟关闭插件的 **asynchronousShutdownFinished** 信号之前保持主线程事件循环运行。

(5). **asynchronousShutdownFinished**: 用于插件异步关闭时，通知 MWORKS 主程序插件异步关闭已完成。

A. MwBatchSimPlugin

MwBatchSimPlugin 继承 **MwPlugin**，实现了 **initialize** 和 **extensionsInitialized** 两个纯虚函数，在 **initialize** 中将批量仿真按钮添加到 MWORKS 工具栏中，**extensionsInitialized** 的实现为空。

```
bool MwBatchSimPlugin::initialize(QString *errorString)
{
    MwWorkbench *work_bench = MwAppPtr->GetWorkbench();
    work_bench->AddActionToToolBar(actionBatchSim);

    return true;
}

void MwBatchSimPlugin::extensionsInitialized()
{
}
```

点击批量仿真按钮时，会触发槽函数 **SlotShowBatchSimDlg**，槽函数中获取 MWORKS 当前模型键值并依赖模型编译结果构造 **MwSimData** 对象，以此创建批量仿真对话框 **MwBatchSimDialog**。MWORKS 相关的对象均借由 **MwAppPtr** 获取，它的定义位于 **MwAPP**（MWORKS 应用程序对象）中。

```
void MwBatchSimPlugin::SlotShowBatchSimDlg()
{
    if (batchSimDlg == nullptr)
    {
        MwMoGraphicsViewController *mo_controller = MwAppPtr->GetWorkbench()->GetMoGVCController();
        MWint cur_key = mo_controller->GetCurrentClassKey();
    }
}
```

```

        if (cur_key == 0)
        {
            return;
        }

        MwClassManager *class_mgr = MwAppPtr->GetClassManager();
        std::wstring sim_path = class_mgr->GetCoreOption()->SimResultPath();
        std::wstring mo_name = QString::fromStdString(class_mgr->GetMoHandler()->GetNameProp(cur_key)).toStdWString();

        std::wstring result_dir = sim_path + L"/" + mo_name;
        std::wstring data_path = result_dir + L"/Result.msr";
        baseSimData = new MwSimData(mo_name, data_path, result_dir);
        if (!baseSimData->InitializeSimInst())
        {
            return;
        }

        batchSimDlg = new MwBatchSimDialog(MwAppPtr->GetClassManager(), cur_key, baseSimData, MwAppPtr->GetWorkbench());
    }

    batchSimDlg->show();
}

```

MwBatchSimPlugin 头文件中还需要注册 MWORKS 插件元数据类型，并配置对应的 json 插件描述文件。

```
Q_PLUGIN_METADATA(IID MwAppId FILE "mw_batch_sim.json")
```

在 mw_batch_sim_plugin.cpp 所在目录下新建 mw_batch_sim.json 文件并进行配置。

```

{
    "Name": "Batch Simulation",
    "Version": "1.0",
    "CompatVersion": "1.0",
    "Vendor": "苏州同元",
    "Copyright": "(c) 2010-2023",
    "License": [],
    "Category": "SDK",
    "Description": "实现带有批量仿真功能的 MWORKS.Sysplorer 插件。",
    "Description_en": "batch simulation plugin for MWORKS.Sysplorer.",
    "Options": [],
    "Url": "http://www.tongyuan.cc",
    "Dependencies": []
}

```

➤ Name: 插件名称

- Version: 插件版本
- CompatVersion: 插件兼容版本
- Vendor: 供应商
- Copyright: 版权
- License: 许可
- Category: 类别
- Description: 描述
- Description_en: 英文描述
- Options: 选项
- Url: 网址
- Dependencies: 依赖的插件

B. [MwBatchSimDialog](#)

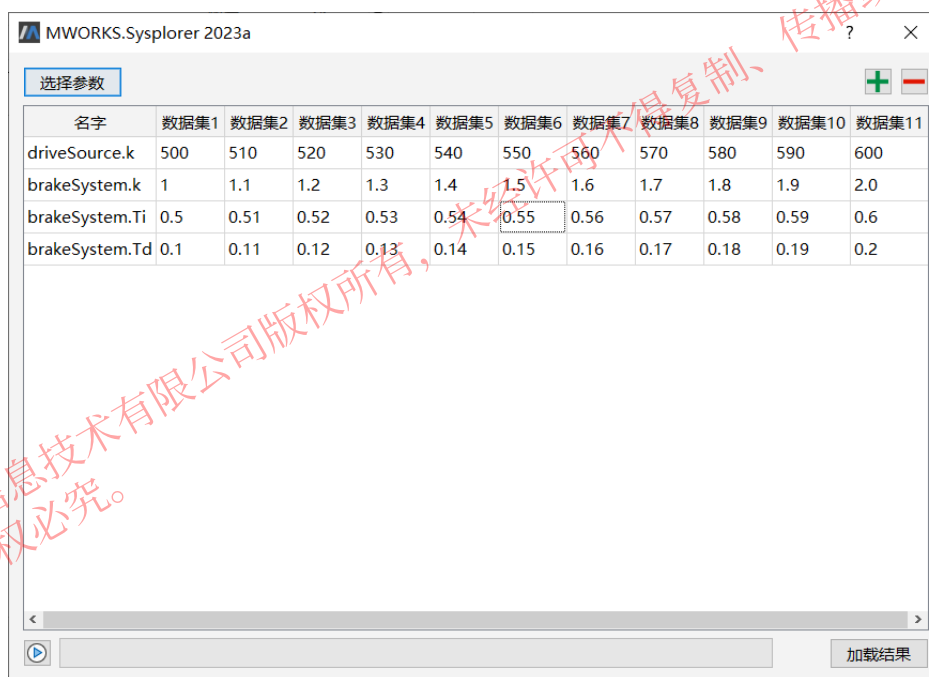


图 3-17 批量仿真对话框

MwBatchSimDialog 是批量仿真插件的主界面，由选择参数、编辑参数、批量仿真三个部分组成。

C. [MwParamSelectDialog](#)

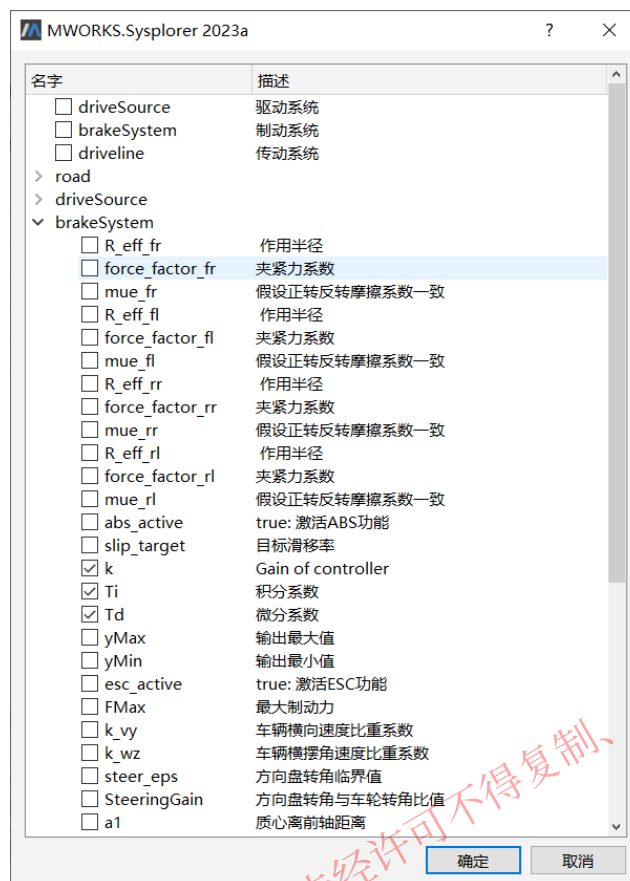


图 3-18 批量仿真参数选择对话框

MwParamSelectDialog 为批量仿真参数选择对话框，内部初始化一棵 MwParamSelectTree 批量仿真参数选择树。

```
void MwParamSelectDialog::SetupUi()
{
    QVBoxLayout *vbox_layout = new QVBoxLayout(this);
    paramTree = new MwParamSelectTree(classMgr, curKey, this);
    vbox_layout->addWidget(paramTree);
}
```

D. MwParamEditTable

MwParamEditTable 继承 QTableWidgetItem，负责显示和编辑批量仿真参数。

3.1.2.2.2 功能实现

选择参数

MwBatchSimDialog 通过接口 SlotSelectParam 弹出选择参数对话框，接口先检查是否之前已经有选择过参数，如果有则读取之前的参数列表来初始化，最后

弹出 MwParamSelectDialog 批量参数选择对话框。

```
void MwBatchSimDialog::SlotSelectParam()
{
    MwParamSelectDialog param_select_dlg(classMgr, curKey, this);
    QStringList param_name_list;
    paramTable->GetParamNameList(param_name_list);
    param_select_dlg.InitParamSelectTreeCheckState(param_name_list);
    if (param_select_dlg.exec() == QDialog::Accepted)
    {
        param_name_list.clear();
        param_select_dlg.GetSelectedParamNameList(param_name_list);
        paramTable->UpdateParamEditTable(param_name_list);
    }
}
```

MwParamSelectDialog 中初始化 MwParamSelectTree。

```
void MwParamSelectDialog::SetupUi()
{
    QVBoxLayout *vbox_layout = new QVBoxLayout(this);
    paramTree = new MwParamSelectTree(classMgr, curKey, this);
    vbox_layout->addWidget(paramTree);
}
```

MwParamSelectTree 中的 InitParamSelectTree 接口负责初始化批量仿真参数选择树。

```
void MwParamSelectTree::InitParamSelectTree()
{
    std::vector<MwParamInfo> top_param_lst;
    if (!classMgr->GetMoHandler()->GetParamInfoEx(curKey, MwStrList(), &top_param_lst))
    {
        return;
    }
    for (auto top_param_info : top_param_lst)
    {
        QTreeWidgetItem *top_item = CreateParamItem("", &top_param_info);
        this->addTopLevelItem(top_item);
    }

    std::vector<MWInt> comp_keys;
    classMgr->GetMoHandler()->GetNestedCompKeys(curKey, comp_keys);
    for (auto nested_comp_key : comp_keys)
    {
        QTreeWidgetItem *comp_item = CreateCompItem(nested_comp_key);
    }
}
```

```

std::vector<MwParamInfo> nested_param_lst;
std::string nested_comp_name = classMgr->GetMoHandler()->GetNameProp(nested_comp_key);
MwStrList comp_nodes;
comp_nodes.push_back(nested_comp_name);
if (classMgr->GetMoHandler()->GetParamInfoEx(curKey, comp_nodes, &nested_param_lst))
{
    if (!nested_param_lst.empty())
    {
        this->addTopLevelItem(comp_item);
    }
    for (auto nested_param_info : nested_param_lst)
    {
        QTreeWidgetItem *nested_param_item = CreateParamItem(QString::fromStdString(nested_comp_name), &nested_param_info);
        comp_item->addChild(nested_param_item);
    }
}
}
}

```

首先通过 MwMoHandler 的 GetParamInfoEx 获取顶层模型的参数列表，接着通过 MwMoHandler 的 GetNestedCompKeys 获取顶层模型下的嵌套组件，最后再使用一次 GetParamInfoEx 获取嵌套组件的参数列表，完成参数树的初始化。

编辑参数

MwParamEditTable 继承 QTableWidgetItem，实现了参数编辑功能。点击表格右上角的加号和减号分别调用 AddDataSet 和 RemoveDataSet 接口，更新参数的数据集。添加数据集时参数值为参数在模型中的默认值，默认值的获取通过 UpdateParamDefaultValue 实现。

```

void MwParamEditTable::UpdateParamDefaultValue()
{
    for (auto param_name : paramList)
    {
        QStringList param_nodes = param_name.split(".");
        MwStrList param_nodes_lst;
        for (auto param_node_name : param_nodes)
        {
            param_nodes_lst.push_back(param_node_name.toStdString());
        }
    }
}

```

```

    QString param_value = QString::fromStdString(classMgr->GetMoHandler()->GetParam
Value(curKey, param_nodes_lst));
    paramDefaultVal.append(param_value);
}
}

```

该接口通过参数名节点列表，调用 MwMoHandler 的 GetParamValue 接口获取参数默认值。

批量仿真

MwBatchSimDialog 中的接口 SlotBatchSim 负责启动批量仿真，接口中从 MwParamEditTable 中获取选择的批量仿真参数和数据集作为批量仿真的输入，提供给 MwBatchSimControl 来启动批量仿真。

```

void MwBatchSimDialog::SlotBatchSim()
{
    std::wstring sim_path = classMgr->GetCoreOption()->SimResultPath();
    batchSimCtrl->RebindBaseData(baseSimData, sim_path);
    QStringList param_name_list;
    QList<QStringList> param_val_list;
    paramTable->GetParamNameList(param_name_list);
    paramTable->GetParamDataSetList(param_val_list);
    batchSimCtrl->SetBatchSimDataSrc(param_name_list, param_val_list);
    progressBar->setRange(0, 100);
    progressBar->setValue(0);
    this->setEnabled(false);
    batchSimCtrl->StartBatchSim();
}

```

MwBatchSimControl 中通过创建多个 MwSimControl 实现并发仿真，启动若干 MwBatchSimThread 线程并为每个线程分配一个 MwSimControl 对象。

```

void MwBatchSimControl::StartBatchSim()
{
    ReSet();
    for (int i = 0; i < simConcurrency; ++i)
    {
        MwBatchSimThread *batch_sim_thread = new MwBatchSimThread(this);
        connect(batch_sim_thread, &QThread::finished, batch_sim_thread, &QThread::deleteLater);
        batch_sim_thread->start();
    }
}

```

每个 `MwSimControl` 线程执行完当前的仿真任务后会互斥地从 `MwBatchSimControl` 中获取下一个需要仿真的数据集，然后开始下一次仿真，直到所有数据集被全部仿真完毕后批量仿真结束。

```
void MwBatchSimThread::run()
{
    MwSimData *clone_data = nullptr;
    bool is_success = false;

    batchSimCtrl->Lock();
    is_success = CreateCloneDataAndMakeDir(clone_data);
    batchSimCtrl->UnLock();

    while (is_success)
    {
        simCtrl->RebindSimData(clone_data);
        simCtrl->StartSimulate(MwSimControl::Sim_ContinueMode, MwSimControl::Sim_Batch);
        semSim->acquire();

        batchSimCtrl->Lock();
        is_success = CreateCloneDataAndMakeDir(clone_data);
        batchSimCtrl->UnLock();
    }
}
```

每次启动一次仿真后，线程内的同步通过信号量完成，确保在单次仿真完成后在取下一个数据集仿真。

加载结果

`MwBatchSimDialog` 中的 `SlotLoadResult` 接口实现将批量仿真结果加载到 MWORKS 结果面板的功能，接口通过调用 `MwWorkbench` 的 `OpenSimResult` 接口实现。

```
void MwBatchSimDialog::SlotLoadResult()
{
    QString sim_path = QString::fromStdWString(classMgr->GetCoreOption()->SimResultPath());
    int data_set_size = batchSimCtrl->GetDataSetSize();
    for (int i = 0; i < data_set_size; ++i)
    {
        QString inst_path = sim_path + QString("/Result-%1/Result.msr").arg(i + 1);
        MwAppPtr->GetWorkbench()->OpenSimResult(inst_path);
    }
}
```

```
}  
}
```

3.1.2.3 APP 测试

为保证 APP 能够正常完整运行，在打包和编译代码期间可对 App 进行测试保证 APP 功能完整。其他详细测试方法查看 1.4.3 章节。

A. 编译调试测试

嵌入式生成的输出文件为 dll 库，dll 库需要依赖于 exe 才可进行调试，这里通过将程序中配置 exe 调试路径对编译的 dll 库进行调试，配置方法如下：

右键项目->属性->调试->命令/命令参数 中传入 mworks.exe 的路径，这里使用 SDK 下自带的 mworks.exe 进行调试。

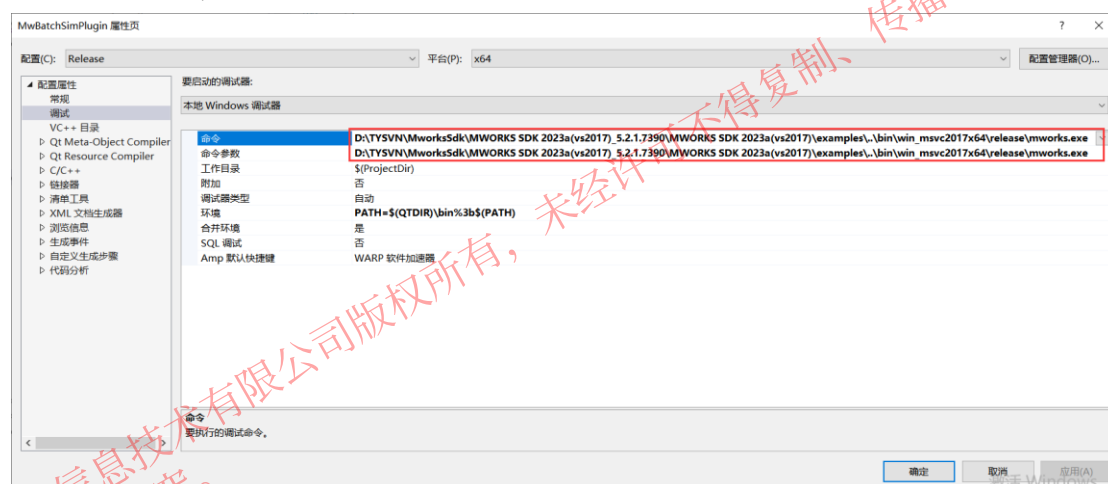


图 3-19 传入 mworks.exe 路径

直接点击运行程序可以直接打开软件进行调试。

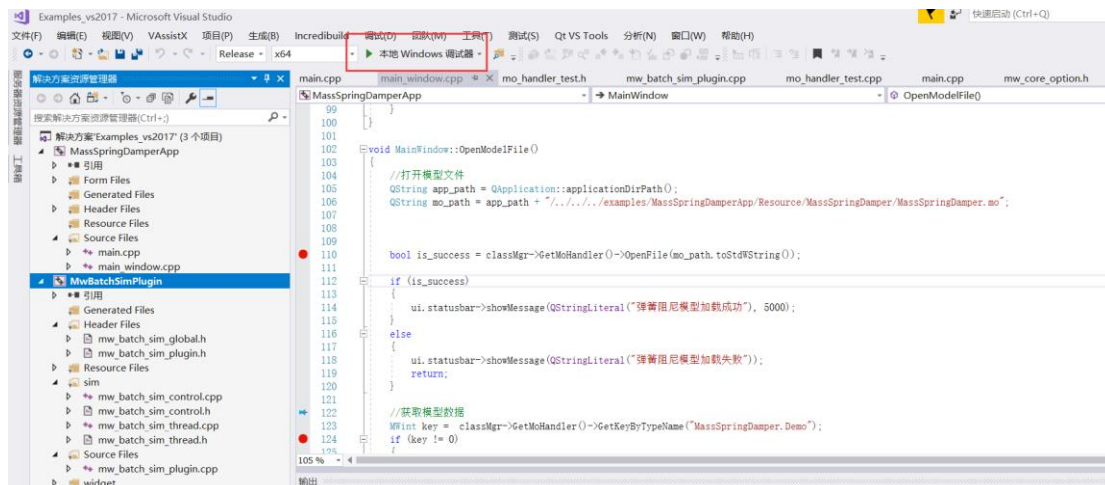


图 3-20 运行程序并调试

Release 调试仍需配置属性，具体配置教程及调试参考 3.1.1.4 章节。

B. 功能测试

APP 开发完成后，按照相应的软件测试方法，对照功能点功能列表进行详细的功能测试，确保软件在功能上完整，保证用户者的体验。

C. 集成测试

将开发完成的 dll 放入不同 Sysplorer 版本软件对应的插件目录(Bin/Addins)，并测试能否在 Sysplorer 软件中启动该工具，若启动成功则测试通过。

3.1.2.4 APP 打包及发布

APP 打包即遵循具体 APP 开发环境的要求，打包好的 APP 程序需独立可运行，不需再另外安装软件或执行其他的操作。将嵌入式生成的 dll 库，放入“SDK 安装目录 /bin/win_msvc2017x64/Release/Addins”目录下的新建文件夹 MwBatchSimPlugin(文件夹名称任意)中即可进行运行。

Release > Addins > MwBatchSimPlugin					搜索"MwBatchSimPlugin"	
	名称	修改日期	类型	大小		
	MwBatchSimPlugin.dll	2023/4/3 8:30	应用程序扩展	253 KB		
	MwBatchSimPlugin.lib	2023/4/3 8:30	Object File Library	8 KB		

图 3-21 插件打包

点击/bin/win_msvc2017x64/Release/mworks.exe 运行 Sysplorer 软件，可查看到放入的插件。

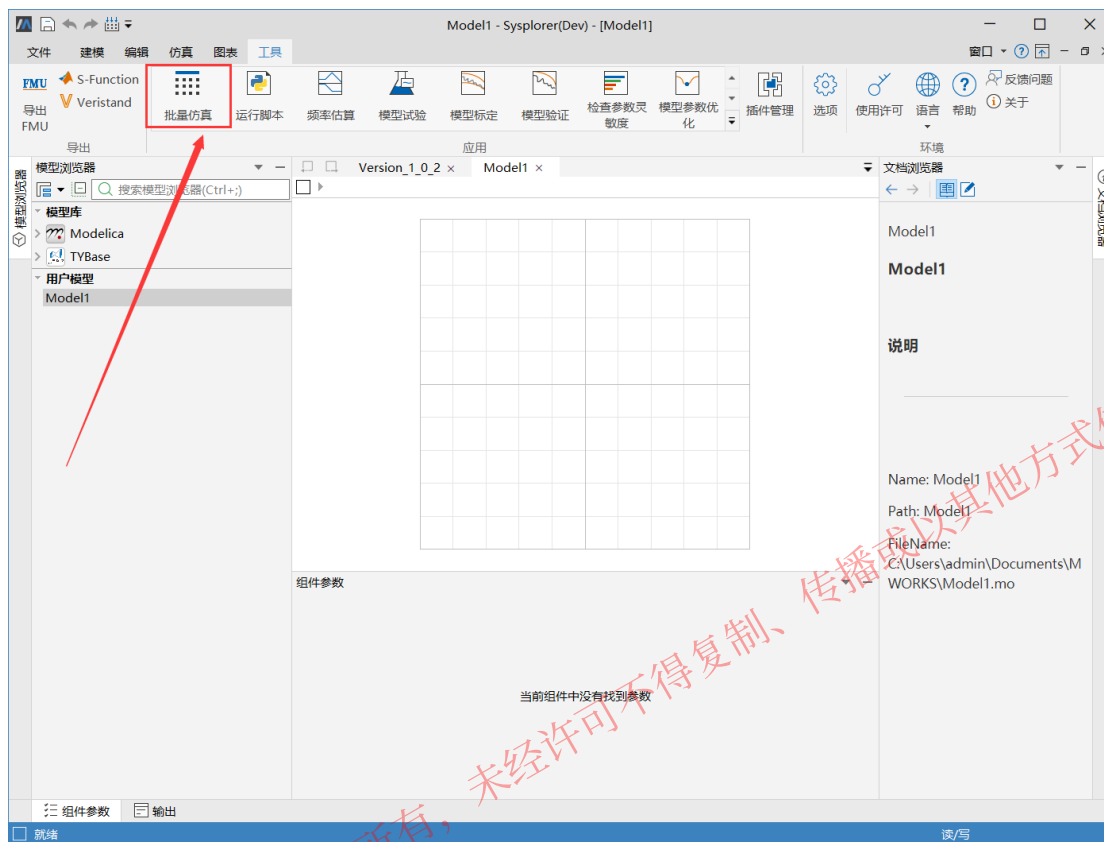


图 3-22 插件效果

将 dll 和文件夹打包，放入任意 Sysplorer 的 bin 目录下的 Addins 文件夹下，然后运行 Sysplorer 可查看到开发的插件功能。

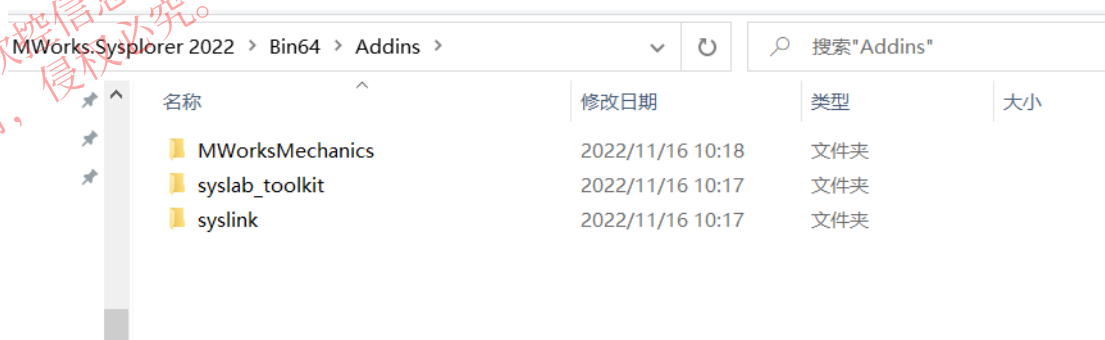


图 3-23 插件发布

4. 专题说明

4.1 Qt 界面开发教程

MWORKS.Sysplorer 依赖于 Qt 对界面进行开发，因此推荐用户对 APP 的界面开发也使用 Qt 程序，以下简单介绍 Qt 相关教程，更详细教程可在网上进行学习。

Qt 是一个跨平台的 C++ 开发库，主要用来开发图形用户界面(Graphical User Interface, GUI) 程序,Qt 还存在 Python、Ruby、Perl 等脚本语言的绑定，也就是说可以使用脚本语言开发基于 Qt 的程序。下面主要介绍 C++平台利用 Qt 进行界面开发。

A. VS2017 中配置 Qt5.14 教程

https://blog.csdn.net/weixin_46098577/article/details/122131488

B. Qt 界面设计教程

1. 基于 VS2017 新建一个 Qt Application 工程：

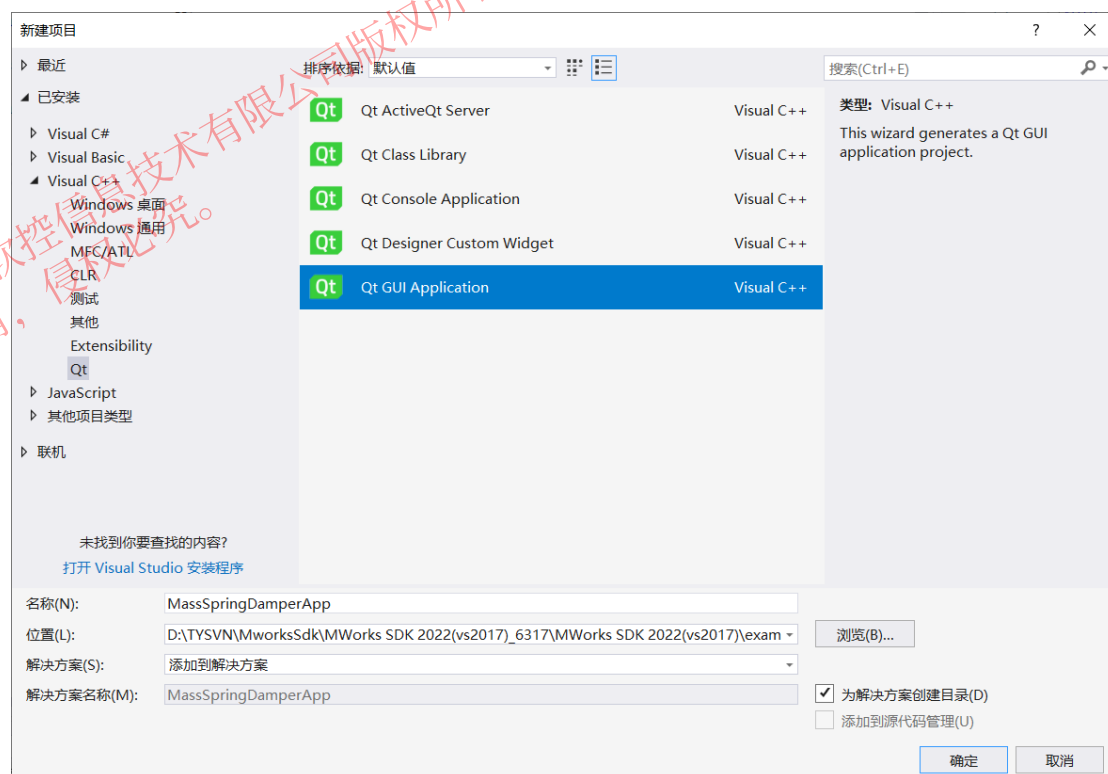


图 4-1 新建工程

2. 双击打开对应的*.ui 文件，进入 designer.exe 程序如下：

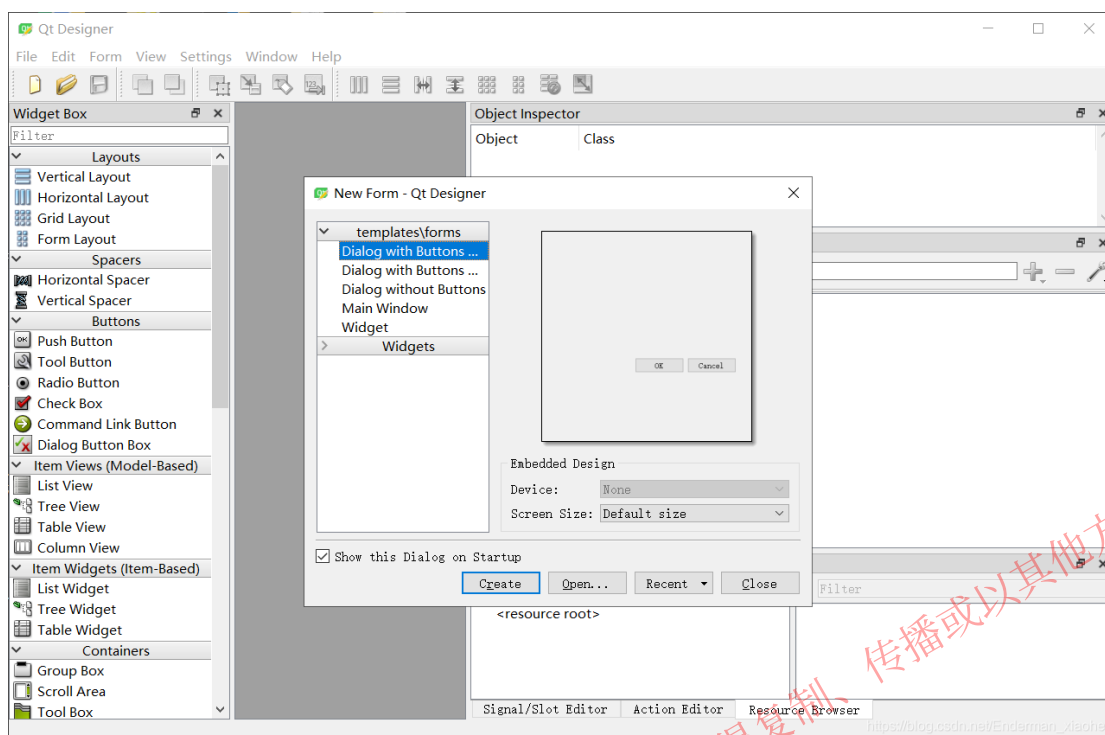


图 4-2 新建 ui 文件

3. 软件主要界面介绍，详细介绍可查看相关官方教程。

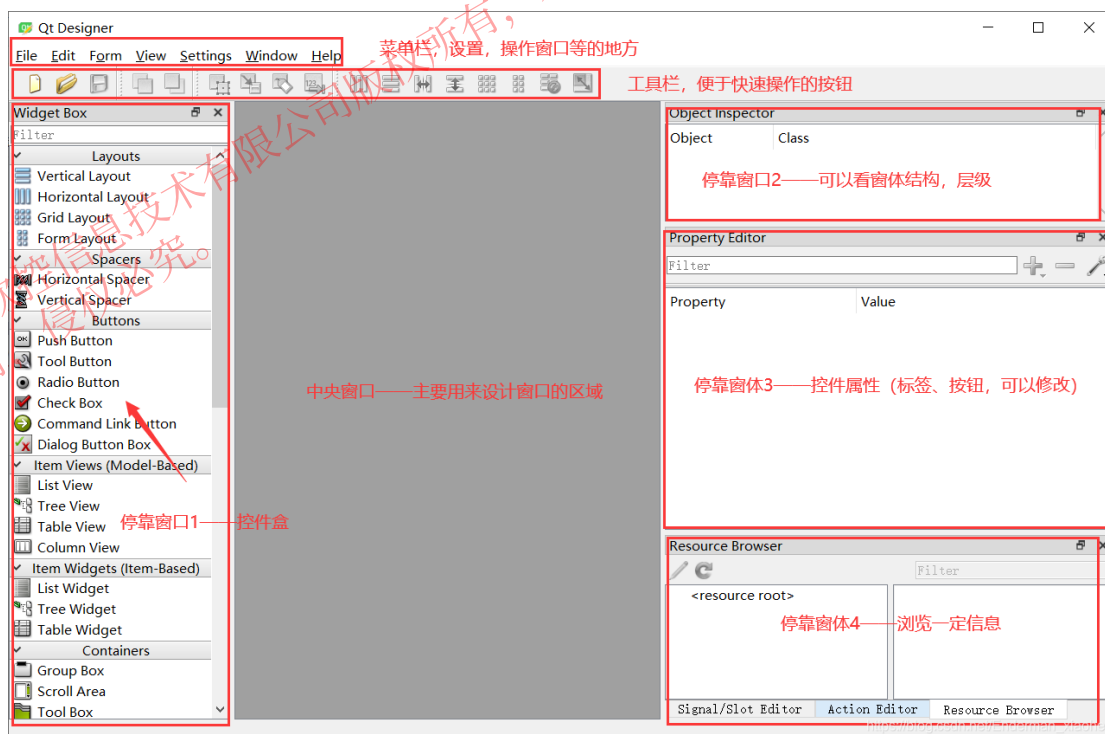


图 4-3 软件界面介绍

4. 创建一个用于设计的窗口：初次打开软件，应该已经直接创建了，若不

存在可在 File->New 中打开，或者工具栏第一个，简略介绍如下：

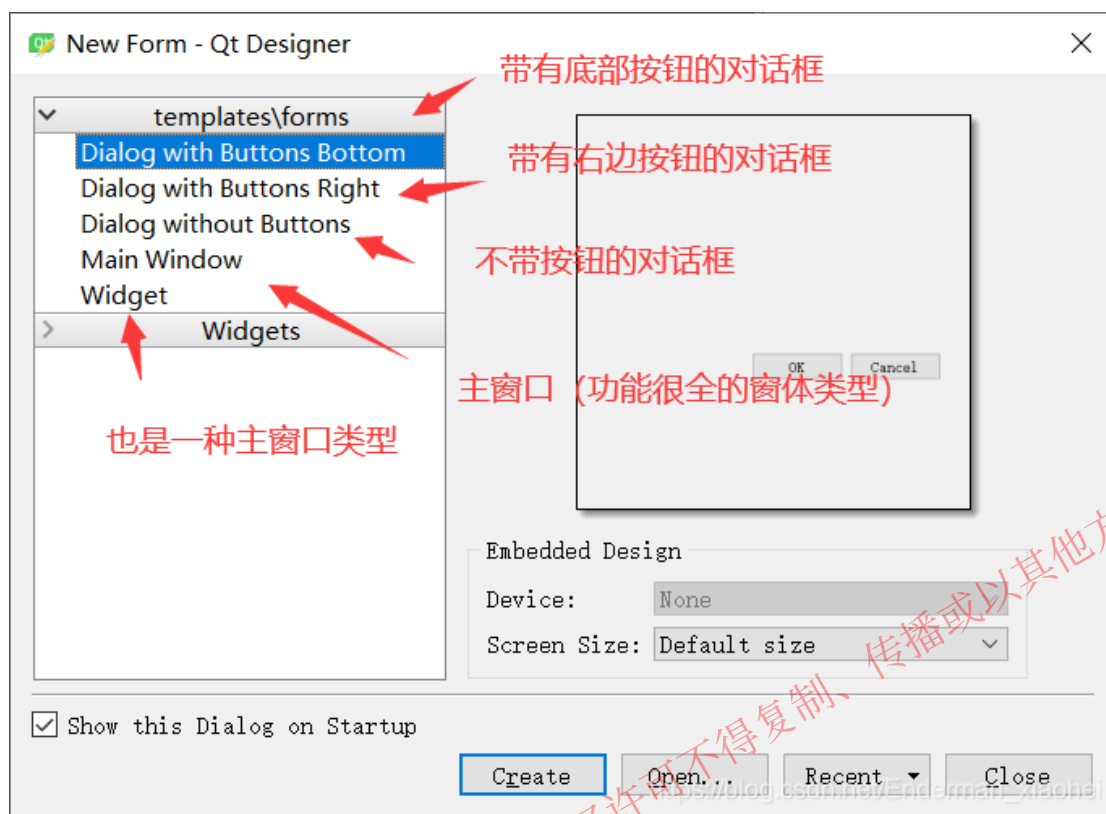


图 4-4 创建窗口介绍

直接选第一个，点击 Create，创建完成之后：

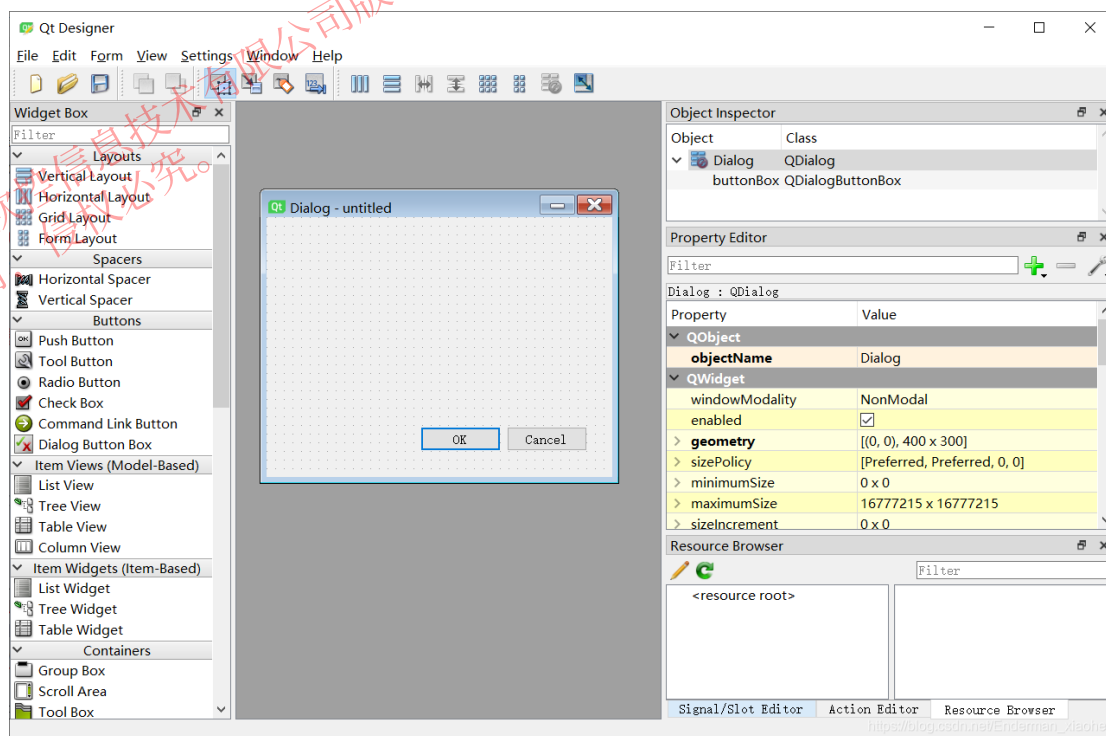


图 4-5 创建一个 dialog 窗口

5. 可将旁边的其他控件拖拽进主窗口中，进行界面的开发和设计，常用控件及介绍如下：

- 标签（Label）——> QLabel（PyQt5）
- 按钮（Button）——> QPushButton
- 行编辑（lineEdit）——> QLineEdit
- 组合框（ComboBox）——> QComboBox
- 复选框（CheckBox）——> QCheckBox

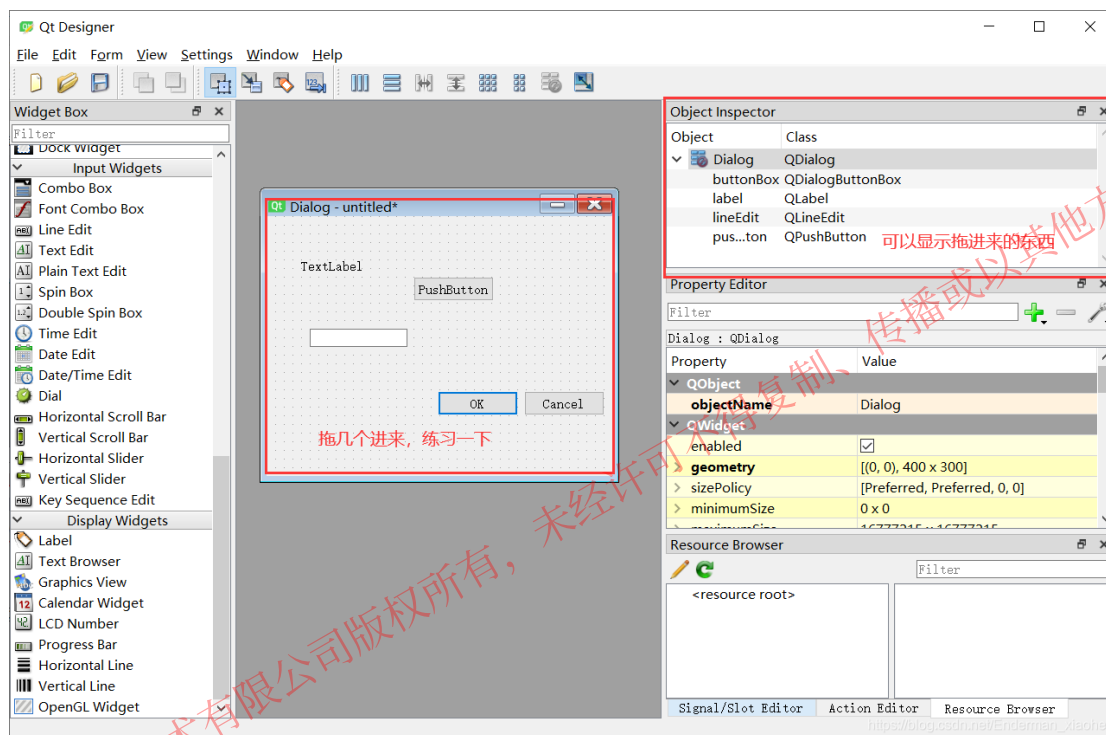


图 4-6 拖拽控件练习

6. 利用 Qt 开发完成的弹簧阻尼 APP 设计如下：

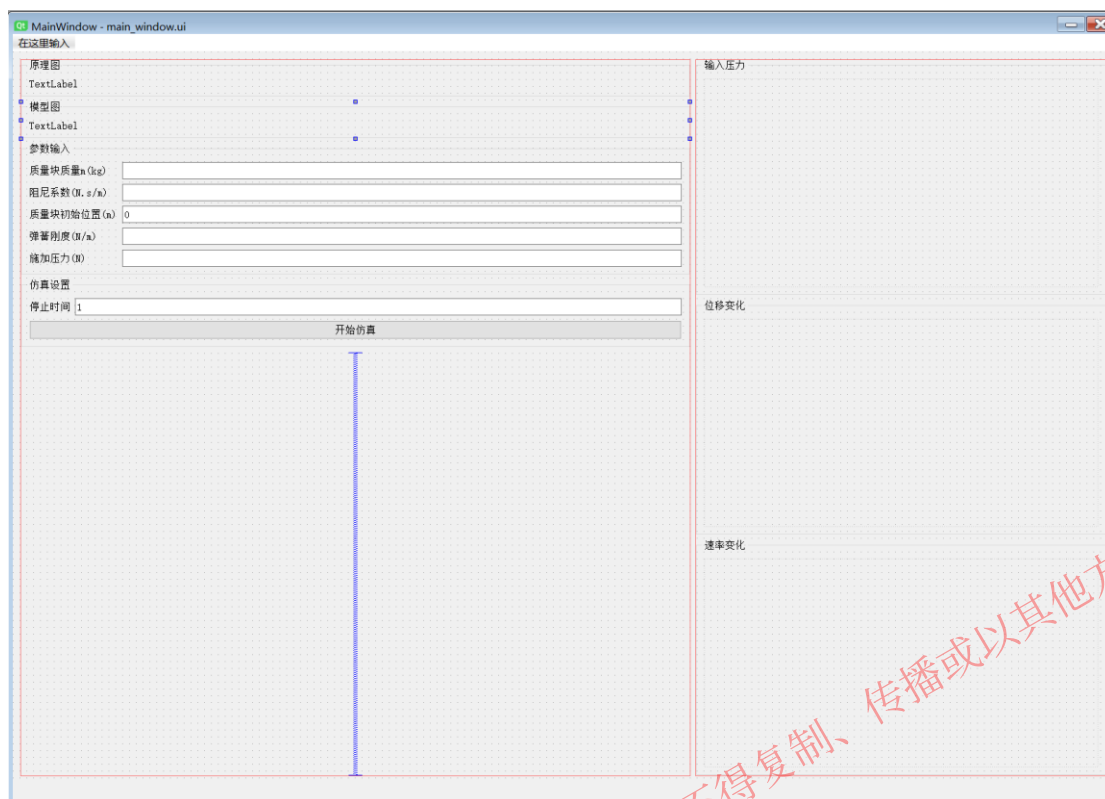


图 4-7 已完成设计界面

C. Qt 信号和槽

1. 什么是信号和槽？

信号和槽是用于对象之间的通信，它是 Qt 的核心机制，在 Qt 编程中有着广泛的应用。

举个例子，在一个十字路口，信号灯变成了绿色，对面的汽车看到后就启动了。信号灯就是发送信号的对象，绿灯亮是它发送的信号 (signal)，汽车是接收对象，汽车行驶是汽车对信号的响应，也叫槽 (slot)。

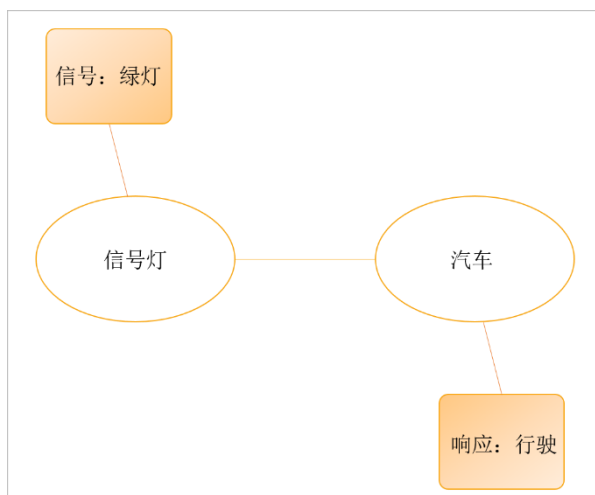


图 4-8 信号和槽函数举例 1

再举一个例子，比如在一个主窗口内有一个关闭按钮，如果点击这个按钮窗口就会关闭，那么关闭按钮是发送信号的对象，它发送的信号是点击，接收信号的对象是窗口，响应信号的槽是关闭窗口。

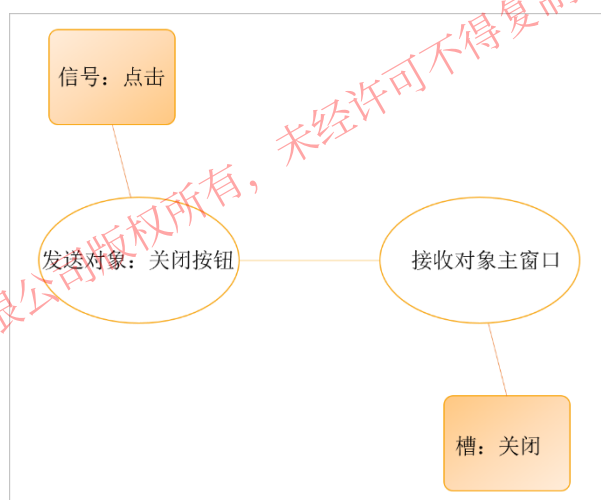


图 4-9 信号和槽函数举例 2

2. 信号和槽代码实例

a) 主要通过 connect + 宏的方式进行通信连接

connect(发送对象, 信号, 接收对象, 槽函数), 其中发送信号和槽函数需要用 SIGNAL() 和 SLOT() 来进行声明。

connect 函数声明如下:

```
[static] QMetaObject::Connection QObject::connect(const QObject *sender, const char *signal,
const QObject *receiver, const char *method, Qt::ConnectionType type = Qt::AutoConnection)
```

比如点击按钮关闭窗口的例子，代码可以这样写：

```
connect(ui->pushButton, SIGNAL(clicked()), this, SLOT(close()));
```

如果想自定义槽函数，需要先将槽函数的声明添加到类的 slots 中。比如我们对一个 QLineEdit 控件添加一个接收 textEdited 信号的槽函数 onTextEdited

```
class MainWindow : public QMainWindow
{
    Q_OBJECT

public:
    MainWindow(QWidget *parent = nullptr);
    ~MainWindow();

private slots:
    void onTextEdited(QString);

private:
    Ui::MainWindow *ui;
};
```

然后实现函数，并用 connect 与信号连接

```
MainWindow::MainWindow(QWidget *parent)
    : QMainWindow(parent)
    , ui(new Ui::MainWindow)
{
    ui->setupUi(this);
    connect(ui->pushButton, SIGNAL(clicked()), this, SLOT(close()));
    connect(ui->lineEdit, SIGNAL(textEdited(QString)), this, SLOT(onTextEdited(QString)));
}

void MainWindow::onTextEdited(QString s)
{
    qDebug() << s;
}
```

b) 使用 Qt Creator 界面添加信号的槽函数

另外一种方式不需要使用 connect 函数，可以通过 Qt Creator 界面来完成发送信号和槽函数的连接，比如我们右键点击一个按钮，然后选择“转到槽”：

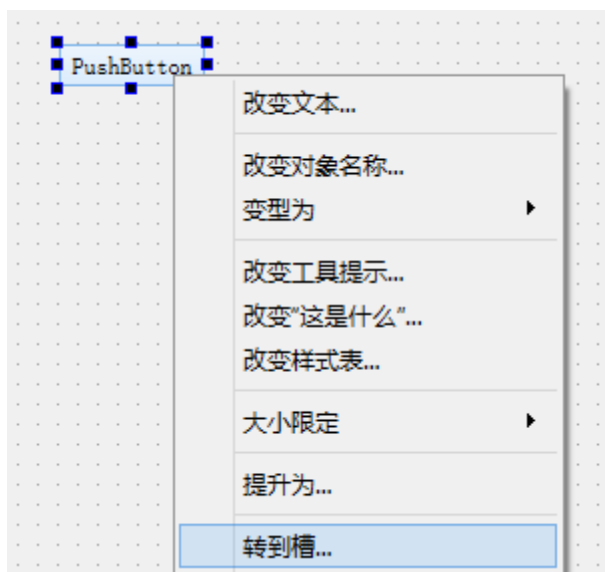


图 4-10 界面转到槽

选择信号，我们点击 QAbstractButton 的 clicked() 信号，表示按钮被点击：

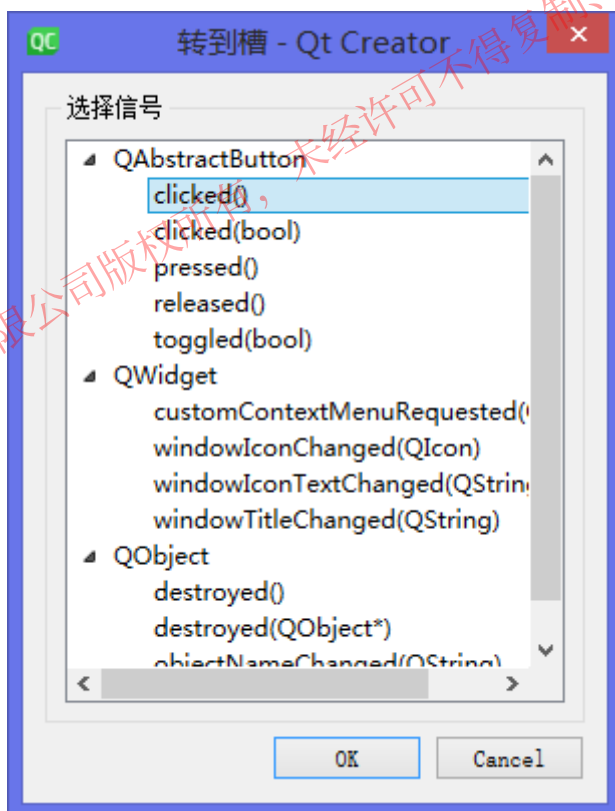


图 4-11 界面选择信号

接下来，Qt Creator 会自动为我们生成如下代码，首先是槽函数的声明：

```
class MainWindow : public QMainWindow
{
```

```
Q_OBJECT

public:
    MainWindow(QWidget *parent = nullptr);
    ~MainWindow();

private slots:
    void on_pushButton_clicked();

private:
    Ui::MainWindow *ui;
};
```

然后是槽函数的实现：

```
void MainWindow::on_pushButton_clicked()
{
}

}
```

使用这种方法我们不需要使用 `connect` 函数将信号与槽函数做连接。这里槽函数的命名有一定的规则，一般是 `on_objectname_signal` 这样来命名的。这种方法优点是减少了自己手动敲代码的工作量，缺点是究竟有哪些信号与槽函数做了连接不易被发现，没有 `connect` 函数看起来直观。

总结：Qt 使用 QDesigner 进行界面设计开发，并利用 Qt 语言结合 C++ 特性以及相关的信号和槽函数机制对界面相关事件进行处理，详细开发教程可在网上资源进行学习：<https://blog.csdn.net/perfectguyipeng/article/details/84611569>

4.2 开发工程构建

基于 VS2017 版本进行开发，请确保已安装 VS2017 Qt 插件，并安装 Qt5.14.2 等相关开发环境。

A. 新建工程

基于 VS2017 新建一个 Qt Application 工程，命名为 MassSpringDamperApp:

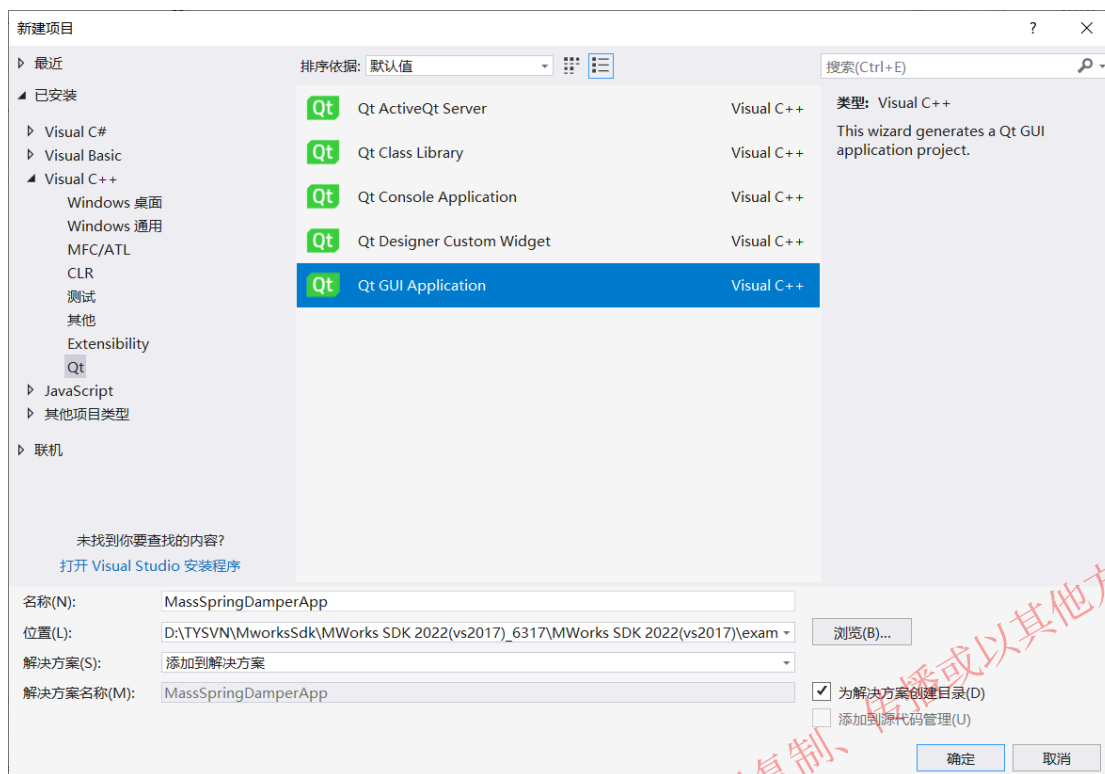


图 4-12 新建工程

B. 输出目录配置

- 将输出目录配置到安装的 SDK 路径的 bin 目录：\$(MWBIn)\$(Configuration)
- 将 MWBin 换为实际 SDK 对应路径：

SDK 安装路径\bin\win_msvc2017x64

苏州同元软控信息技术有限公司所有，未经许可不得复制、传播或以其他方式使用；
版权所有，侵权必究。

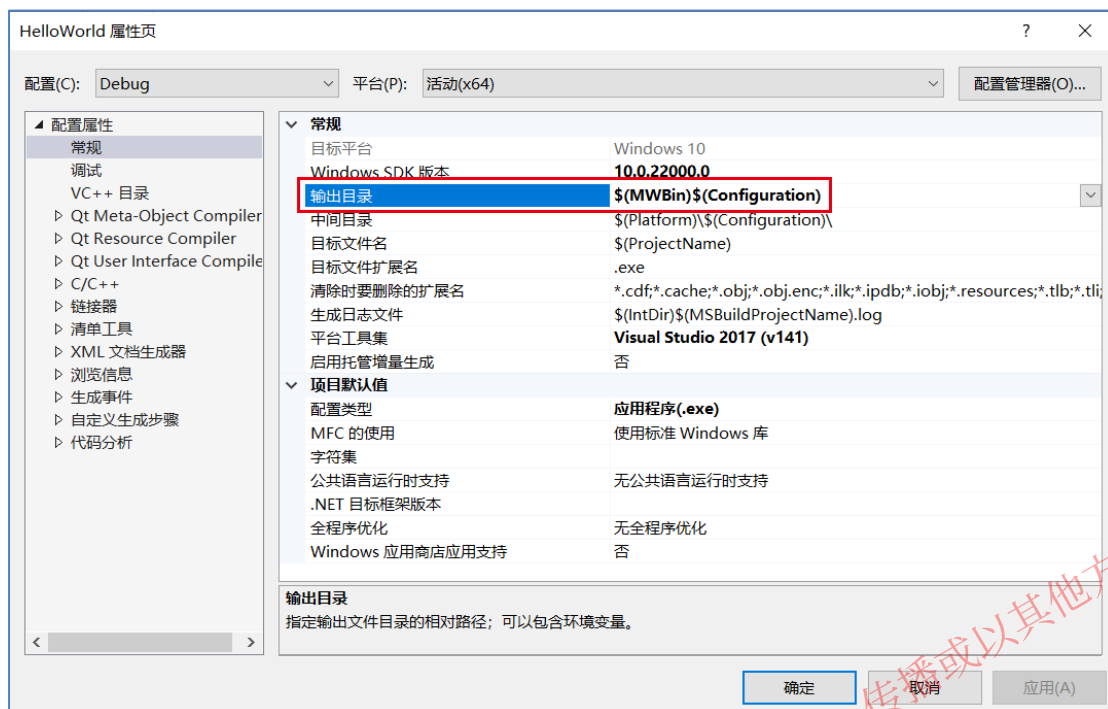


图 4-13 输出目录配置

\$(MWBIn)、\$(MWInterface)、\$(MWInclude)需在电脑中的系统环境变量中将 SDK 安装路径配置上：

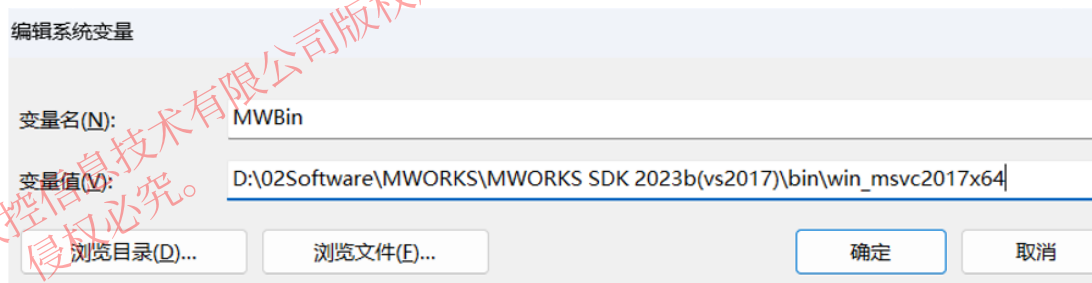


图 4-14 环境变量拍配置

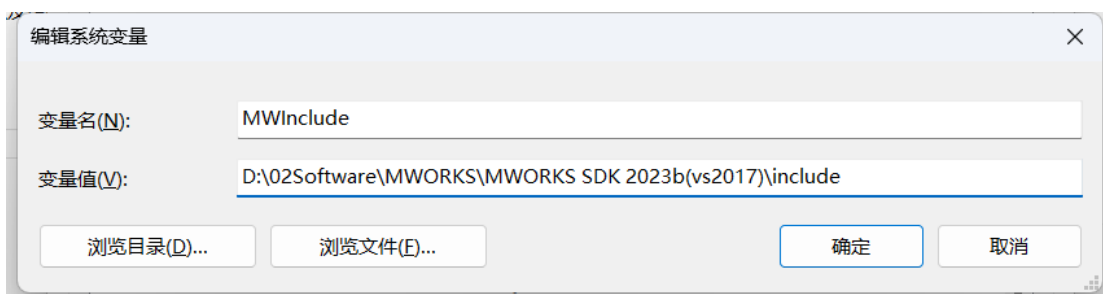


图 4-15 MWInclude 环境变量配置

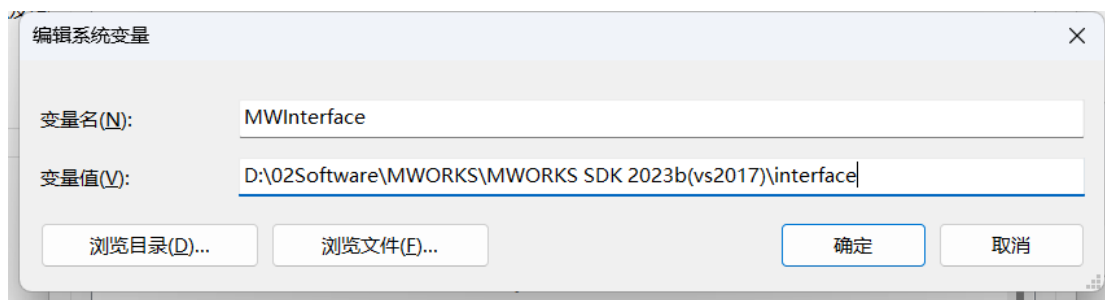


图 4-16 MWInterface 环境变量配置

C. 附加包含目录配置

- 包含目录添加 SDK 的 include 与 interface 目录：

.\GeneratedFiles

.

\$(QTDIR)\include

.\GeneratedFiles\\$(ConfigurationName)

\$(QTDIR)\include\QtCore

\$(QTDIR)\include\QtGui

\$(QTDIR)\include\QtWidgets

\$(MWInclude)

\$(MWInclude)\boost161

\$(MWInterface)

\$(MWInterface)\modelica_services

\$(MWInterface)\common_kits

- MWInclude——SDK 安装路径/include
- MWInterface——SDK 安装路径/interface

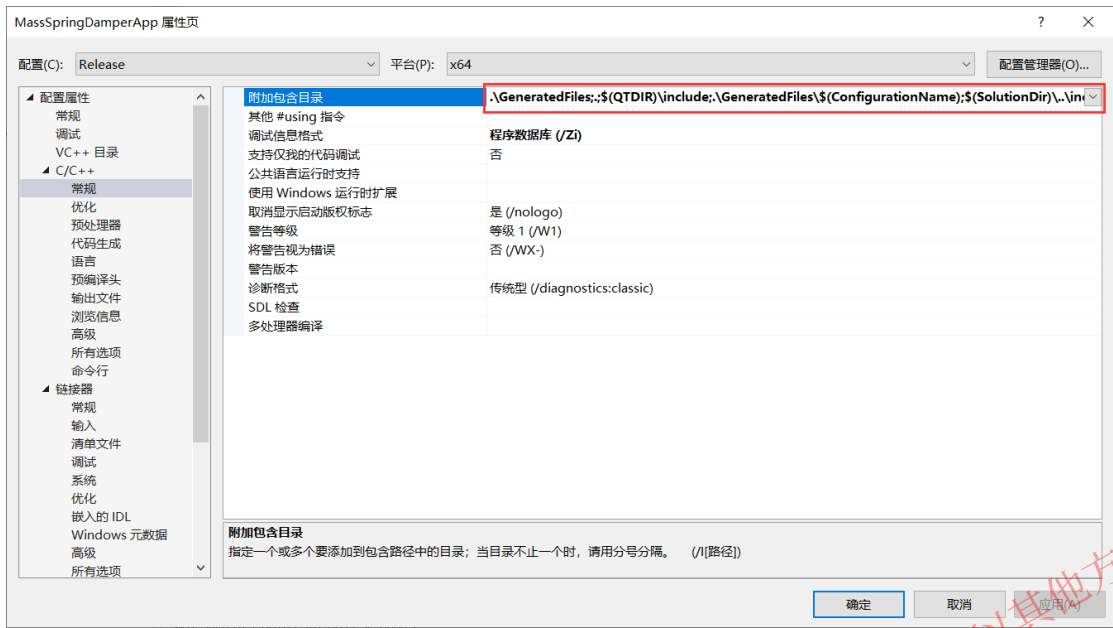


图 4-17 附加包含目录配置

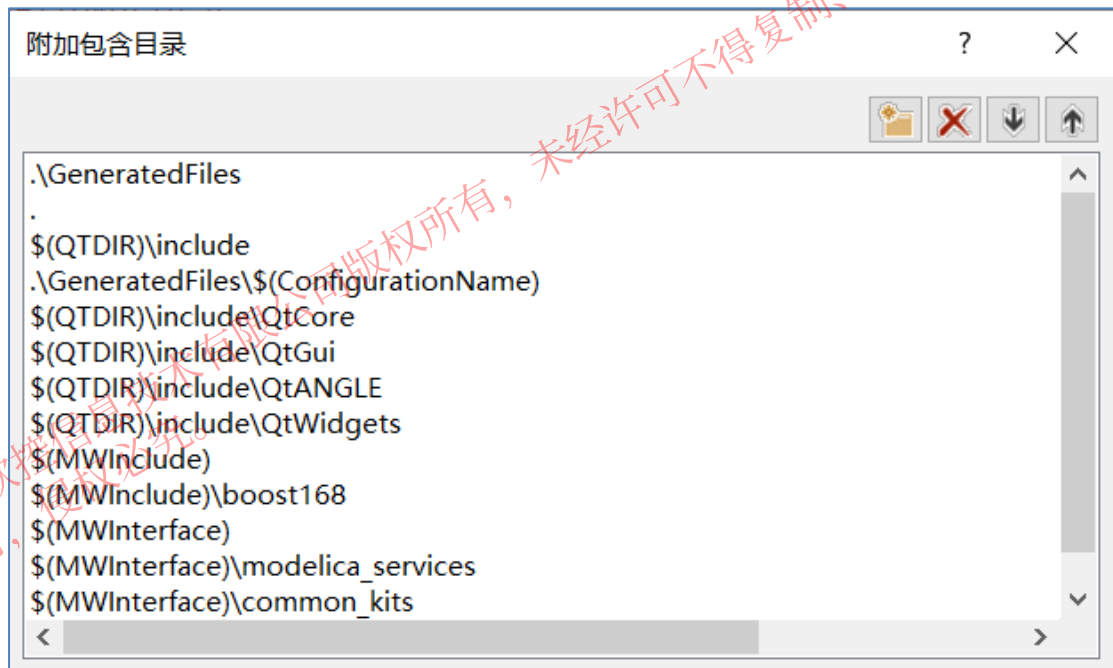


图 4-18 附加包含目录

D. 链接库依赖项配置

- 依赖库目录添加 SDK 的 bin\lib 与文件输出目录
\$(OutDir)
\$(QTDIR)\lib
\$(MWBIn)\lib

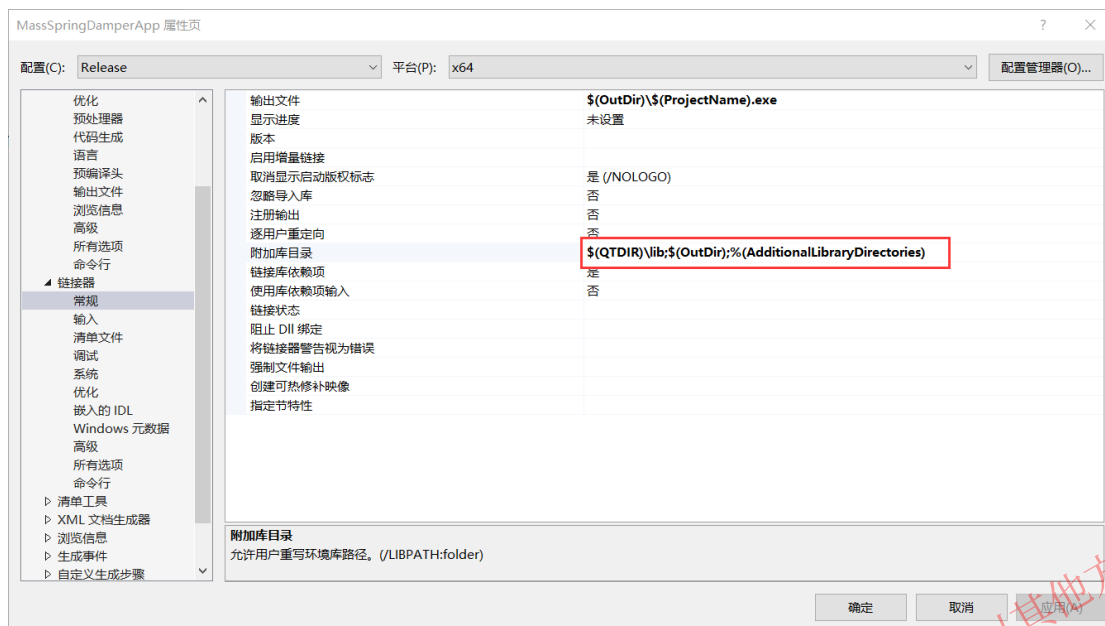


图 4-19 附加依赖项配置

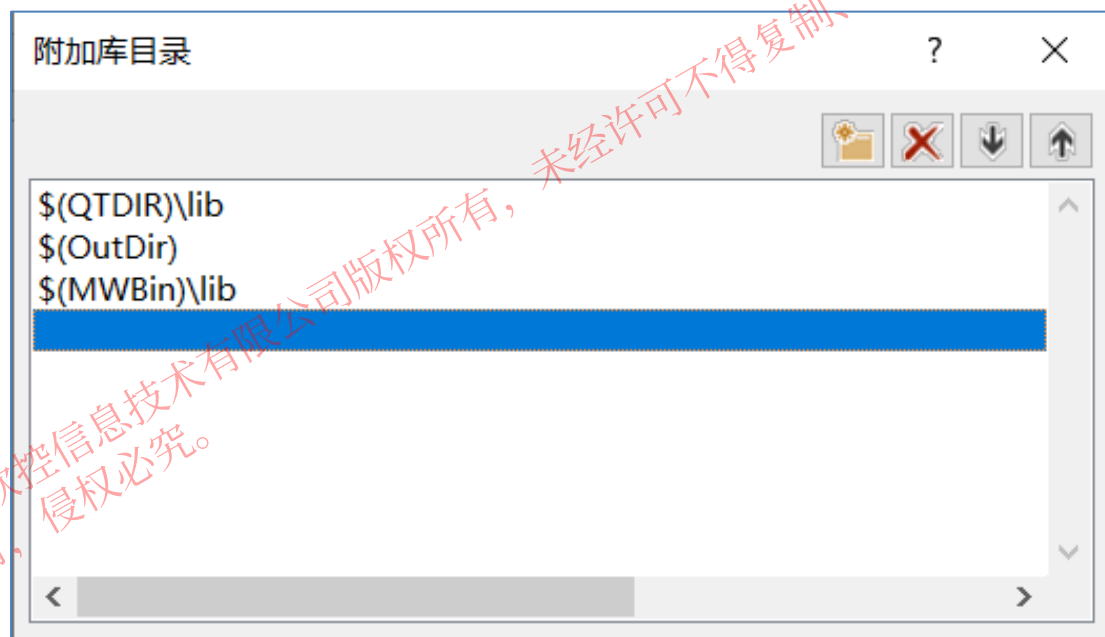


图 4-20 附加库目录

E. 附加依赖项配置

- Debug 下的附加依赖项设置为:

qtmaind.lib

Qt5Cored.lib

Qt5Guid.lib

Qt5Widgets.lib

mw_develop_d.lib

modelica_compiler_d.lib
mw_graphics_view_d.lib
mw_class_manager_d.lib
mw_global_d.lib
mw_help_d.lib
mw_sim_inst_d.lib
mw_sim_plot_d.lib
model_var_tree_d.lib

● Release 下的附件依赖项设置为:

qtmmain.lib
Qt5Core.lib
Qt5Gui.lib
Qt5Widgets.lib
mw_develop.lib
modelica_compiler.lib
mw_graphics_view.lib
mw_class_manager.lib
mw_global.lib
mw_help.lib
mw_sim_inst.lib
mw_sim_plot.lib
model_var_tree.lib

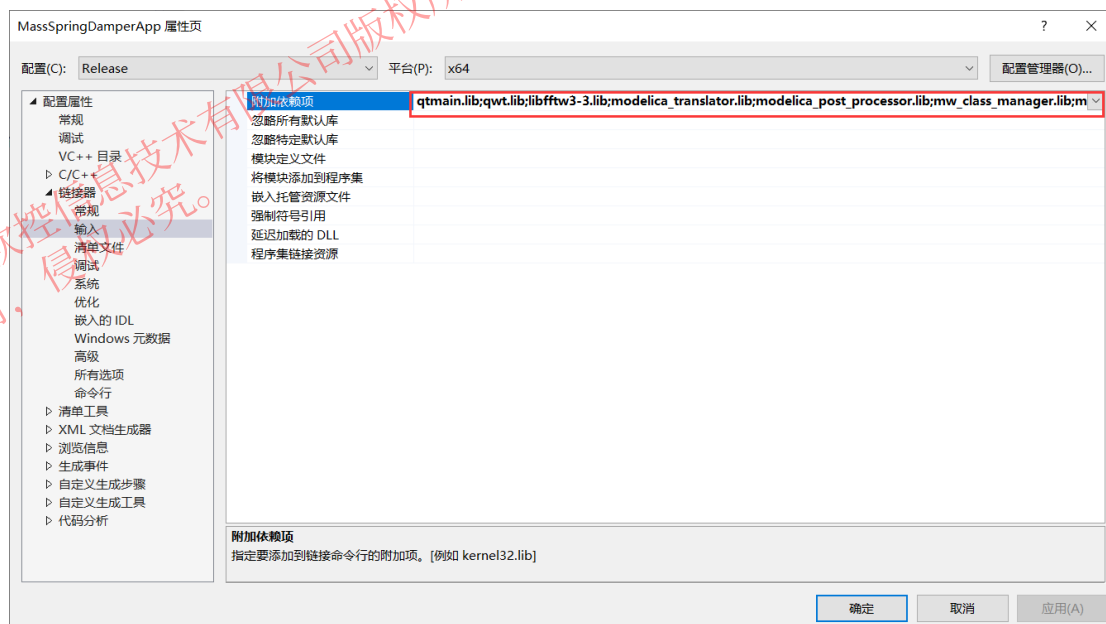


图 4-21 附加依赖项配置

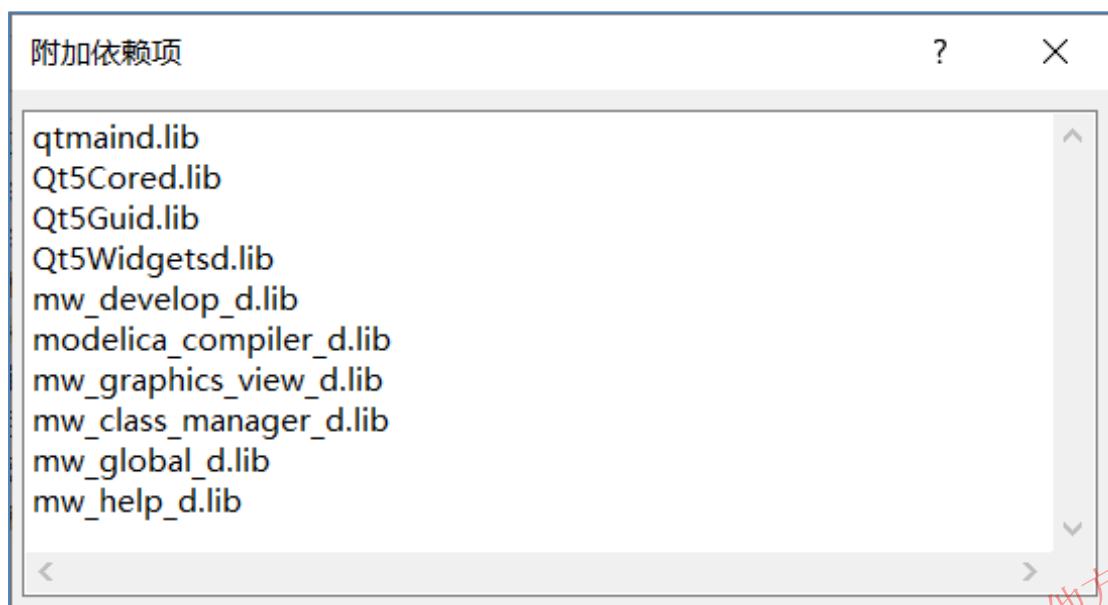


图 4-22 附加依赖项

4.3 内置案例库

SDK 安装完成后，SDK 安装目录下的 `examples` 存在以下案例，使用 Visual Studio 2017 打开 **Examples_vs2017.sln** 可查看并运行对应案例的源码，文件目录如下所示：

名称	修改日期	类型	大小
CompParameter	2023/4/18 14:40	文件夹	
GraphicsViewDropEvent	2023/4/18 14:40	文件夹	
MassSpringDamperApp	2023/4/18 14:41	文件夹	
MwBatchSimPlugin	2023/4/18 14:41	文件夹	
MwVehicleApp	2023/4/18 14:43	文件夹	
StepForward	2023/4/18 14:43	文件夹	
Examples_vs2017.sln	2023/4/17 9:22	Visual Studio 解...	5 KB

图 4-23 example 目录结构

案例中的代码运行环境需满足以下配置：

表 4-1 案例运行环境

类型	环境
操作系统	Win7 SP1 及以上版本
Qt	Qt5.14.2 x86 或 x64 版本

IDE	Microsoft Visual Studio 2017
IDE 插件	VS2017 的 Qt 开发插件
编译环境	Release x64 或 Debug x64

4.3.1 MassSpringDamperApp

质量-弹簧-阻尼仿真模型软件，软件界面如下所示，代码详解请看第 2.1 章节：

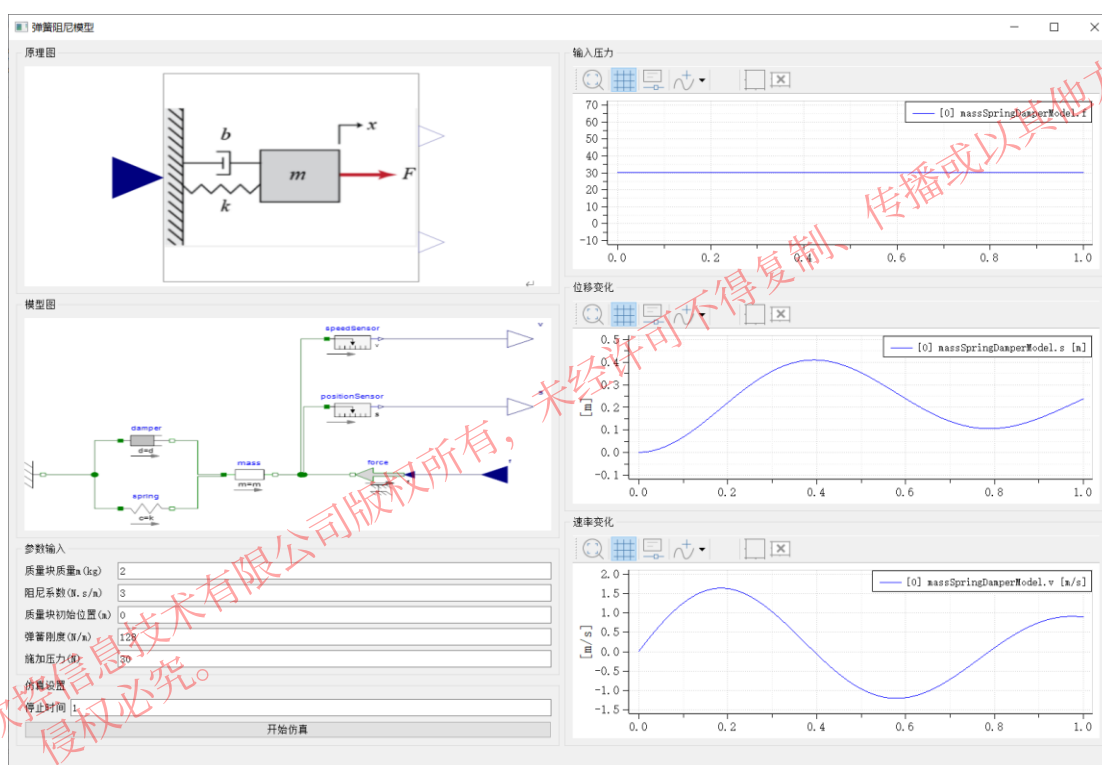


图 4-24 MassSpringDamperApp 运行效果

4.3.2 MwBatchSimPlugin

批量仿真插件开发，软件界面如下所示，代码详解请查看 3 章节：

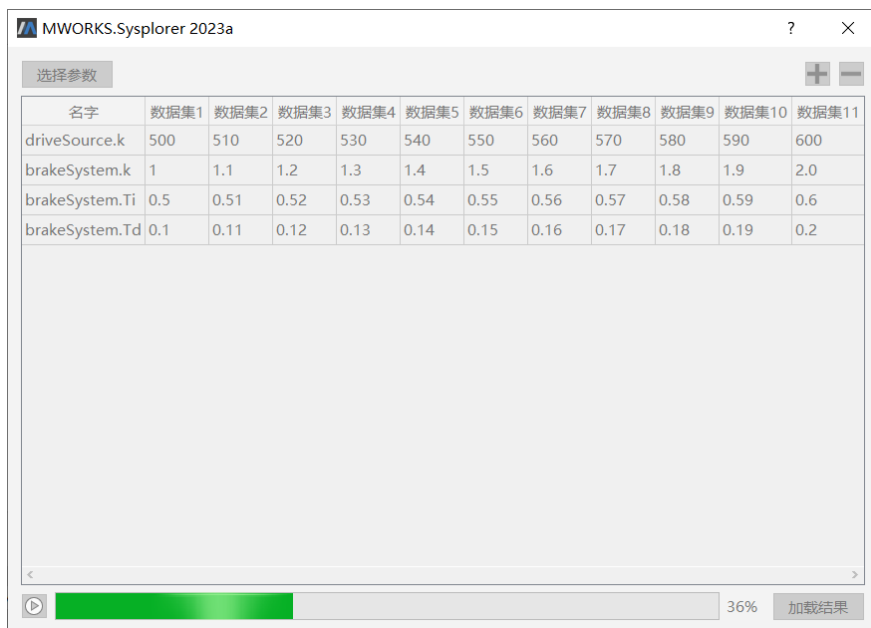


图 4-25 MwBatchSimPlugin 运行效果

4.3.3 MwVehicleApp

车辆综合模型软件，软件界面如下所示，代码详解查看 3.1.2 章节：

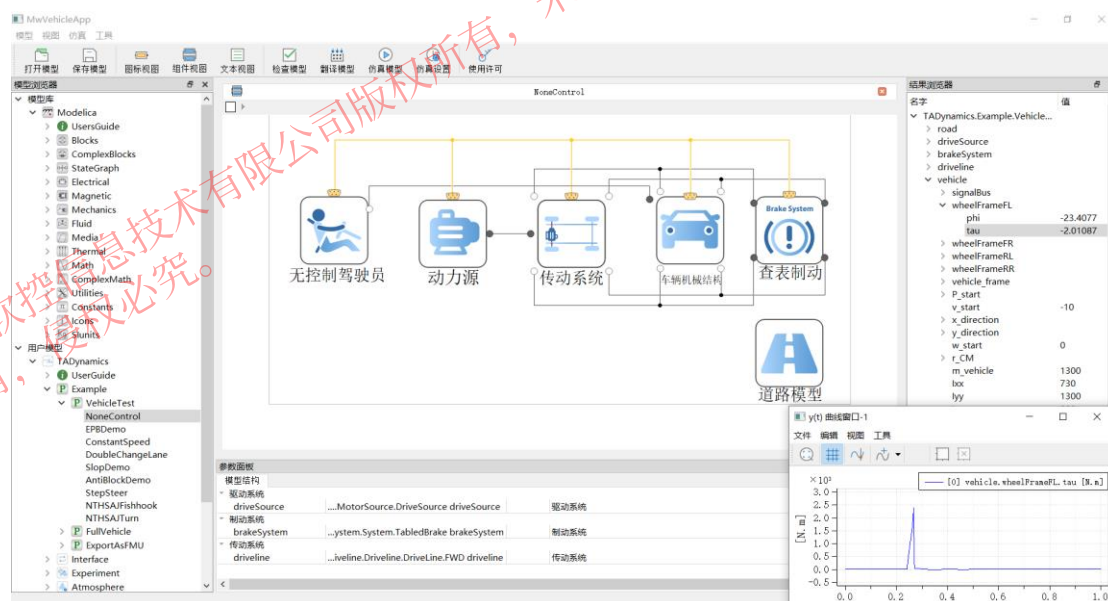


图 4-26 MwVehicleApp 运行效果

4.3.4 Step Forward Demo

本示例主要展示单步仿真，包括：

- 单步模式启动仿真、控制仿真
- 仿真结束显示变量曲线

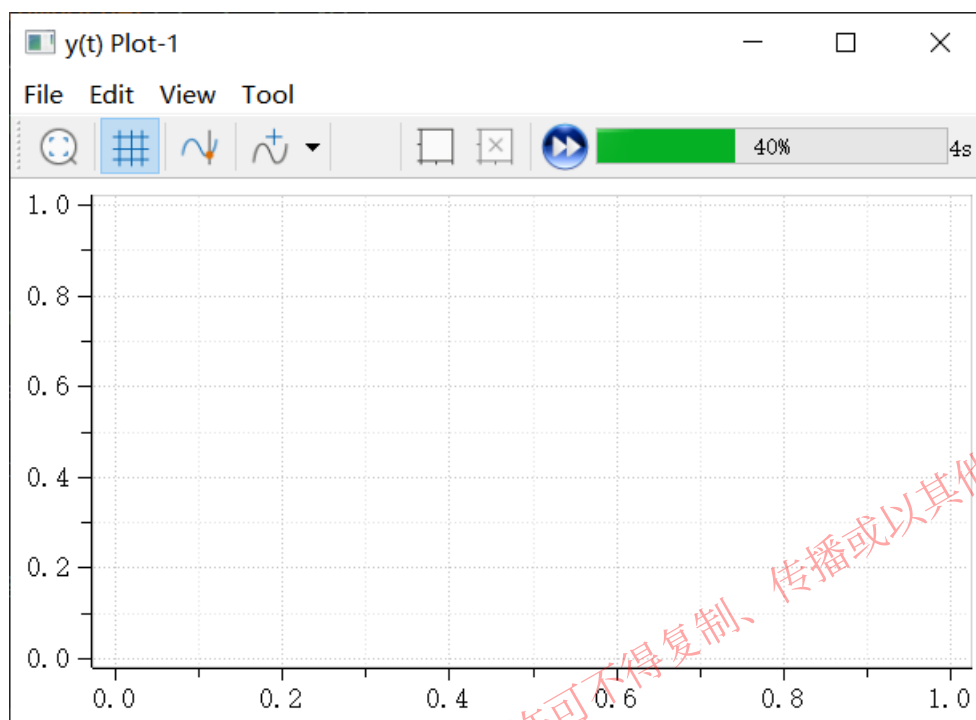


图 4-27 结束显示变量曲线

MWorks SDK 曲线窗口类定义为 [MwSimPlotWindow](#)，本示例 [CustomPlotWindow](#) 继承自该类。

编译 PID_Controller 模型成功后，创建 [MwSimData](#) 并设置仿真模式为 Sim_InteractiveStepMode，然后启动仿真。

```
void CustomPlotWindow::SlotCompileFinish()
{
    compileThread->quit();
    this->setEnabled(true);

    QString sim_dir = QFileInfo(QApplication::applicationDirPath() +
        "\\..\\..\\examples\\StepForward\\Sim").absoluteFilePath();
    QString sim_result_path = QFileInfo(sim_dir + "\\Result.msr").absoluteFilePath();
    simData = new MwSimData(L"PID_Controller", sim_result_path.toStdWString(),
        sim_dir.toStdWString());
    if (!simData->InitializeSimInst())
    {
        simData = nullptr;
        return;
    }
}
```

```

simCtrl->RebindSimData(simData);
MwExperimentData exp_data;
exp_data.stopTime = 10.0;
exp_data.intervalLength = 1;
exp_data.numberOfIntervals = 10;
simData->ApplyExperimentData(&exp_data);
simCtrl->StartSimulate(MwSimControl::Sim_InteractiveStepMode);
progressBar->setValue(0);
labelTime->setText("0.0s");
}

```

单步模式下，每一步都会触发 [SigSimStep](#) 信号，并暂停仿真。需调用 [SimulateNextStep](#) 恢复仿真。

```

void CustomPlotWindow::SlotSimStep(double cur_time)
{
    progressBar->setValue(cur_time * 10);
    labelTime->setText(QString::number(cur_time) + "s");
    this->setEnabled(true);
}

```

仿真结束后，调用 [MwSimPlotWindow](#) 的接口 `AddCurveToCurrentView` 将变量曲线添加到曲线视图中。

```

void CustomPlotWindow::SlotSimStopped(int exit_code)
{
    this->setEnabled(true);
    progressBar->reset();
    labelTime->setText("0.0s");
    this->AddCurveToCurrentView("PI.y", simData);
    actionSimStep->setEnabled(false);
}

```

4.3.5 Component Parameter Panel Demo

本示例主要展示自定义修改组件参数面板，包括：

- 调整参数面板的列顺序
- 组件参数的显示与隐藏

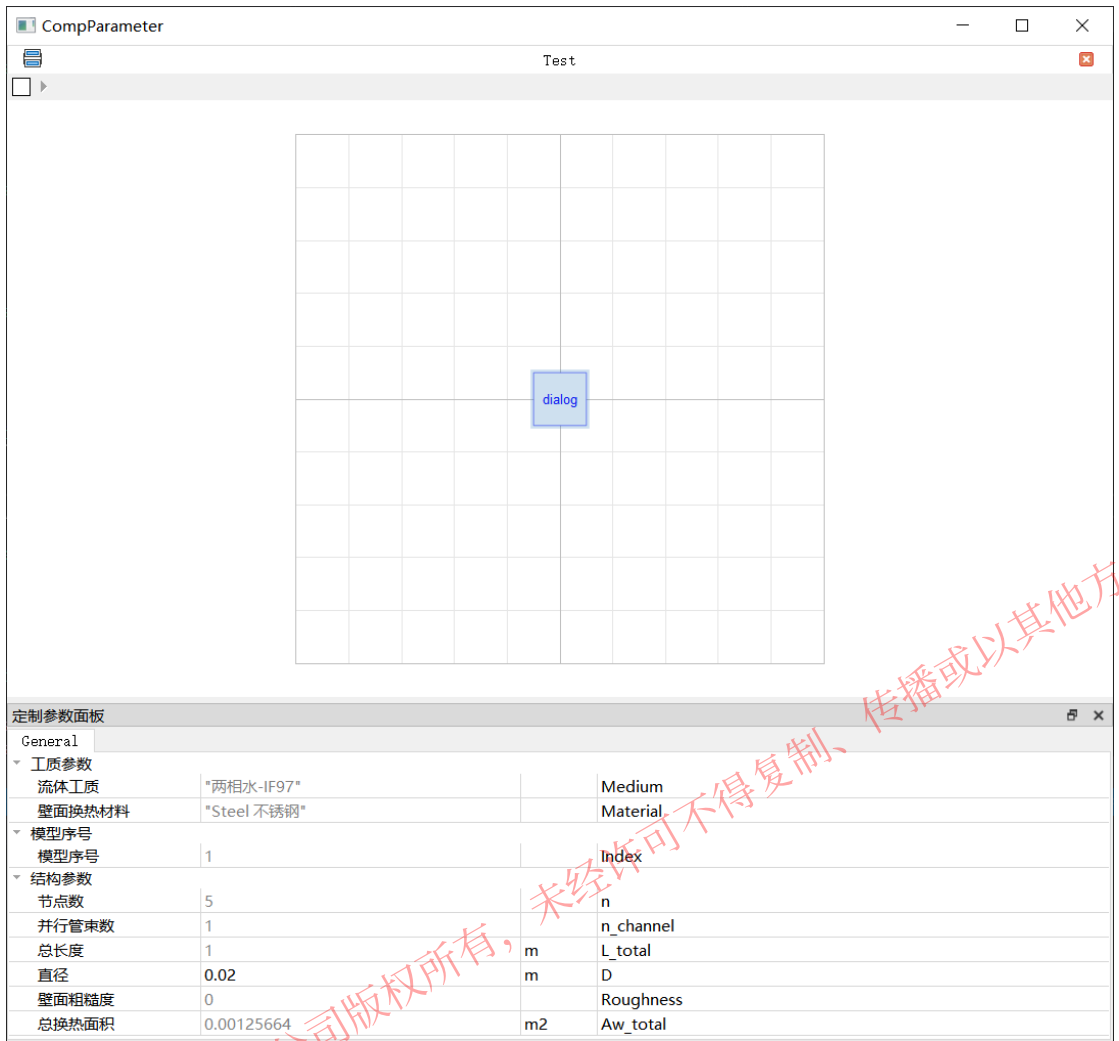


图 4-28 组件参数面板

MWorks SDK 组件参数面板的类定义为 [MwModelParameterTabWidget](#)，本示例 [CustomParameterTabWidget](#) 继承自该类。

MWorks 组件参数面板的默认列顺序为：参数名称、参数值、参数单位、参数说明，本示例将其调整为：参数说明、参数值、参数单位、参数名称。

此外，函数 [IsNeedToDisplay\(\)](#) 让用户决定每个参数的显示或隐藏。系统缺省是每个参数都显示。

```
CustomParameterTabWidget::CustomParameterTabWidget(MwMoGraphicsViewController*
mo_controller,
    MwParamEditMode pem, QWidget* parent)
: MwModelParameterTabWidget(mo_controller, pem, parent)
{
    //调整列顺序
    this->SetColumnOrder(3, 1, 2, 0);
}
```

```
}

CustomParameterTabWidget::~CustomParameterTabWidget()
{
}

bool CustomParameterTabWidget::IsNeedToDisplay(const MwParamInfo& param) const
{
    if (iModelKey == 0)
    {
        return true;
    }
    MwClassManager *class_mgr = MainWindow::GetInstance()->GetClassManager();
    //模型全名
    std::string type_name = class_mgr->GetMoHandler()->GetFullnameProp(iModelKey);

    //隐藏参数集
    QList<std::string> hidden_params;
    hidden_params << "pi";

    if (hidden_params.count(param.ident))
    {
        return false;
    }
    return true;
}
```

4.3.6 GraphicsViewDropEvent

本实例主要展示，将模型树上的组件拖拽到模型视图中：

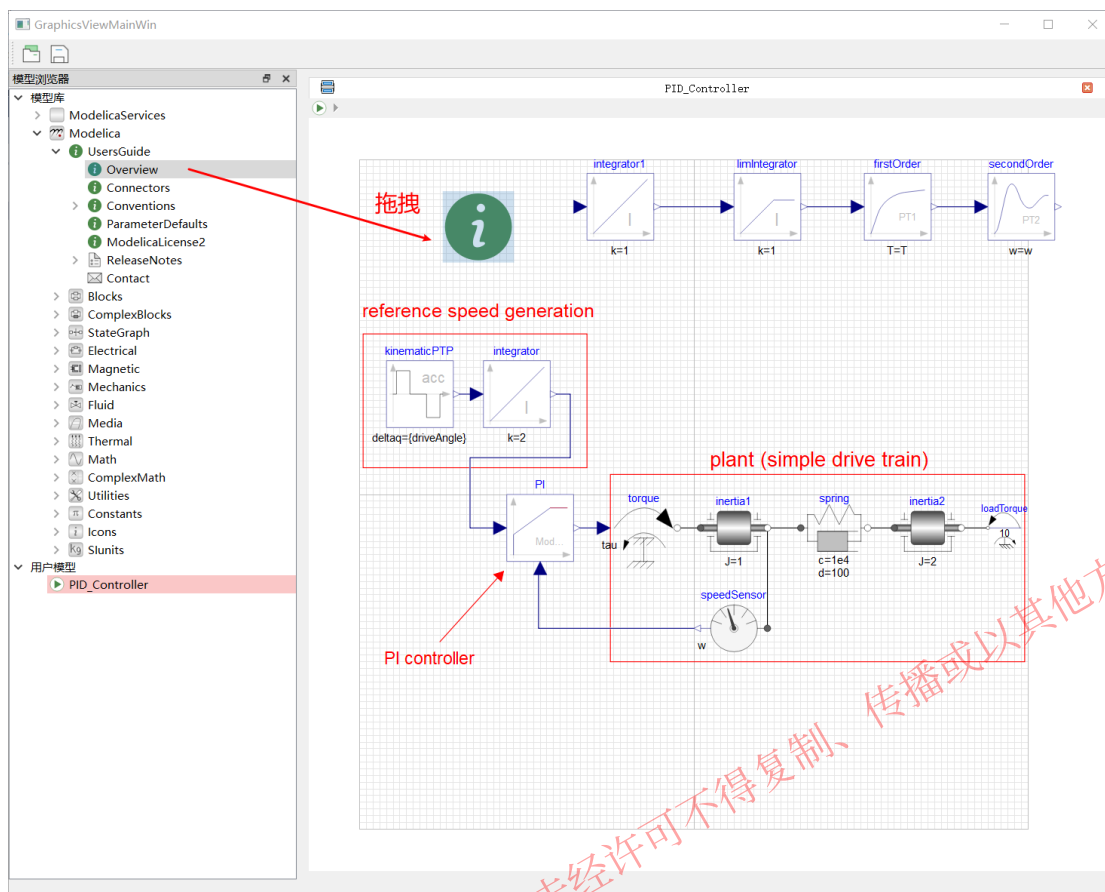


图 4-29 拖拽组件到模型视图

主要通过是在 TreeView 中重写 startDrag 函数，将拖拽的组件全名字符串与 "image/modelica-type" 进行绑定，设置到 QmimeType 数据中，主要代码如下：

```
void ClassBrowserTreeWidget::startDrag(Qt::DropActions supported_actions)
{
    MWInt selected_key = GetSelectedKey();
    if (selected_key == 0)
    {
        return;
    }
    QStandardItem* item = treeModel->itemFromIndex(currentIndex());
    QModelIndexList index_list;
    index_list << currentIndex();
    QMimeType* mime_data = treeModel->mimeType(index_list);
    QByteArray item_data;
    QDataStream dataStream(&item_data, QIODevice::WriteOnly);
    QString selected_name =
    QString::fromStdString(ClassMgrPtr->GetMoHandler()->GetFullnameProp(selected_key));
    dataStream << selected_name;
    mime_data->setData("image/modelica-type", item_data);
}
```

```

QDrag *drag = new QDrag(this);
drag->setMimeData(mime_data);
//Drag 时图标
{
    QIcon icon = item->icon();
    QPixmap pixmap;
    if (icon.isNull())
    {
        //获得默认图标
        pixmap = QPixmap();
    }
    else
    {
        pixmap = icon.pixmap(QSize(50, 50));
    }
    drag->setPixmap(pixmap);
    qreal adjust = 25;
    drag->setHotSpot(QPoint(adjust, adjust));
}
drag->exec(supported_actions);
}

```

4.4 License 配置

为保证大型模型能够正常仿真，及相关功能能够正常使用，再调用 SDK API 前需提前设置 license 文件，SDK 提供 [MwLicenseService](#) 类用于设置相关 license，该类中提供了两种设置方法，通过有 UI 方式弹出 License 设置对话框或无 UI 方式接口配置 License。

MwLicenseService 类初始化：

```

//license
MwLicenseService* pLicenseService = new MwLicenseService(nullptr);
pLicenseService->Initialize();

```

(1) 弹出 License 设置对话框：

```

pLicenseService->StartupLicenseSetDialog();

```

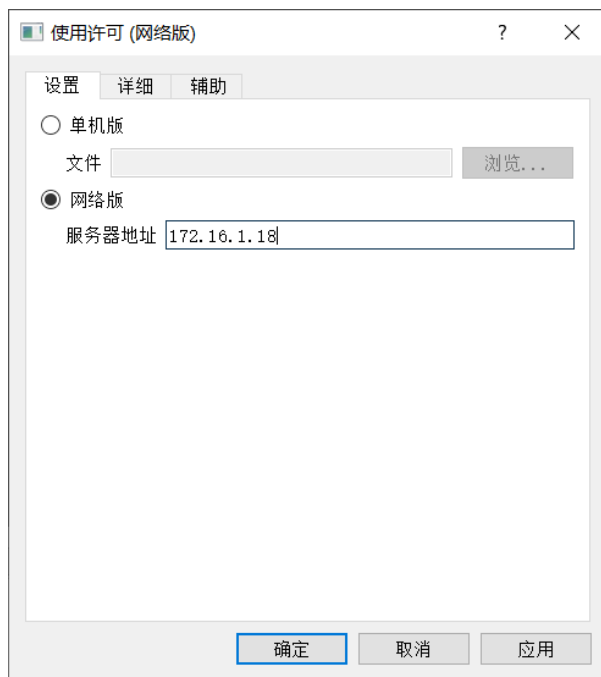


图 4-30 License 配置对话框

- (2) 接口方式配置 License，可以设置单机版 License 文件路径，也可以设置网络版 License 的 IP 地址和端口号：

```
bool load_license_success =  
pLicenseService->SetLicenseFilePath(L"E:\\File\\MWorks.Sysplorer\\License\\mworks_sdk.lic");  
  
bool load_license_success = pLicenseService->SetLicenseIPAddress("172.16.1.18", "27000");
```

4.5 MwGraphicsView 响应机制

4.5.1 模块结构

MwGraphicsView 模块是 Sysplorer.SDK 的图形模块，同时也是核心模块，为系统提供图形化建模和文本建模能力，其结构如下图所示：

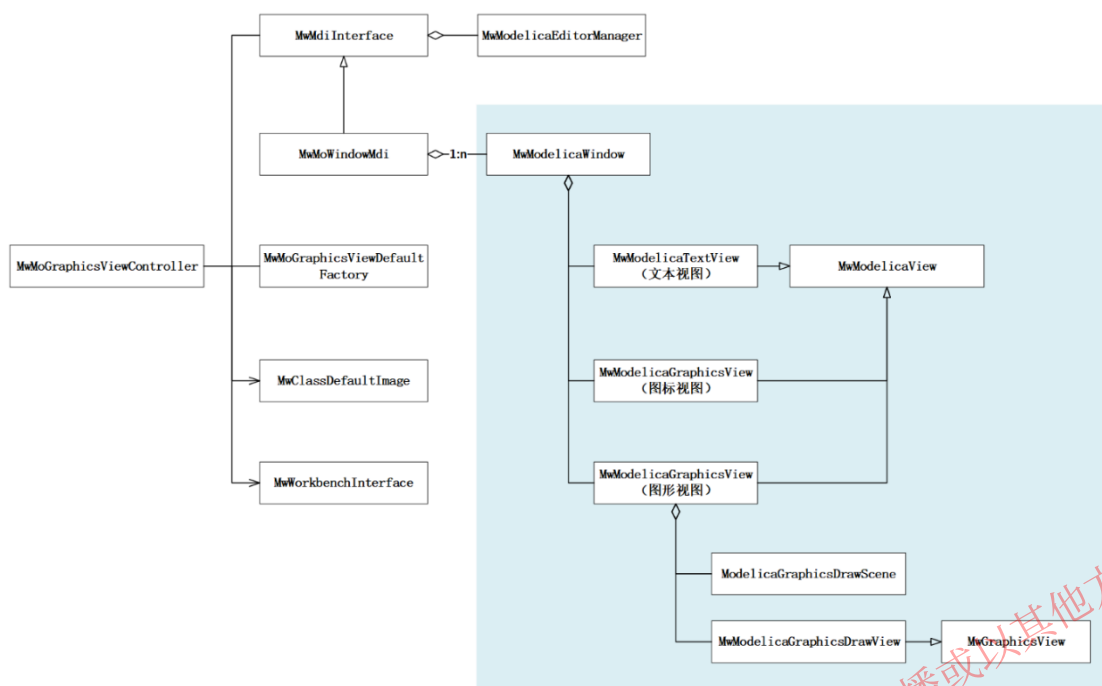


图 4-31 图形模块结构

整个 MwGraphicsView 模块使用 MVC（Model、View、Controller）设计模式。

图形模块中的基本图元充当 MVC 中的“M”（Model），基本实体主要包括：

- 线段
- 矩形
- 椭圆
- 多边形
- 文字
- 图片，支持多种格式 bmp、jpg、png、gif 等

具体结构如下：

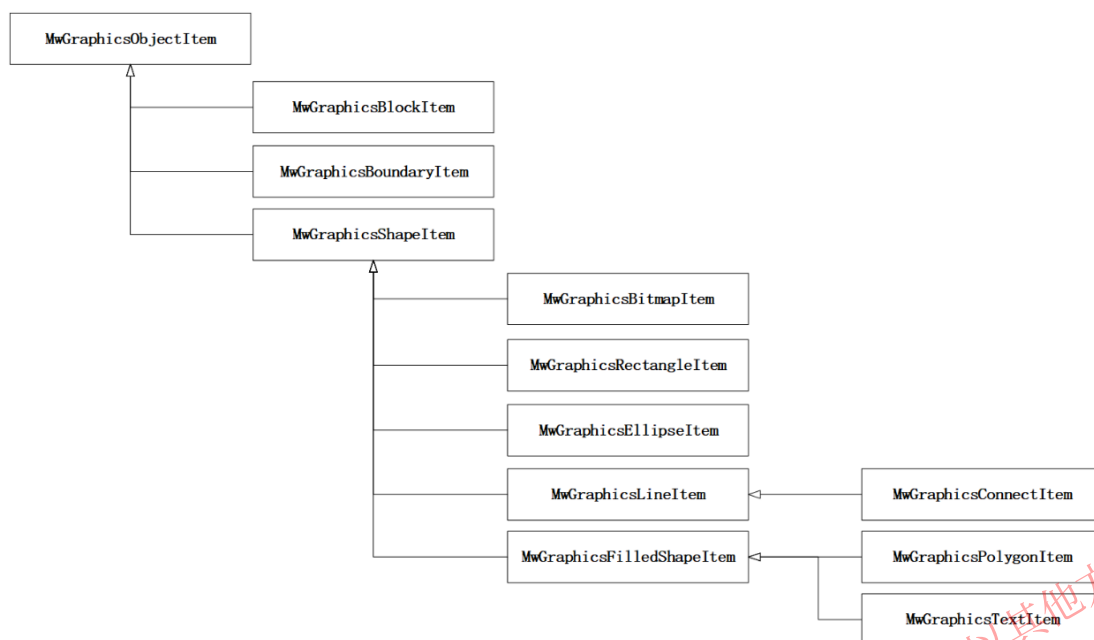


图 4-32 基本图元结构

文本视图、图标视图和图形视图三种视图充当 MVC 中的“V”（View），相同的 Model 在不同 View 中有不同的表现形式，同样的组件在图标视图中表示图标，在组件视图表示组件，在文本视图中表示 Modelica 文本；图形视图（MwModelicaGraphicsView）是系统可视化建模最主要的场所，主要负责显示和编辑模型的组件图形数据：

- 显示图形元素；
- 交互添加图形元素；
- 编辑图形元素：拣选，拖动，旋转，翻转，对齐，调整显示顺序等；
- 进入组件模式，显示组件类型对应的模型视图；

图形视图控制器（MwMoGraphicsViewController）充当 MVC 中的“C”（Controller），主要负责响应底层模型变化，刷新图形视图状态并同步转发模型变化信号，以同步更新界面的其他组件状态。

4.5.2 运行机制

图形视图控制器（MwMoGraphicsViewController）定义了一组视图事件（MoGvEvent），这组事件包含：

- 非法事件：None
- 图元选择变化：SelectChanged
- 视图中组件层次变化：ComponentLevelChanged
- 视图类型变化：ViewChanged

- 模型窗口变化: MoWindowChanged
- 活动窗口变化: ActivateWindowChanged
- 模型数据变化: ModelDataChanged
- 模型添加信号: ModelAppended
- 模型移除信号: ModelRemoved

界面组件一方面通过视图控制器向全局发送视图事件信号,另一方面每个组件自身定义了 SlotUpdate 槽函数并与视图控制器的 SigUpdate 信号连接,各自响应全局视图事件信号。

除了视图事件信号外,图形视图控制器还响应内核模型添加(AppendClassSignals)、模型移除(RemoveClassSignals)、模型替换(ReplaceClassSignals)和模型更新(UpdateClassSignals)信号。响应完毕后同步转发到全局供界面控件各自刷新。

5. 常见问题

使用 SDK 时可能会遇到如下问题:

- (1). 弹出 License 对话框后无法读取之前配置好的 License 信息
 - MwLicenseService 必须在 MwClassManager 初始化之后再进行初始化。
- (2). 创建 MwMoWindowMdi 后,析构时软件崩溃
 - 在析构 MwMoWindowMdi 之前,需要先调用它的 DestroyAllWindows 接口。
- (3). 使用 SDK 提供的控件时,当模型被修改,控件没有刷新
 - 使用 SDK 控件,务必创建 MwMoGraphicsViewController,并将其 SigUpdate 信号和对应 SDK 控件的 SlotUpdate 槽函数相连接。